Memory Virtualization

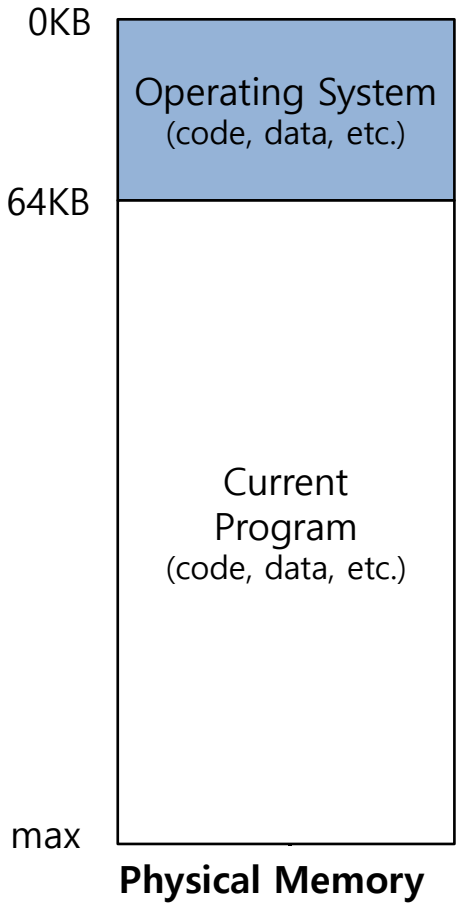
Sungyong Park, Ph.D
Data-Intensive AI Computing and Systems (DISCOS) Laboratory
Department of Computer Science and Engineering, Sogang University
https://discos.sogang.ac.kr

Acknowledgement: Modified from lecture slides made by Prof. Youjip Won (KAIST)
based on the Operating Systems: Three Easy Pieces (OSTEP) textbook

13. The Abstraction: Address Spaces

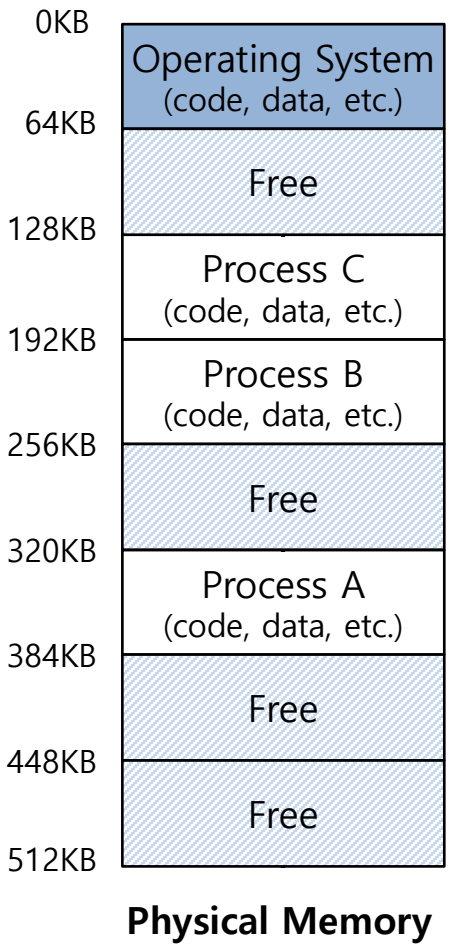
OS in The Early System

- Load only one process in memory.
  - Poor utilization and efficiency



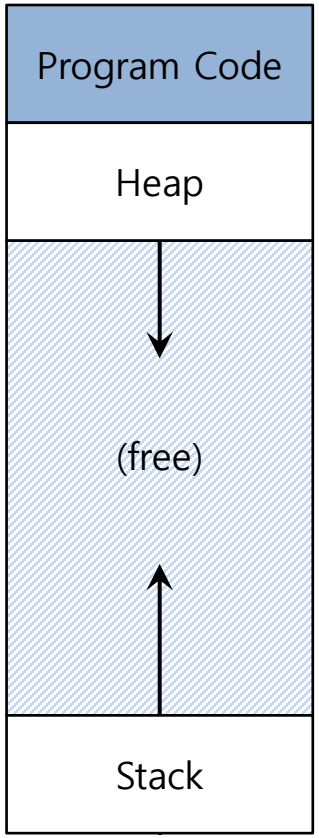
Multi-programming and Multi-tasking

- Multi-programming
  - Load multiple processes in memory.
  - Execute one for a short while and switch processes between them in memory.
  - Increase utilization and efficiency.
- Time-sharing (multitasking)
  - Logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating interactive computing.
- Cause an important protection issue.
  - Errant memory accesses from other processes.



Process Address Space (Virtual Address Space)

- OS creates an abstraction of physical memory.
  - Contains all about a running process.
  - Consists of program code, heap, stack, etc.
    - Code : Where instructions live.
    - Heap : Dynamically allocate memory.
    - Stack : Store return addresses or values; Contain local variables and arguments to routines.



Process Address Space

Virtual Address Space (1)

- Every address in a running program is virtual.
  - OS translates the virtual address to physical address.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]){

    printf("location of code  : %p\n", (void *) main);
    printf("location of heap  : %p\n", (void *) malloc(1));
    int x = 3;
    printf("location of stack : %p\n", (void *) &x);

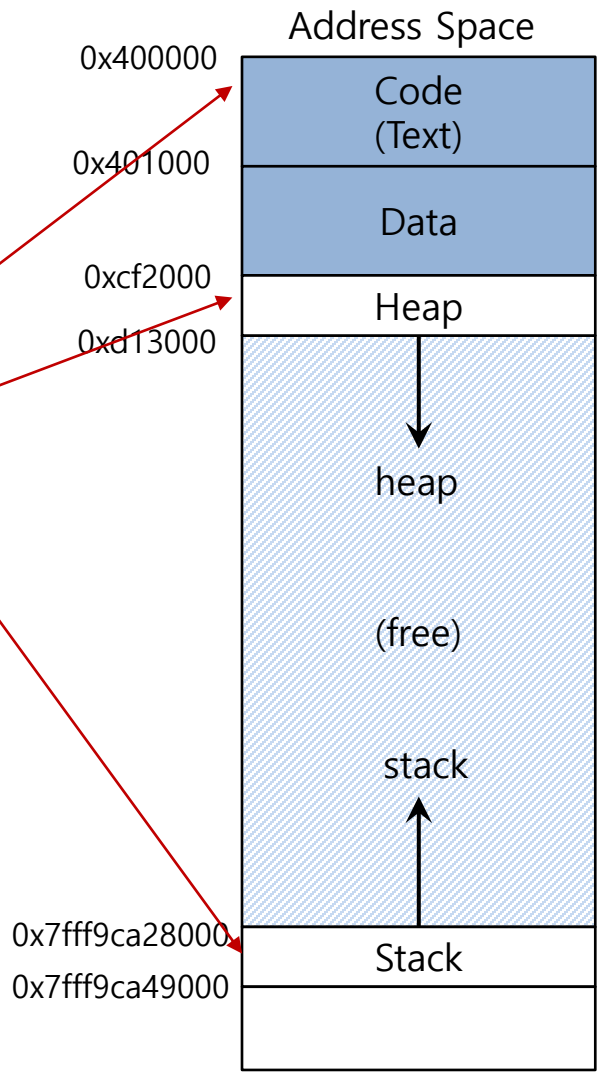
    return x;
}
```

A simple program that prints out addresses

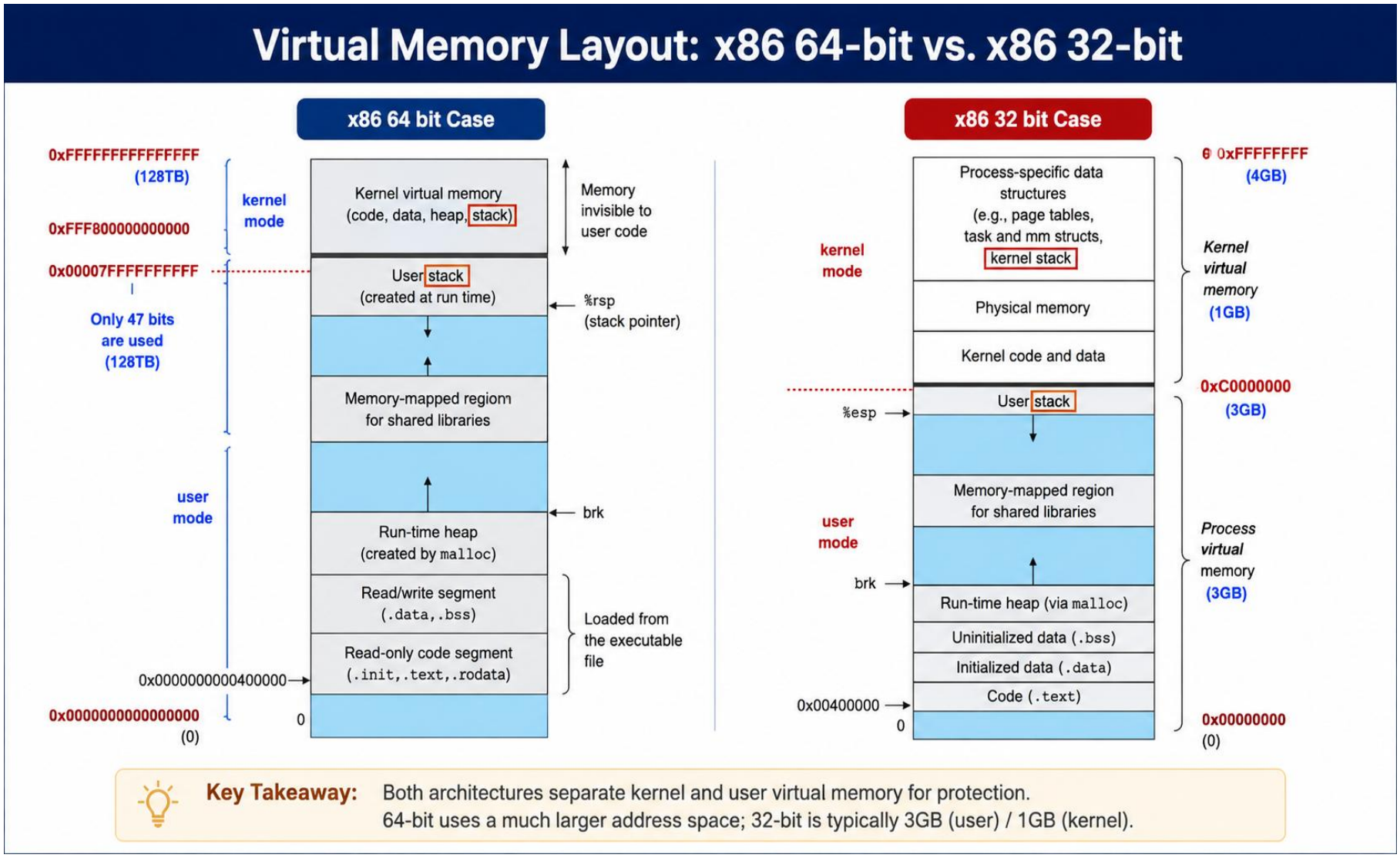
Virtual Address Space (2)

- The output in 64-bit Linux machine

```
location of code  : 0x40057d
location of heap  : 0xcf2010
location of stack : 0x7fff9ca45fcc
```



Example of Virtual Address Space

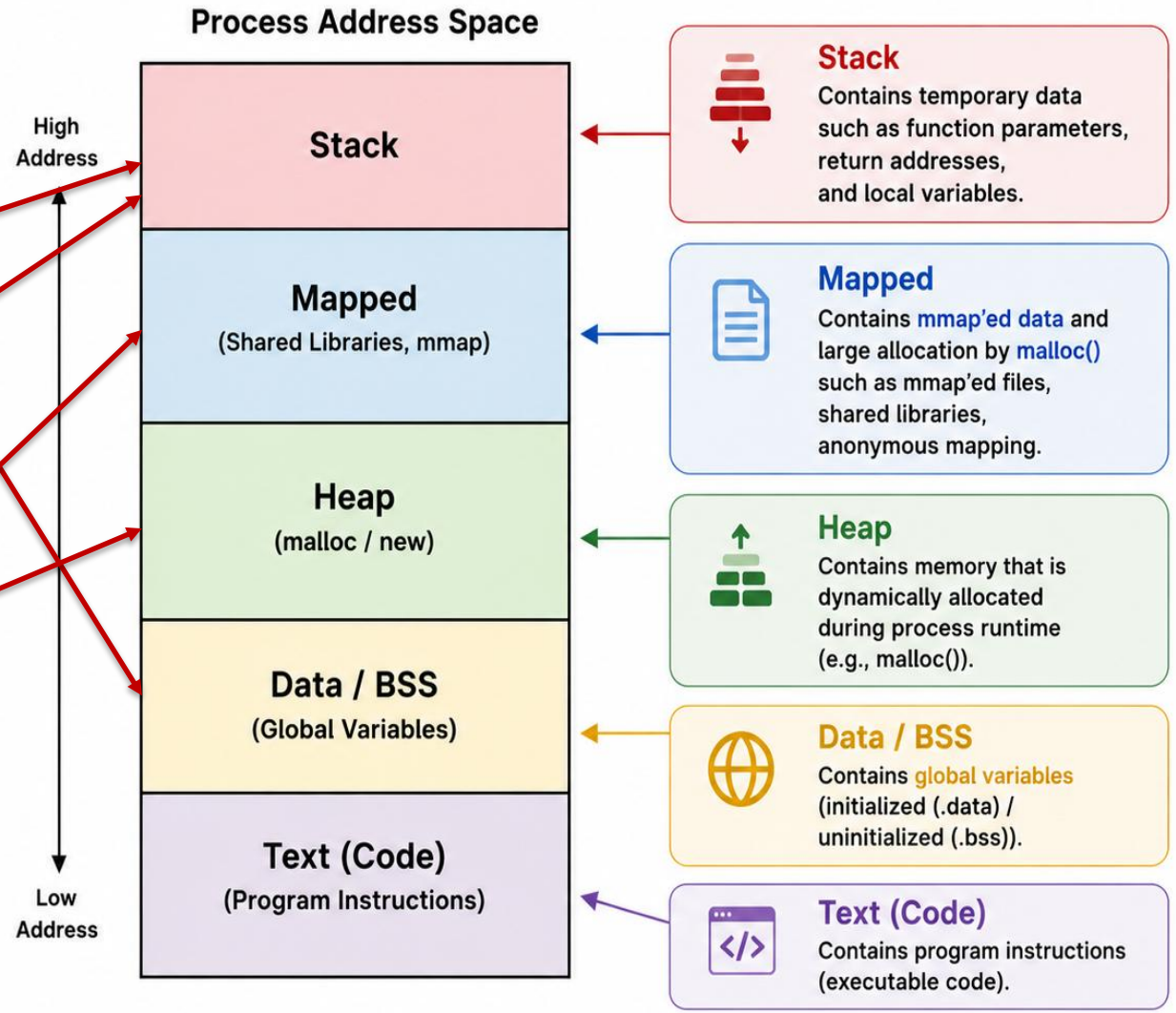


AI-generated illustration using ChatGPT (OpenAI)

Recap: Memory Mapping in a Process

```
int tmp;
int cnt = 0;
int main(void)
{
    int i, j, sum;
    char *heapdata, *mapped;

    i = 10;
    j = 20;
    sum = add(i, j, ++cnt);
    printf("Sum is %d\n", sum);
    mapped = mmap(...);
    heapdata = (char *) malloc(sum);
    ....
}
int add(int k, int l, int m)
{
    int sum;
    sum = k + l + m;
    return sum;
}
```



AI-generated illustration using ChatGPT (OpenAI)

Virtual (Logical) vs. Physical Address Space

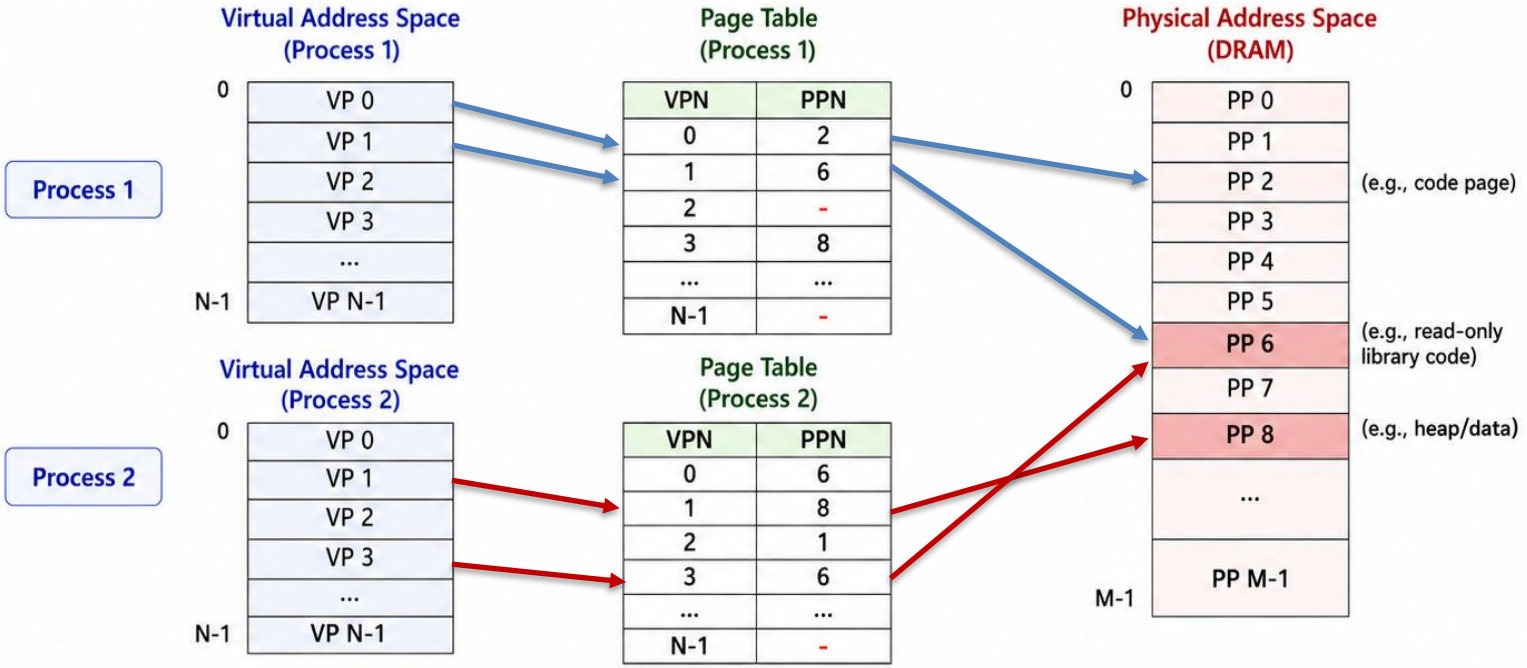
- The user program deals with *virtual (logical)* addresses; it never sees the *real* physical addresses.
  - Virtual address – generated by the CPU; also referred to as *logical address*.
  - Physical address – address seen by the memory unit.

- The concept of a virtual address space that is bound to a separate physical address space (i.e., mapping between virtual address space and physical address space) is central to proper memory virtualization.

- The run-time mapping from virtual to physical addresses is done by a hardware device called the memory management unit (MMU).

Recap: Mapping Between Virtual to Physical Memory

- Virtual/physical address spaces are partitioned into fixed-size pages
- Each virtual page can be mapped to any physical page
- A virtual page can be stored in different physical pages at different times
- Multiple virtual pages can map to the same physical page (e.g., PP 6)



Each process has its own page table.

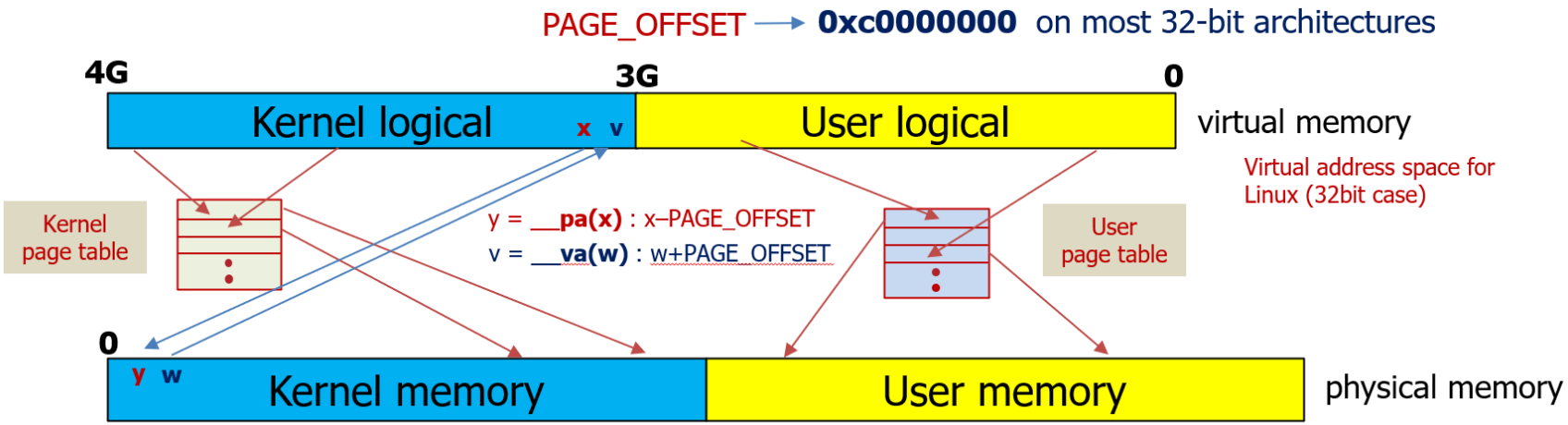
The page table maps virtual pages (VPN) to physical pages (PPN).

Multiple virtual pages (from same or different processes) can map to the same physical page.

AI-generated illustration using ChatGPT (OpenAI)

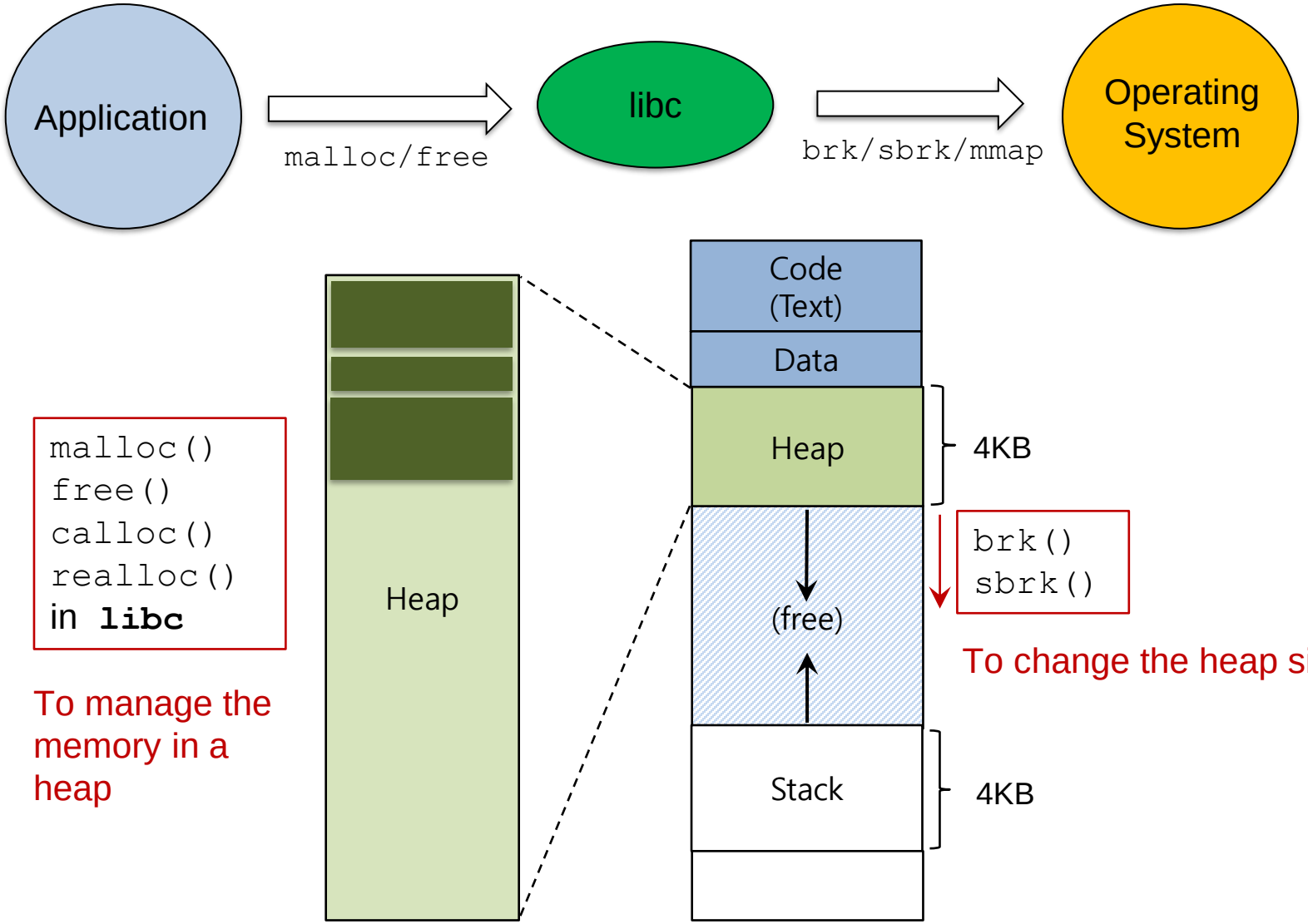
What About Kernel?

- Each OS process has its own virtual address space and part of each virtual address space is reserved for the kernel.
  - In Linux (32 bit case), user : kernel = 3 : 1, but configurable (e.g., 1:1, 1:3, etc.)
- When a process traps into the kernel, kernel uses the mapped space reserved for the kernel (kernel logical memory).
- Although the kernel uses virtual address mapping, most of the kernel memory is mapped **linearly** onto physical addresses.



14. Interlude: Memory API

Overview of Memory APIs



malloc()

```
#include <stdlib.h>

void* malloc(size_t size)
```

- Allocate a memory region on the heap.
  - Argument
    - `size_t size` : size of the memory block (in bytes).
    - `size_t` is an unsigned integer type.
  - Return
    - Success : a void type pointer to the memory block allocated by `malloc`.
    - Fail : null pointer.
- When a process asks for dynamic memory (e.g., through *malloc()*), it *doesn't get additional physical memory*; instead, it gets the right to use a new range of virtual address space.
- Allocate a logically contiguous memory but the mapped physical memory may not be contiguous.

sizeof()

- Routines and macros are utilized for `size` in `malloc` instead typing in a number directly.
- Two types of results of `sizeof` with variables
  - The size of `'x'` is the size of a pointer to an integer (not the allocated area).

```
int *x = malloc(10 * sizeof(int));
printf("%d\n", sizeof(x));
```

```
4
```

8 for 64-bit machine !

  - The actual size of `'x'` is known.

```
int x[10];
printf("%d\n", sizeof(x));
```

```
40
```

free()

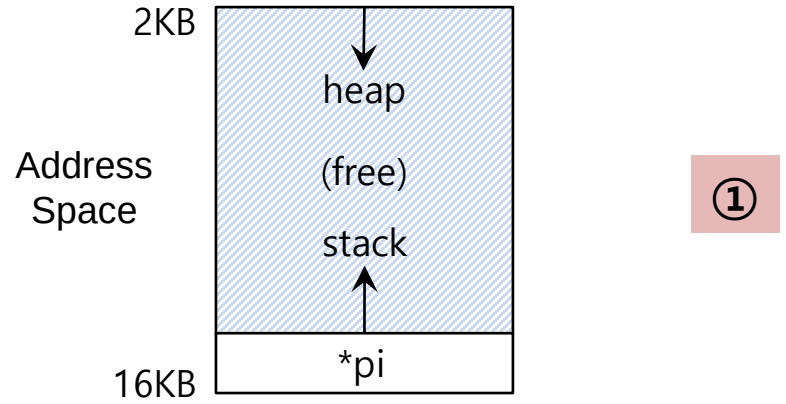
```
#include <stdlib.h>

void free(void* ptr)
```

- Free a memory region allocated by a call to `malloc`.
  - Argument
    - `void *ptr` : a pointer to a memory block allocated with `malloc`
  - Return
    - none

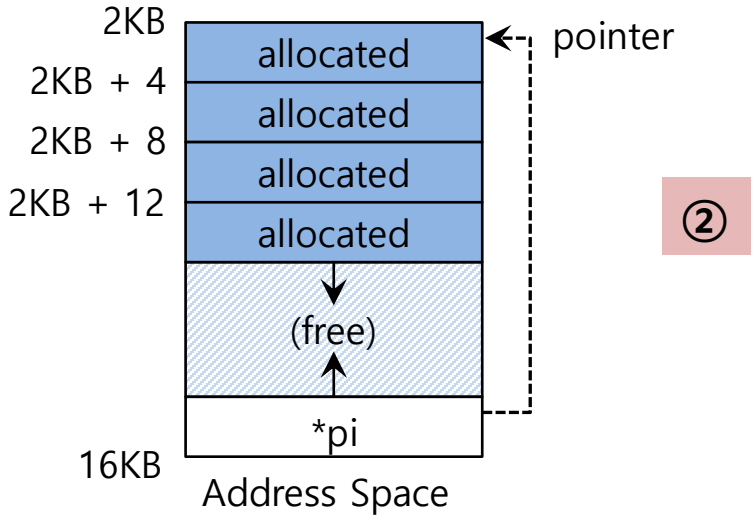
Example: Memory Allocation and Free

```
int *pi; // local variable
```



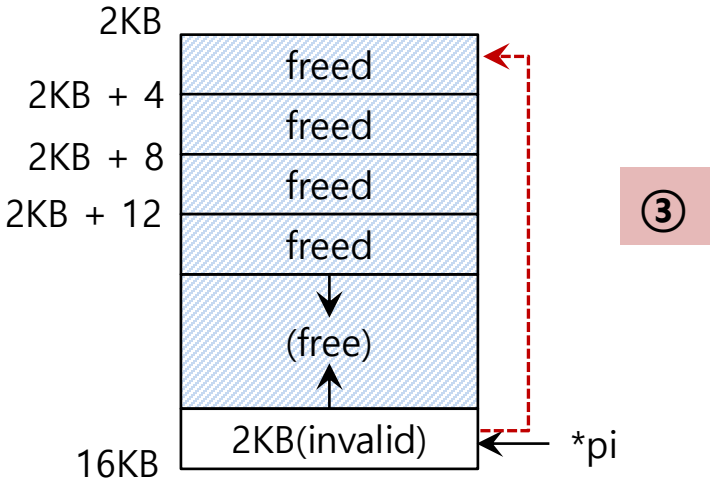
①

```
pi = (int *)malloc(sizeof(int)* 4);
```

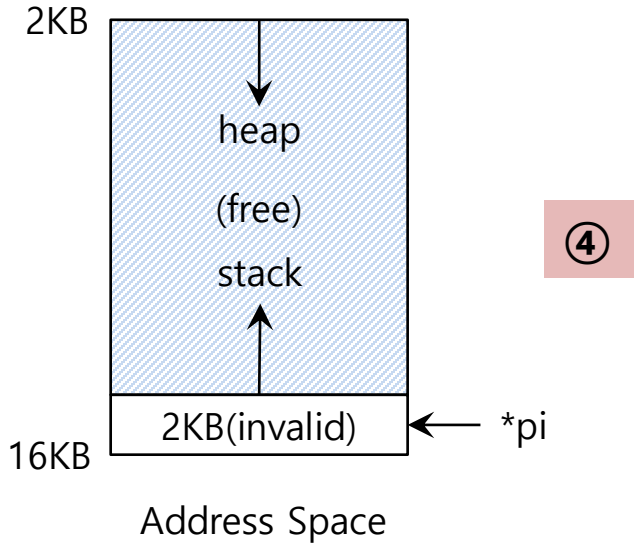


②

```
free(pi);
```



③

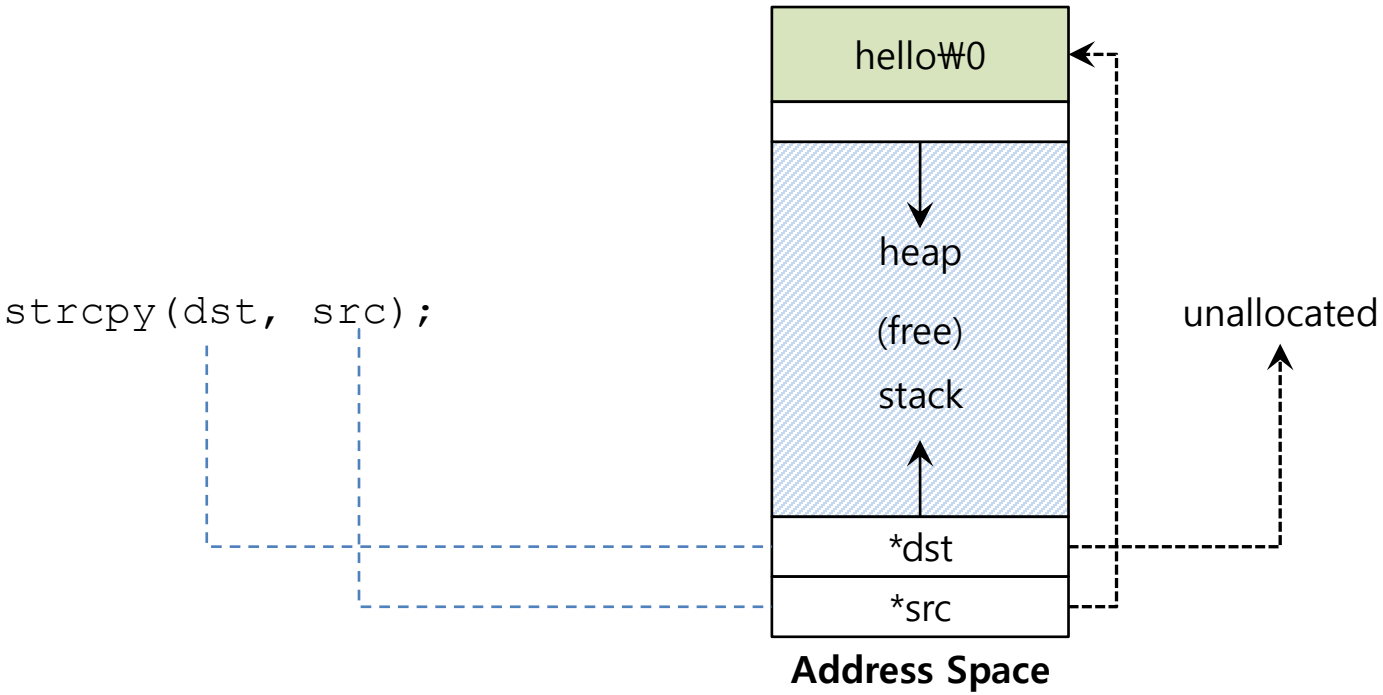


④

Common Error 1: Forgetting To Allocate Memory (1)

- Incorrect code

```
char *src = "hello"; //character string constant
char *dst;           //unallocated
strcpy(dst, src);    //segfault and die
```

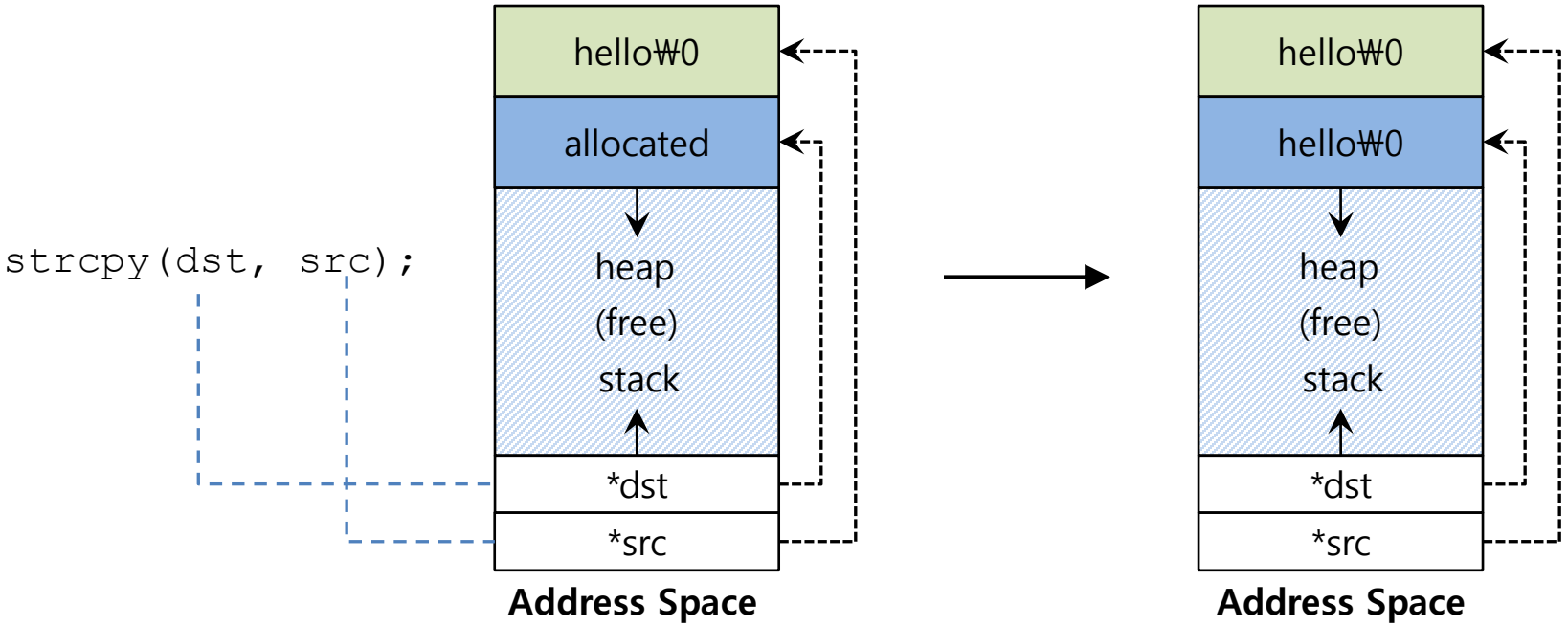


Common Error 1: Forgetting To Allocate Memory (2)

- Correct code

Room for the end-of-string character

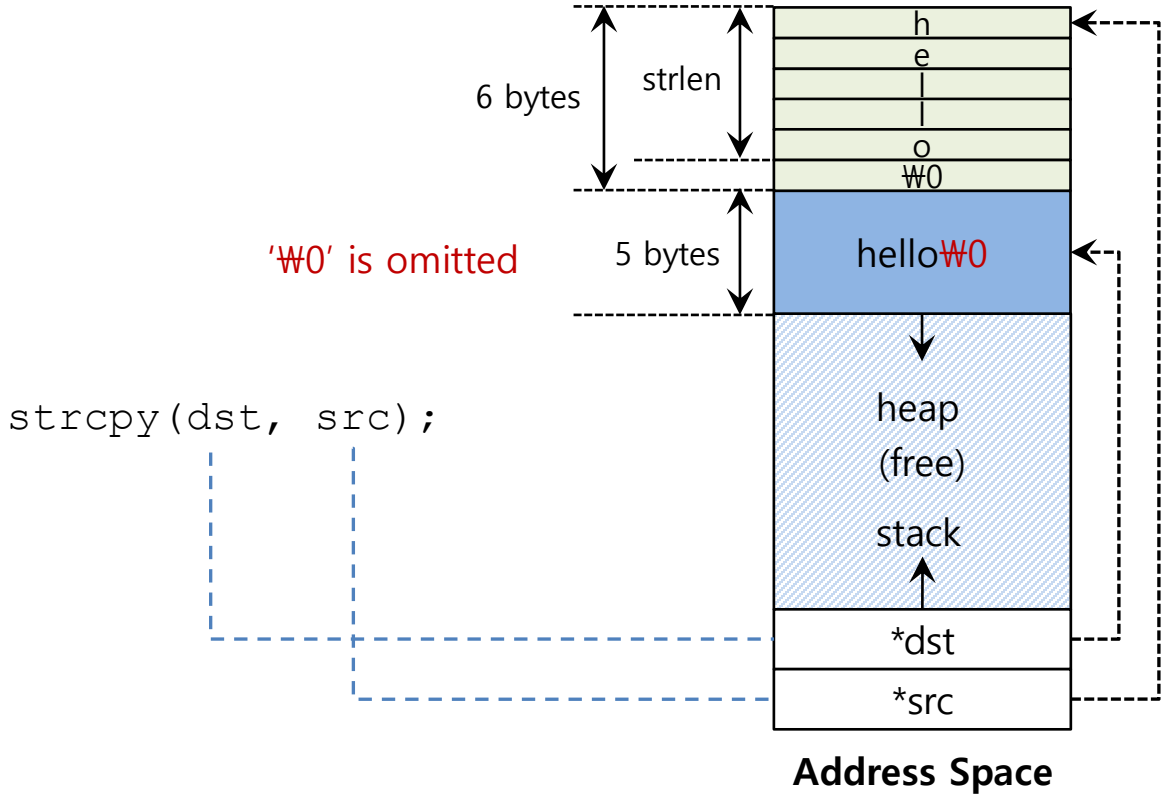
```
char *src = "hello";   //character string constant
char *dst (char *)malloc(strlen(src) + 1 ); // allocated
strcpy(dst, src);      //work properly
```



Common Error 2: Not Allocating Enough Memory

- Incorrect code, but may or may not work properly.

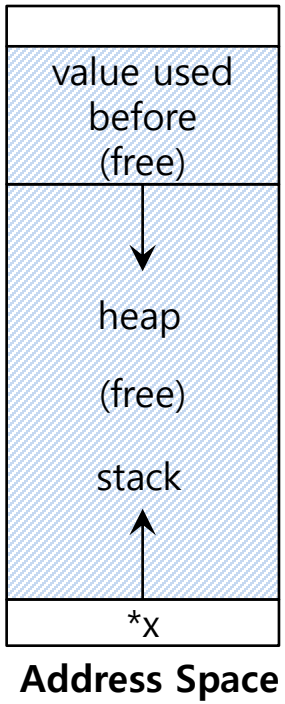
```
char *src = "hello"; //character string constant
char *dst (char *)malloc(strlen(src)); // too small
strcpy(dst, src);    //work properly ?
```

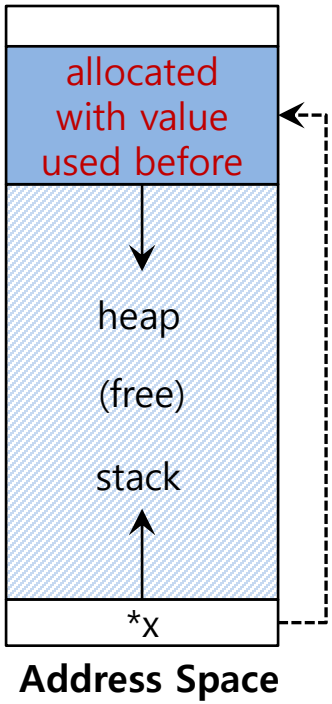


Common Error 3: Forgetting To Initialize Allocated Memory

- Encounter an uninitialized read

```
int *x = (int *)malloc(sizeof(int)); // allocated
printf("*x = %d\n", *x); // uninitialized memory access
```

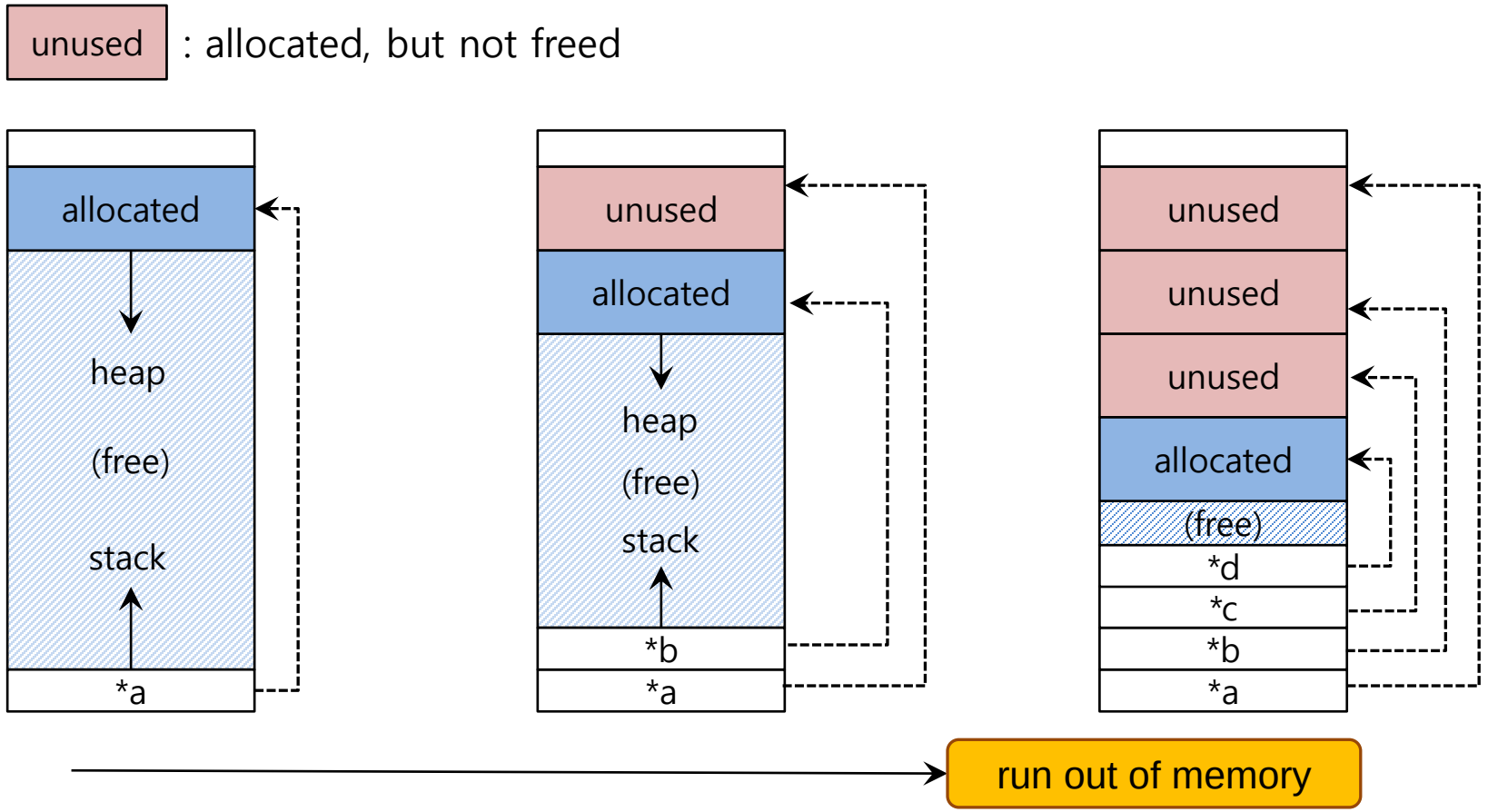




Common Error 4: Forgetting To Free Memory (Memory Leak)

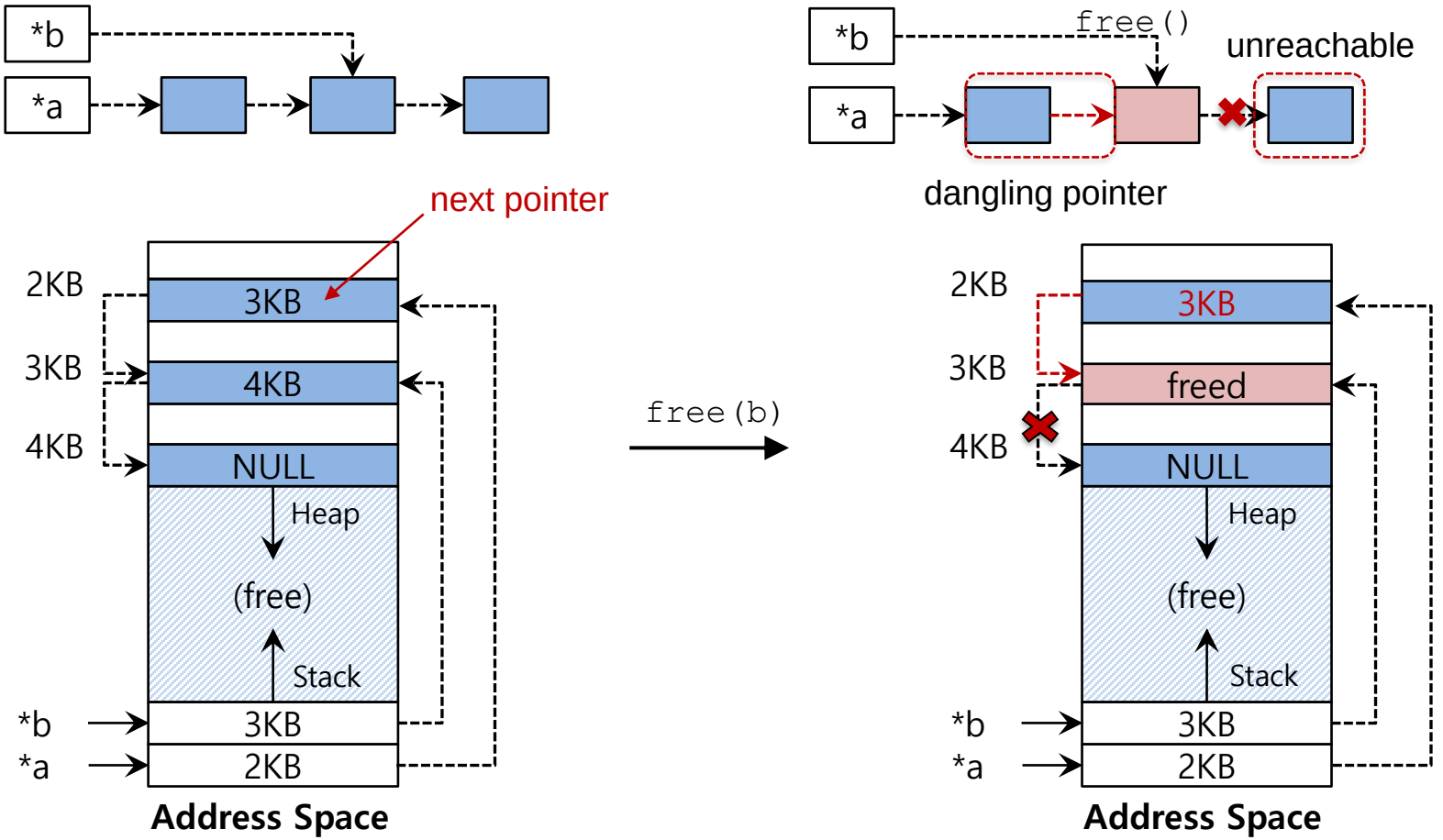
- A program keeps allocating memory without freeing it.
- A program runs out of memory and eventually is killed by OS.

```
while(1) malloc(4) ;
```



Common Error 5: Dangling Pointer

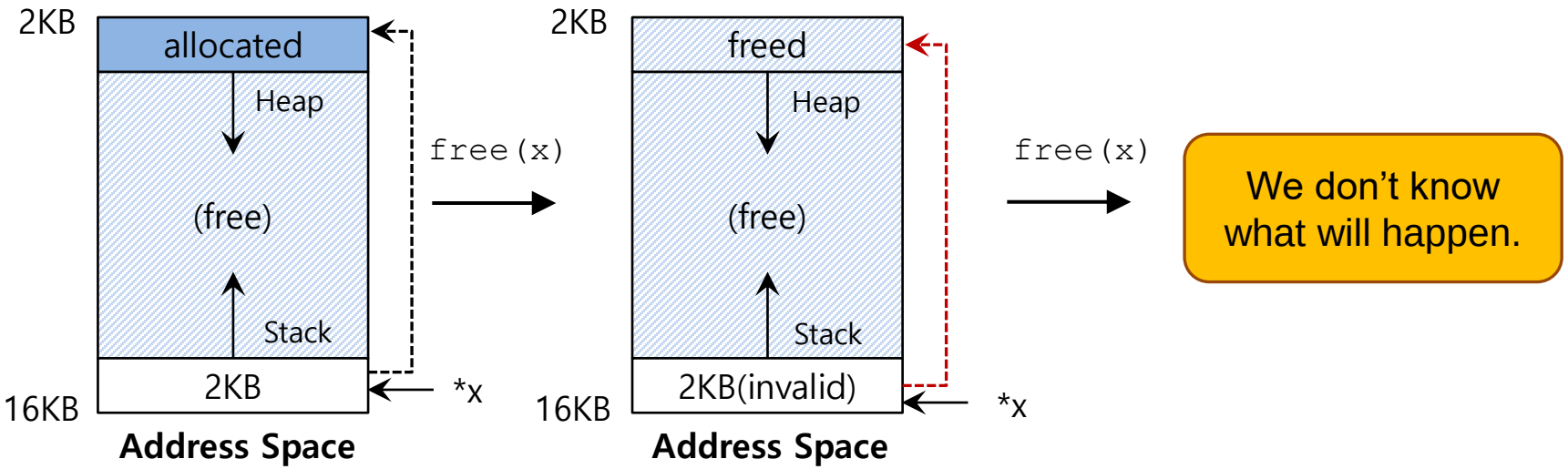
- Freeing memory while it is being used.
  - A program accesses to memory with an invalid pointer.



Common Error 6: Freeing Memory Repeatedly (Double Free)

- Free the memory that was freed already.

```
int *x = (int *)malloc(sizeof(int)); // allocated
free(x); // free memory
free(x); // free repeatedly
```



- Free the memory that was not allocated via `malloc()`.

```
int *x = (int *)malloc(sizeof(int)); // allocated
free(x+12); // free memory
```

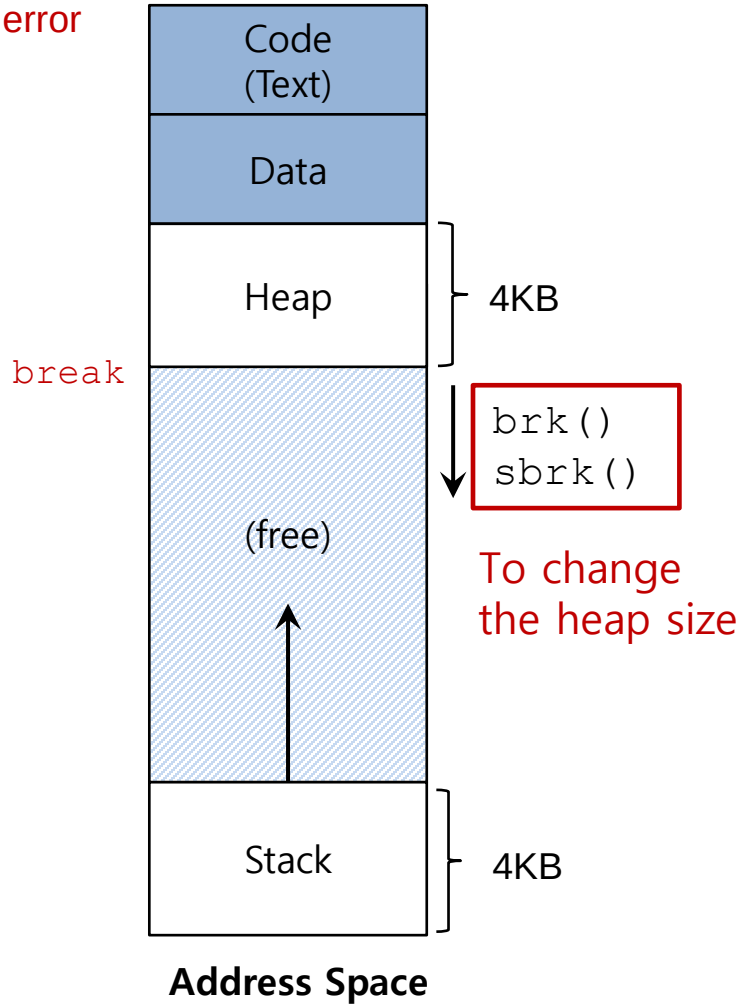
brk()/sbrk()

```
#include <unistd.h>
int brk(void *addr);
void *sbrk(intptr_t increment);
```

Returns 0 if success; -1 if error

Returns previous `break` (i.e., sbrk(0) returns current break)

- Ask OS to expand the heap space.
- `malloc` uses **`brk`** system call.
- `break`: The location of the end of the heap in address space.
- `brk` sets the new `break` to the value specified by `addr`.
- `sbrk` is similar to `brk` but increments or decrements the `break` by `increment` bytes.
- Do not directly call either `brk` or `sbrk`.



Example: brk()/sbrk()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    void *curr_brk, *tmp_brk = NULL;

    printf("Welcome to sbrk example:%d\n", getpid());

    /* sbrk(0) gives current program break location */
    tmp_brk = curr_brk = sbrk(0);
    printf("Program Break Location1:%p\n", curr_brk);

    /* brk(addr) inc/dec program break location */
    brk(curr_brk+4096);

    curr_brk = sbrk(0);
    printf("Program break Location2:%p\n", curr_brk);

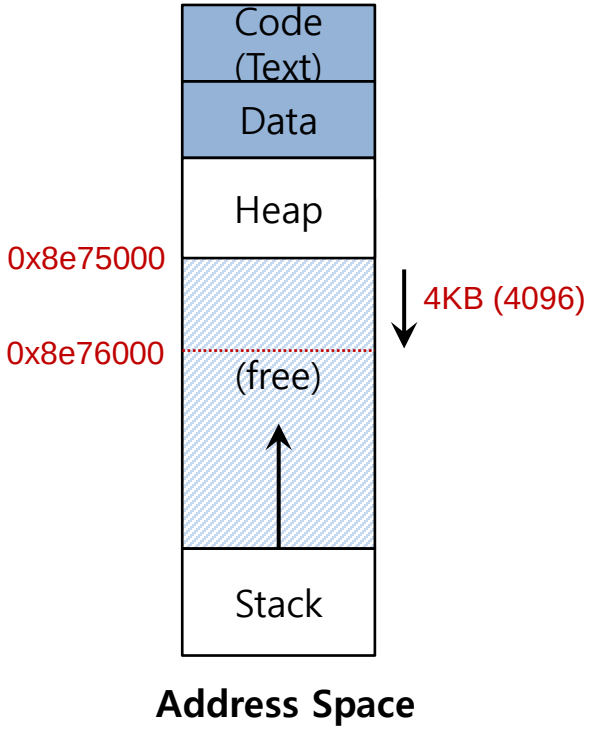
    brk(tmp_brk);
    curr_brk = sbrk(0);
    printf("Program Break Location3:%p\n", curr_brk);

    return 0;
}
```

```
root@ubuntu:~# ./sbrk
Welcome to sbrk example:19100
Program Break Location1:0x8e75000

Program break Location2:0x8e76000

Program Break Location3:0x8e75000
```

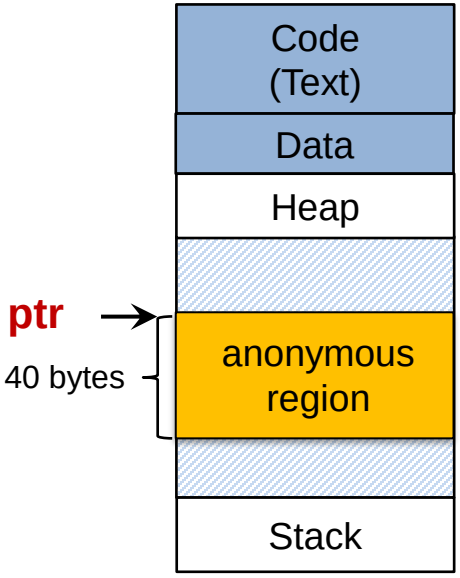


mmap(): Creating Anonymous Mapping Region

```
#include <sys/mman.h>

void *mmap(void *ptr, size_t length, int prot, int flags,
           int fd, off_t offset)
```

- Allocate a memory region of `length` at `ptr` (not in a heap).
- Some `malloc` implementations use `mmap` for allocating large blocks.
- The `fd` argument should be -1 and `offset` should be 0.



Ask OS to choose the mapped address

fd

offset

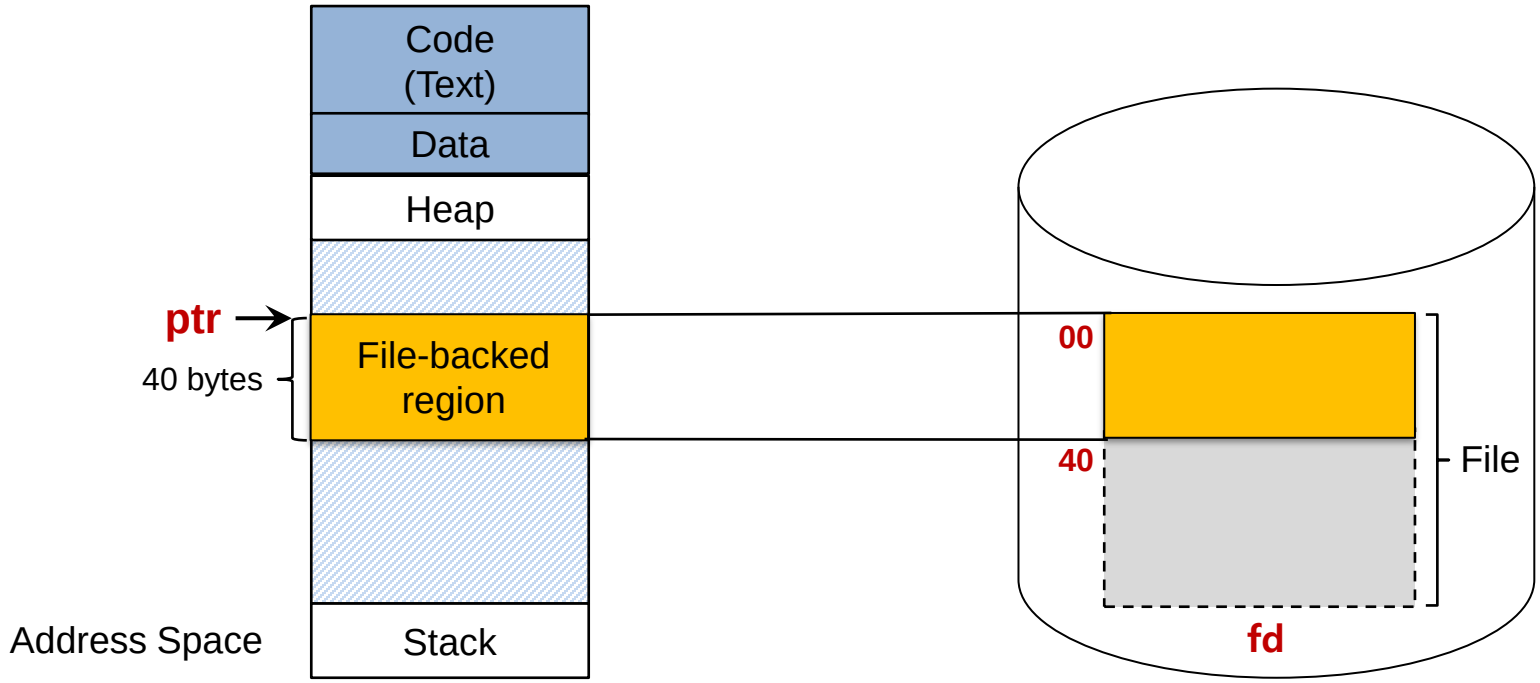
```
ptr = mmap(NULL, 40, PROT_READ |
                     PROT_WRITE, MAP_SHARED |
                     MAP_ANONYMOUS, -1, 0);

if ( ptr == MAP_FAILED ) exit(EXIT_FAILURE);
```

mmap(): Creating File-Backed Mapping Region

```
ptr = mmap(0, 40, flag, MAP_SHARED, fd, 0)) ;
if ( ptr == MAP_FAILED ) exit(EXIT_FAILURE);
```

- Map the contents of a file referred to by the file descriptor `fd` using `length` bytes starting at offset `offset` in the file.
- `offset` must be a multiple of the page size.



Example: *mmapdemo.c*

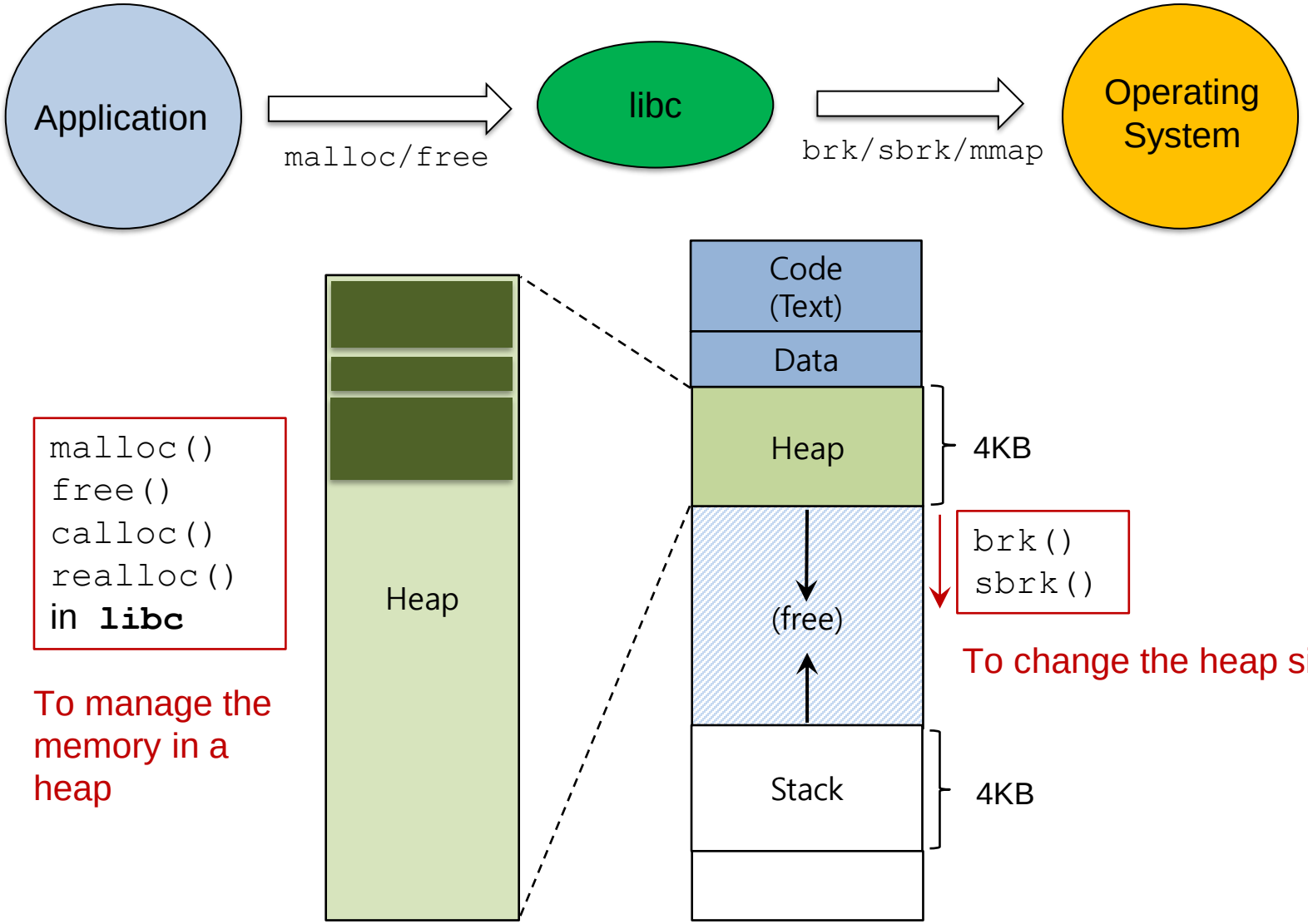
```
#include <stdio.h>
/* include more header files : stdlib.h, fcntl.h, unistd.h, sys/types.h, sys/mmap.h,
sys/stat.h errno.h  */
…
int main(int argc, char *argv[])
{
     int fd, offset;
     char *data;
     struct stat sbuf;
                    get file size
     …
     if ( ( fd = open("mmapdemo.c", O_RDONLY) ) == -1 ) { perror("open"); exit(1); }
     if ( stat("mmapdemo.c", &sbuf) == -1 ) { perror("stat"); exit(1); }
     offset = atoi(argv[1]); /* get offset */
     …
     data = mmap((caddr_t)0, sbuf.st_size, PROT_READ, MAP_SHARED, fd, 0));
     …
     printf("byte at offset %d is '%c'\n", offset, data[offset]);
     return 0;
 }
```

allocate a new range of virtual address space and map it to the process

```
$ mmapdemo 30
byte at offset 30 is e
```

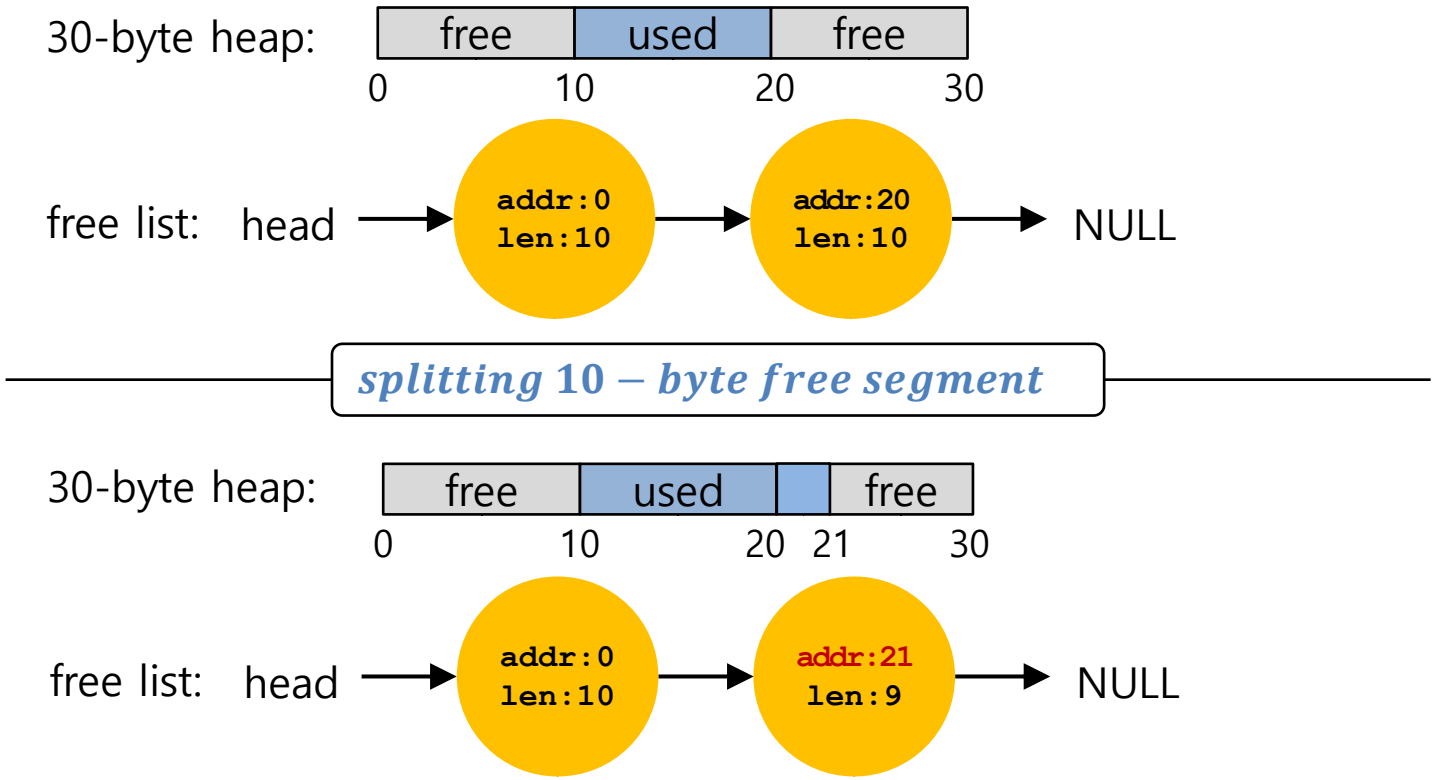
17. Free Space Management

Recap: Heap Management



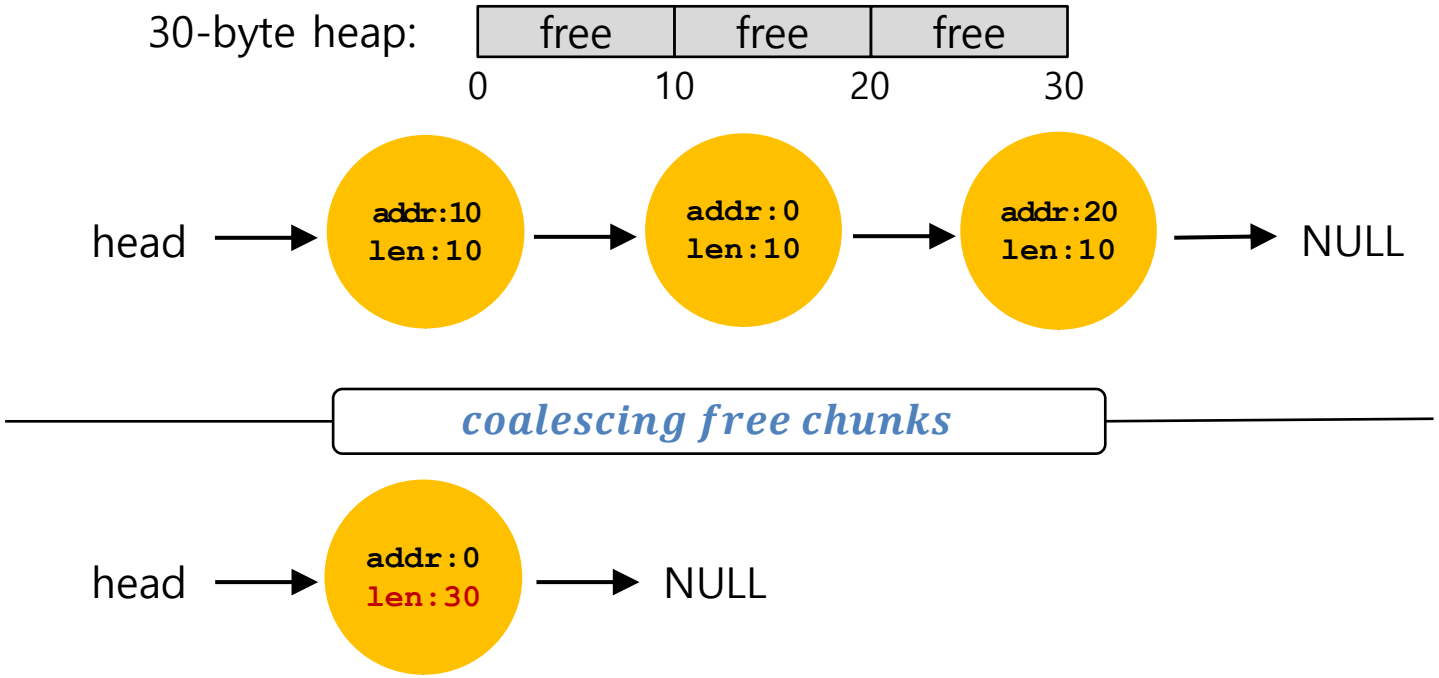
Basic Mechanism: Splitting

- Finding a free chunk of memory that can satisfy the request and splitting it into two.
- Example: Two 10-byte free segments with 1-byte request (in general, the minimum size is the word size – round size up to the word size).



Basic Mechanism: Coalescing

- If a user requests memory that is bigger than free chunk size, the list will not find such a free chunk.
- Coalescing: **Merge** returning a free chunk with existing chunks into a large single free chunk if addresses of them are **nearby**.

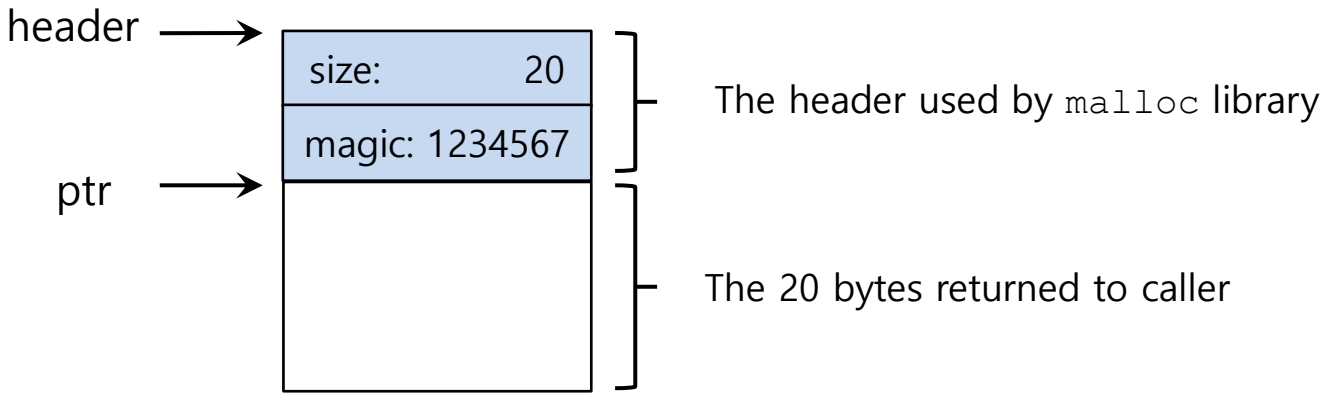


Tracking Allocated Regions (1)

- Store a little bit of extra information in a header block, usually **just before** the handed-out chunk of memory.
  - Each header consists of the size of the allocated region and a magic number to provide integrity checking when freeing the region (magic number should always be the same).

```
typedef struct __header_t {
        int size;
        int magic;
} header_t;
```

  - Example: `ptr = malloc(20);`



An Allocated Region Plus Header

Actual chuck size of malloc(N) = N + size of header. Here, 28 bytes !

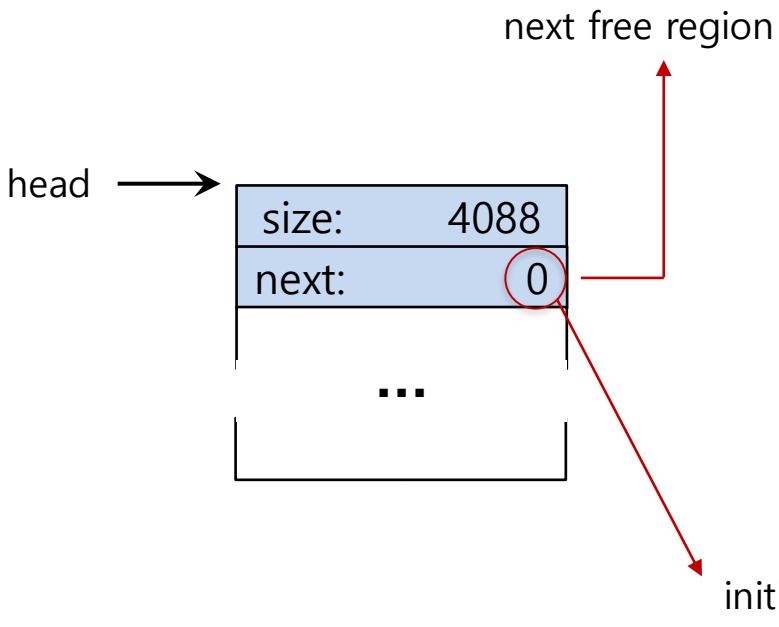
Tracking Allocated Regions (2)

- The interface to `free(void *ptr)` does not take a size parameter.
  - How does the library know the size of memory region that will be back into free list?
    - Subtract size of header from address to get the header's address.
  - Use the magic number to check the integrity.

```
void free(void *ptr) {
    header_t *header = (void *)ptr - sizeof(header_t);
    …
    assert(header->magic==1234567) ;
    …
}
```

Tracking Free Regions

- Initially, initialize the one big free region as a free list with one entry.
- Whenever the free region is splitting into multiple small regions which are not adjacent, maintain a linked list (free list) for them.
  - Each free region consists of the **size** of the free region and a **pointer** to the next free region.

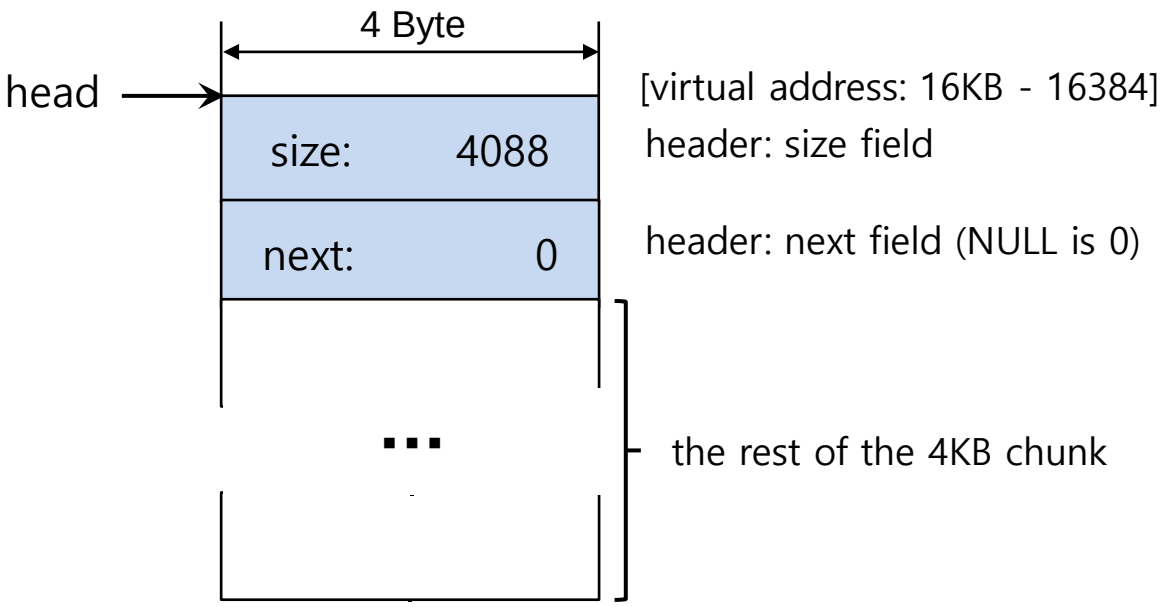



```
typedef struct __node_t {
        int size;
        struct __node_t *next;
} nodet_t;
```

Example: Heap Initialization

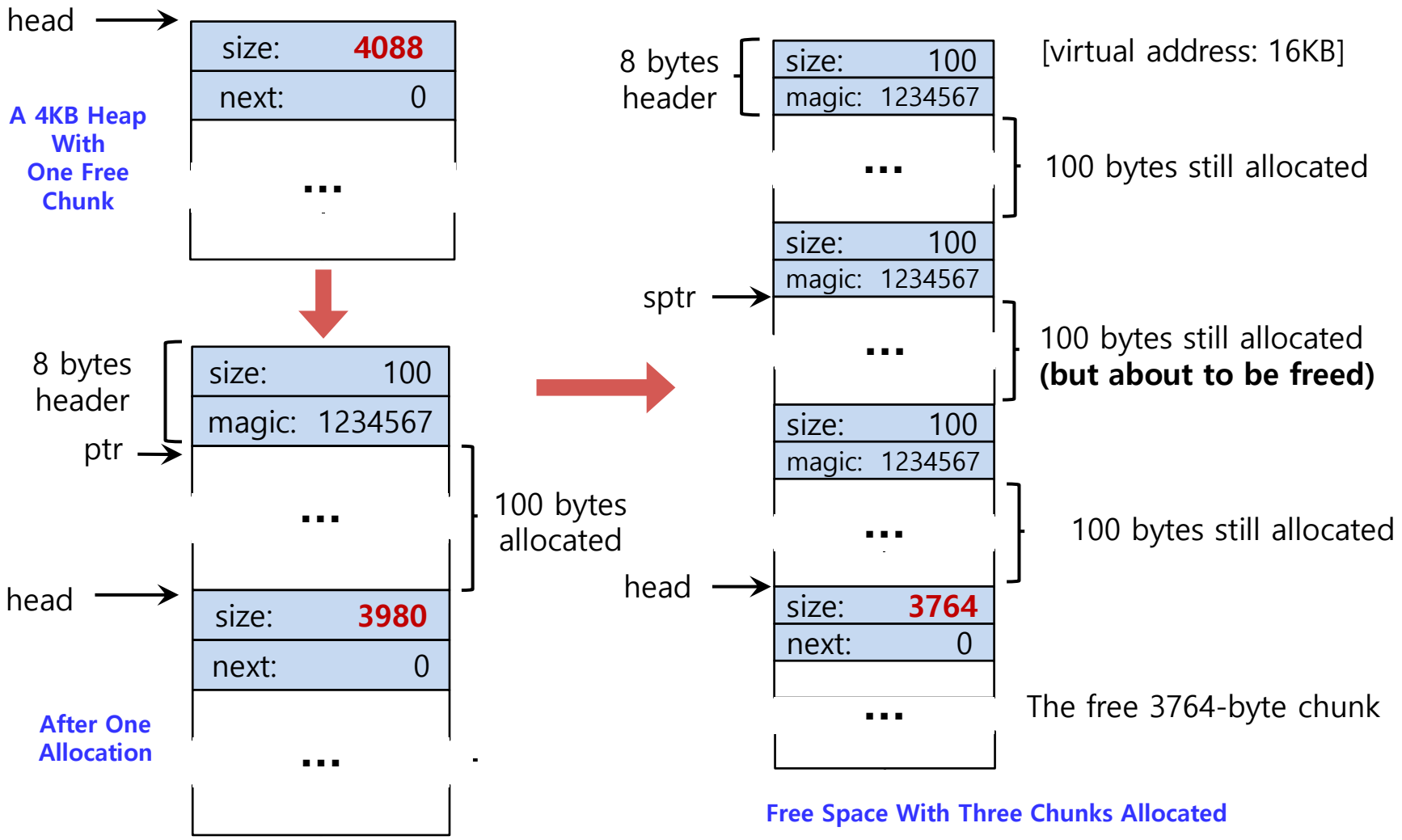
- Assume that the heap is initially built via `mmap()` system call; this is not the only way to build such as heap.

```
// mmap() returns a pointer to a chunk of free space
node_t *head = mmap(NULL, 4096, PROT_READ|PROT_WRITE,
           MAP_ANON|MAP_PRIVATE, -1, 0);
head->size = 4096 - sizeof(node_t);
head->next = NULL;
```



Example: Allocation

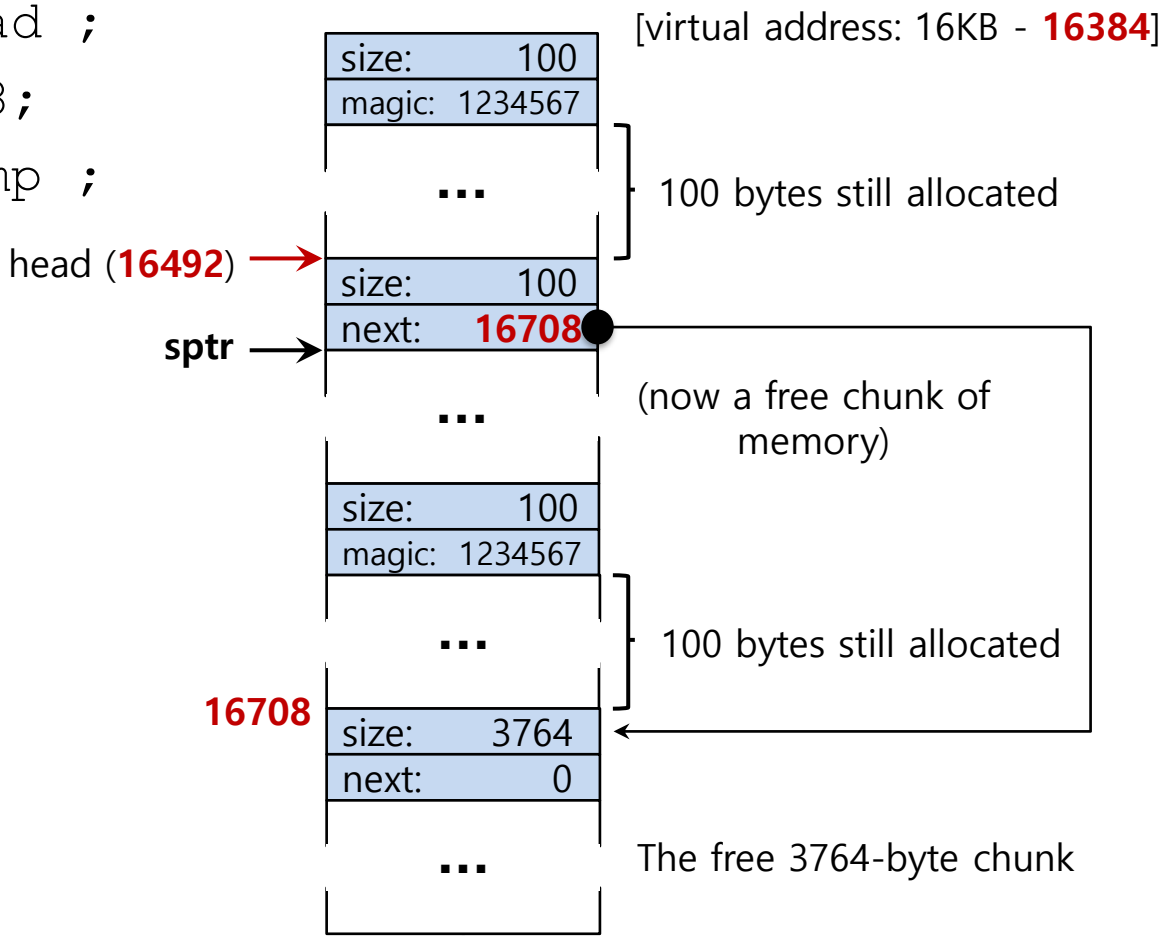
- Example: a request for 100 bytes by `ptr = malloc(100)`



Example: Free (1)

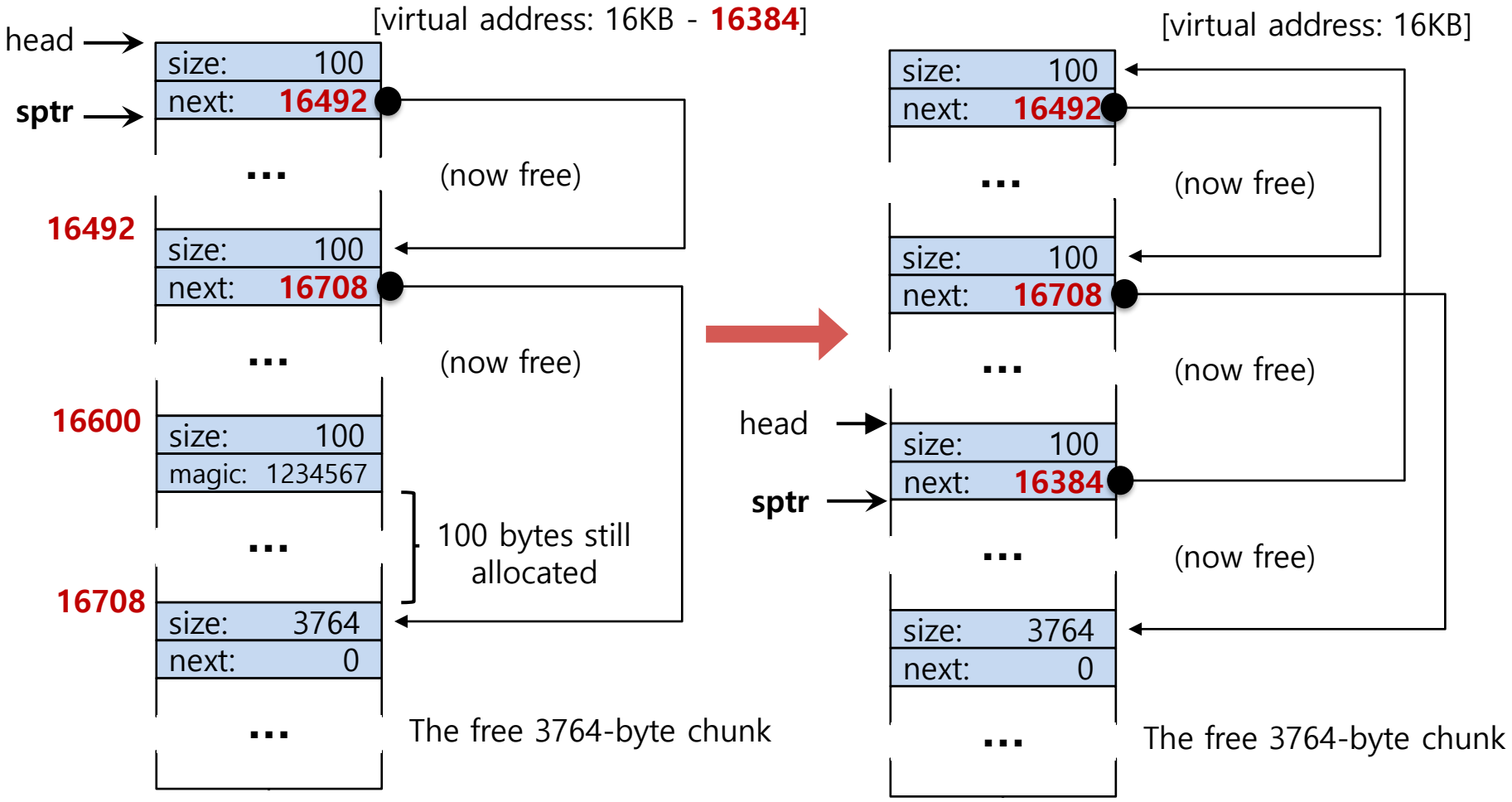
- `free(sptr)→ free(16500)`

```
void* tmp = head ;
head = sptr - 8;
head->next = tmp ;
```



Example: Free (2)

- `free(sptr)` → `free(16392)`
- `free(sptr)` → `free(16608)`

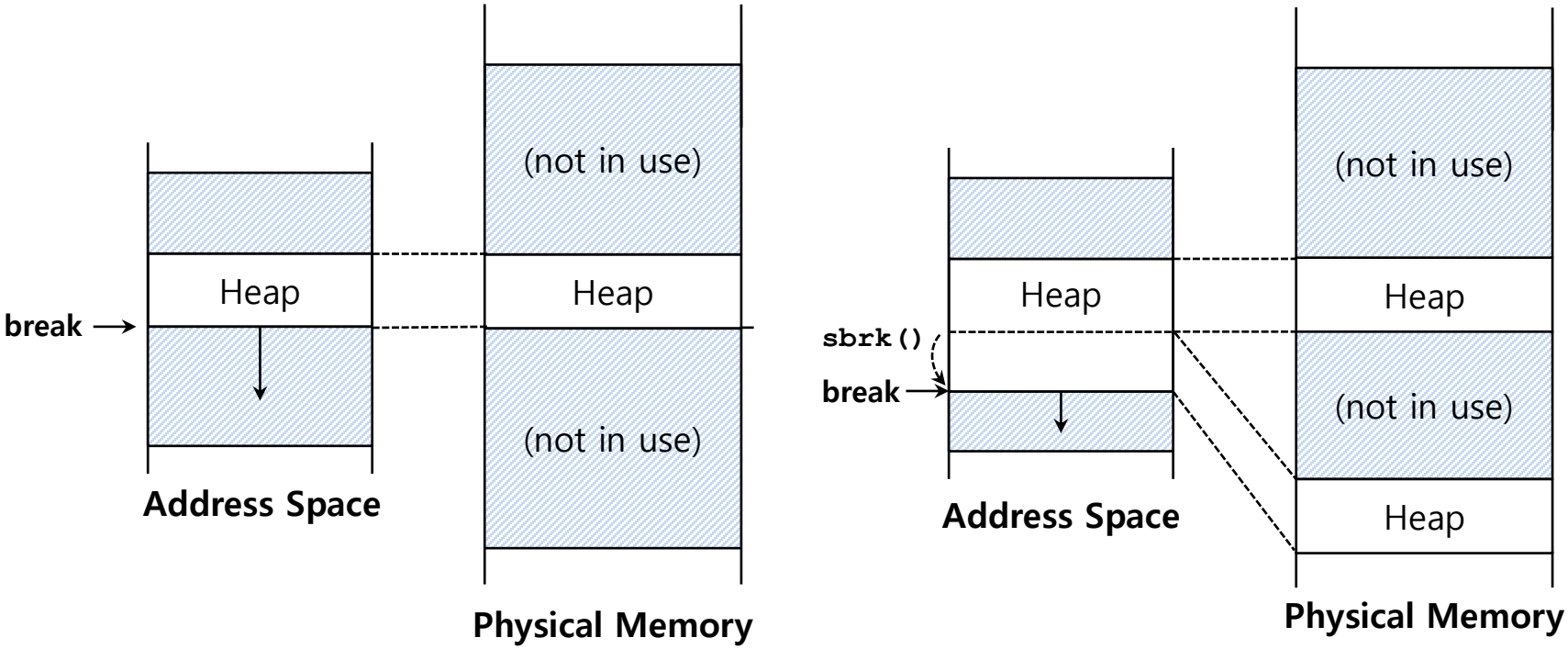


Coalescing is needed in the list !

Free needs to find the two regions on either side of the region being freed.

Growing The Heap

- Most allocators **start** with **a small-sized heap** and then **request more** memory from the OS when they run out.
  - e.g., `sbrk()`, `brk()` in most UNIX systems.

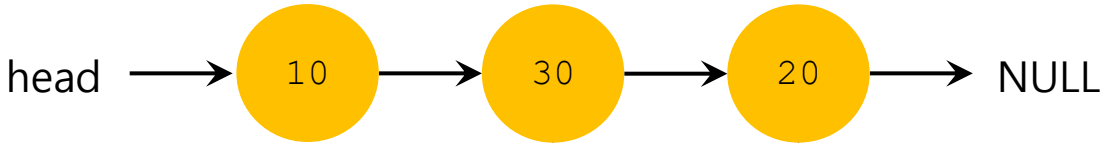


Locating a Free Region

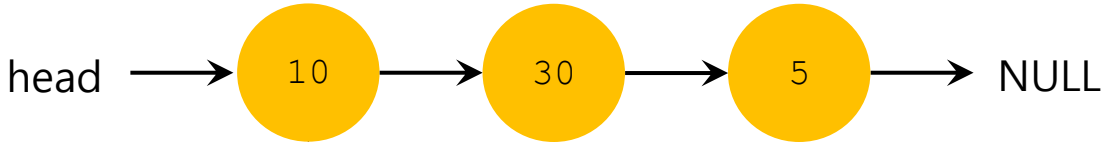
- Best Fit:
  - Finding free chunks that are big or bigger than the request.
  - Returning the one of **smallest** in the chunks in the group of candidates.
- Worst Fit:
  - Finding the **largest** free chunks and allocate the amount of the request.
  - Keeping the remaining chunk on the free list.
- First Fit:
  - Finding the **first** chunk that is big enough for the request.
  - Returning the requested amount and remaining the rest of the chunk.
- Next Fit:
  - Finding the first chunk that is big enough for the request.
  - Searching at **where one was looking** at instead of the beginning of the list.

Example: Locating a Free Region

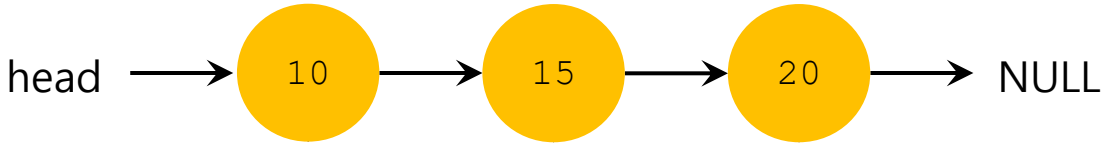
- Allocation request size is 15:



- Best-fit

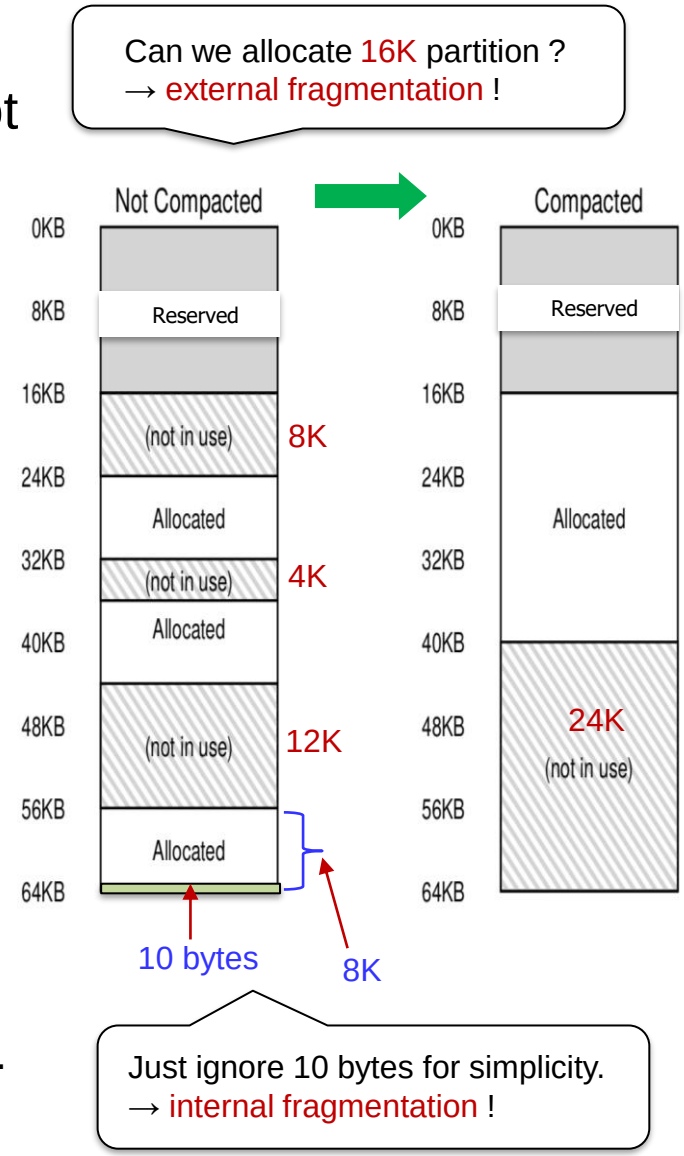


- Worst-fit



Fragmentation

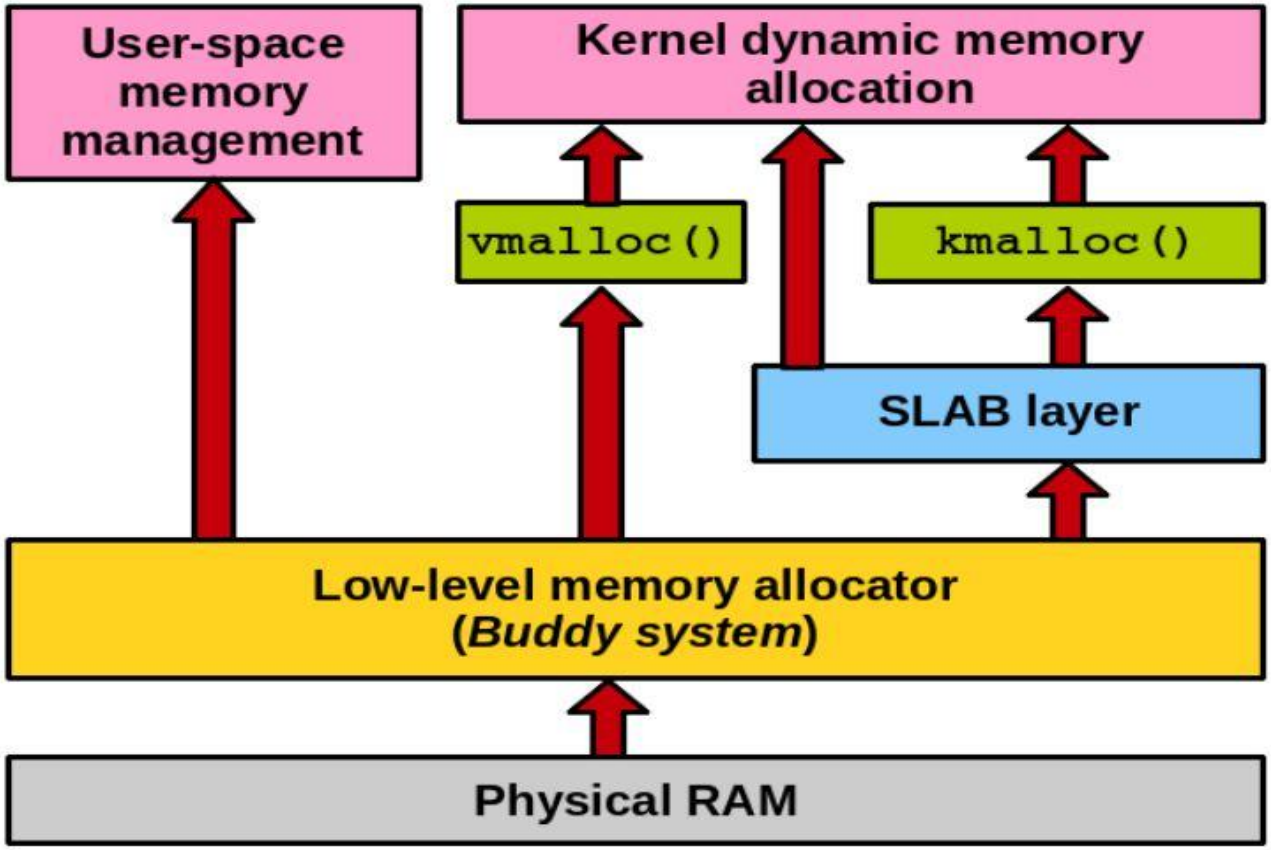
- External Fragmentation – total memory space exists to satisfy a request, but it is not contiguous.
- Internal Fragmentation – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used.
- Reduce external fragmentation by compaction.
  - Shuffle memory contents to place all free memory together in one large block.
  - Compaction is possible only if relocation is dynamic, and is done at execution time.
  - I/O problem
    - Latch job in memory while it is involved in I/O.
    - Do I/O only into OS buffers.



Segregated Lists

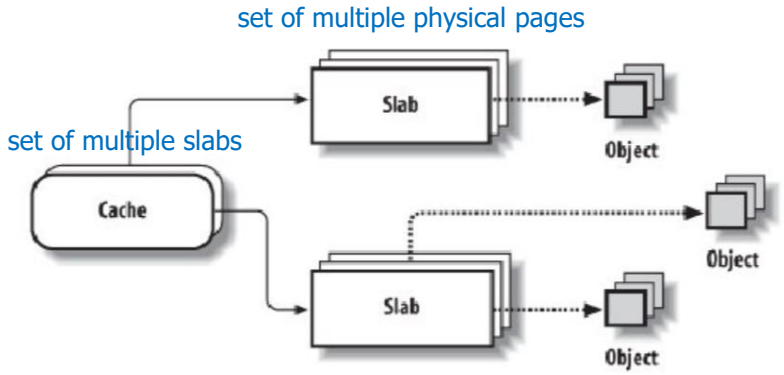
- Problems of linked-list based approach
  - May suffer from external fragmentation.
  - Not efficient for requests with the same size (popular-sized request).
    - Allocation and free take times.
- Segregated list
  - By having a chunk of memory dedicated for one particular size of requests, fragmentation is much less of a concern and allocation/free requests can be served quite quickly.
  - Other requests are forwarded to a more general memory allocator.
  - Things to consider: how much memory should one dedicate to the pool of memory that serves specialized requests of a given size.

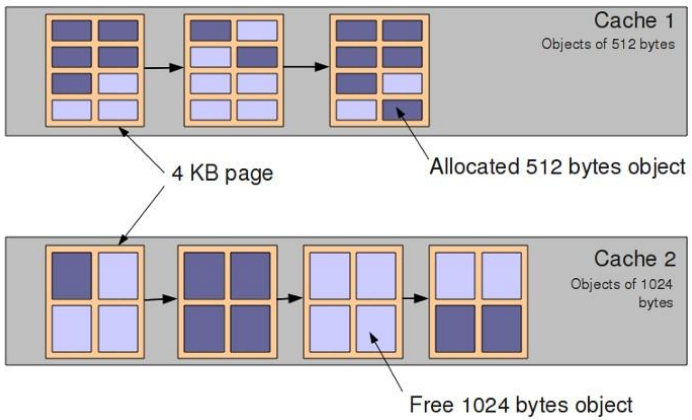
Layers of Memory Allocation



Kernel Memory Allocator: Slab Allocator (Brief Overview)

- Advanced form of segregated list
- Slab: a set of kernel pages
  - Consists of same objects with the same size (e.g. inodes, locks, sockets, etc.) and initialized before allocation.
- Using object
  - Allocate memory → Construct the object → Use the object → Release the object.
  - Reuse possible data structures rather than release.





Kernel Memory Allocator: Buddy Allocator

- Free memory is conceptually thought of as one big space of size 2^n .
- When a request for memory is made, the search for free space recursively divides free space by two until a block that is big enough to accommodate the request is found.
  - Suffer from internal fragmentation.
- When the buffer is split, each half is called buddy of the other.
  - Keep the status of each buddy to see whether it is occupied or not.
- When the allocated blocks are returned, try to merge them with free buddy blocks to create larger blocks.
- Use a bitmap to monitor to check the status of each block.
- Maintain a free list for each block size.

Example: Buddy Allocator (1)

- Maintain free list for every buffer size (32-512).

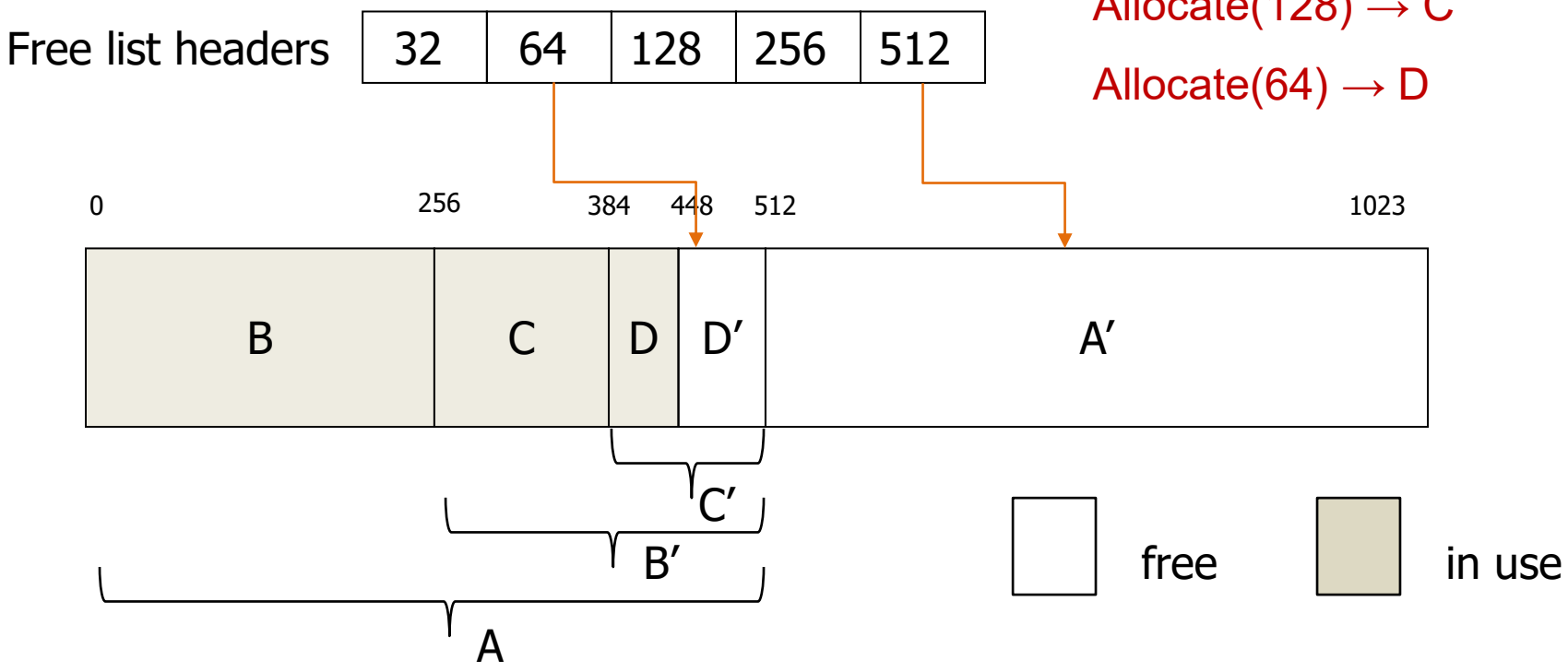
Bitmap (32 bits – 1 bit represent a status of 32-byte block).

1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Allocate(256) → B

Allocate(128) → C

Allocate(64) → D

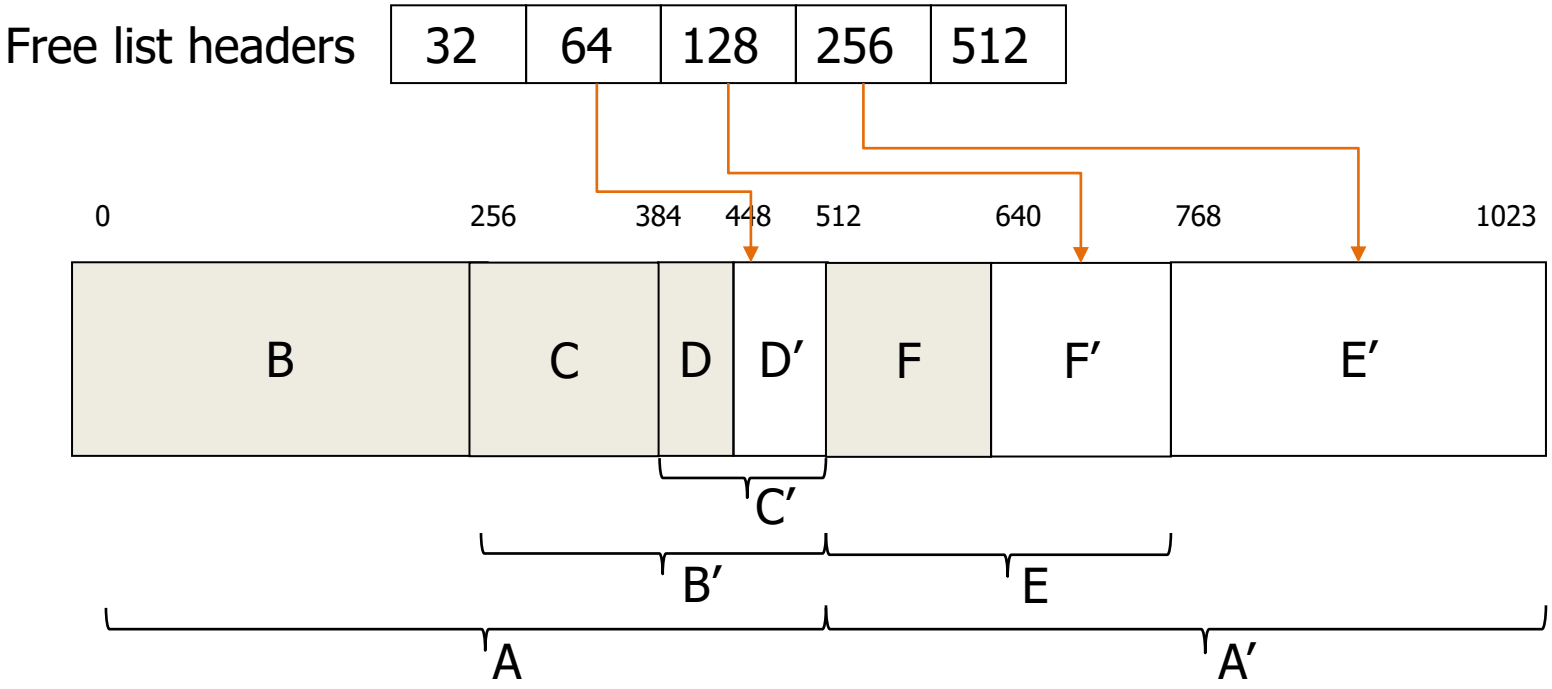


Example: Buddy Allocator (2)

Bitmap

1	1	1	1	1	1	1	1	0	0	0	0	1	1	0	0	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0

Allocate(128) → F

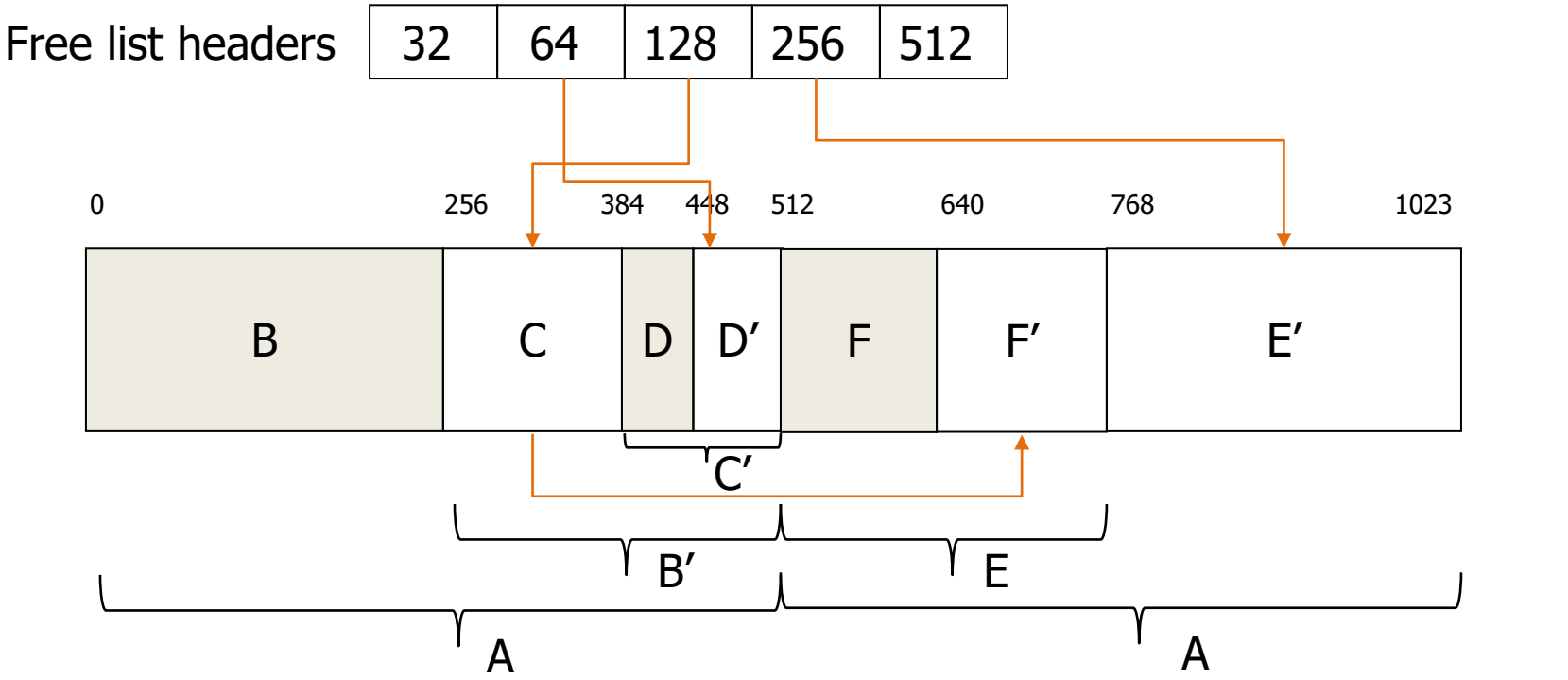


Example: Buddy Allocator (3)

Bitmap

1	1	1	1	1	1	1	1	0	0	0	0	1	1	0	0	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0

Release(C, 128)

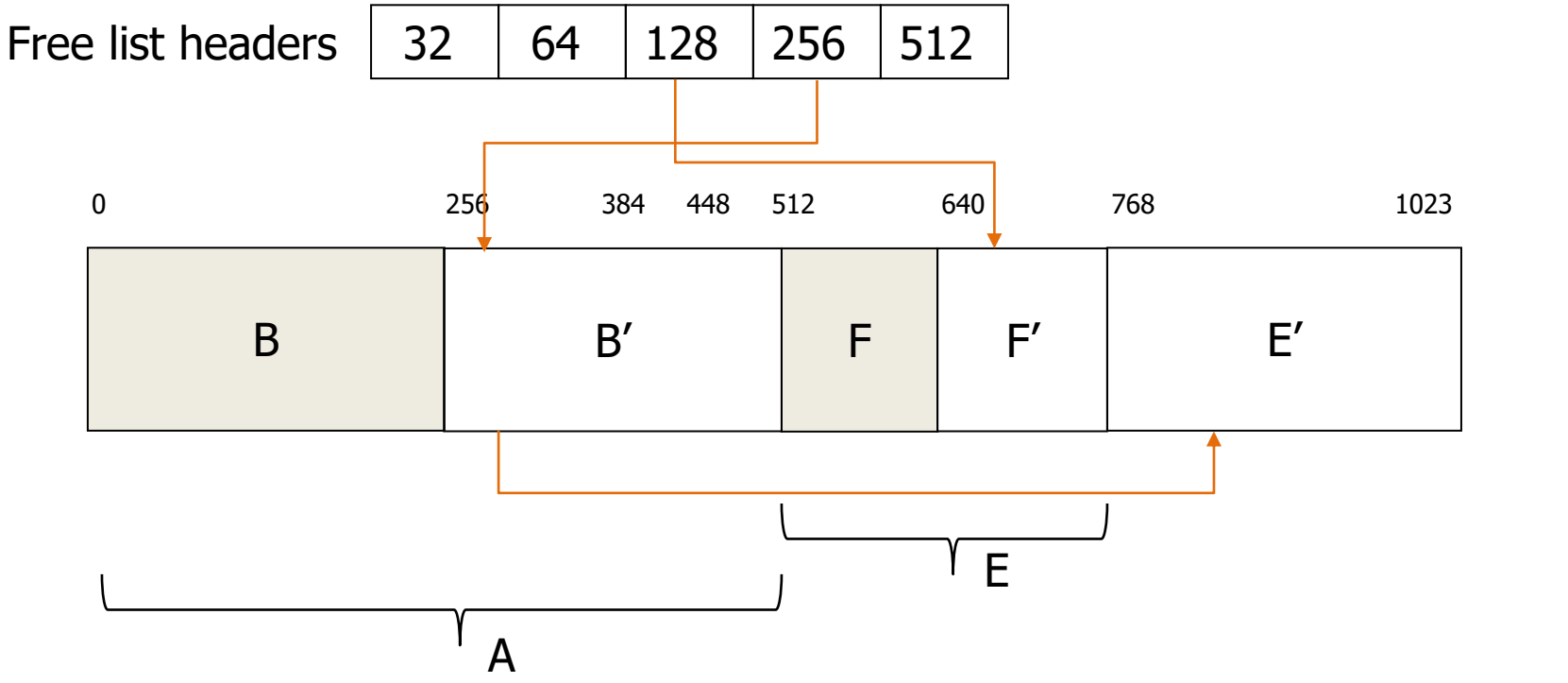


Example: Buddy Allocator (4)

Bitmap

1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0

Release(D, 64)



Buddy Allocator - Analysis

- Characteristic
  - Internal fragmentation by power-of-two allocation.
  - Easy to find a buddy of a buffer by address and size.
  - Use bitmap for coalescing.
- Advantage
  - Does a good job of coalescing adjacent free buffers.
  - Minimize the external fragmentation.
- Disadvantage
  - Performance degrade: every time a buffer is released, the allocator tries to coalesce as much as possible.
  - Release routine needs both the address and size of the buffer.

15. Mechanism: Address Translation

Memory Virtualization

- What is memory virtualization?
  - Provides an illusion of memory space per each process.
  - It seems to be seen like each process uses the whole memory.
- Benefits
  - Ease of use in programming.
  - Memory efficiency.
  - The guarantee of isolation for processes as well as OS.
    - Protection from errant accesses of other processes.
- Initial assumptions
  - The size of the virtual address space is less than the size of physical memory.
  - The virtual address space must be placed contiguously in physical memory.
  - Each address space is exactly the same size.

Example: Memory Accesses

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    int x;
    x = x + 3;
}
```

objdump on Linux -> disassembler



```
0x10: movl 0x8(%rbp), %edi
0x13: addl  $0x3, %edi
0x19: movl %edi, 0x8(%rbp)
```

- Register %**rbp** is the base pointer that points to base of current stack frame.
- Register %**rip** is instruction pointer (or program counter).
- Assume that initial %rip = **0x10** and %rbp = **0x200**.



① Fetch instruction at address 0x10
   Exec: load from address 0x208
② Fetch instruction at address 0x13
   Exec: no memory access
③ Fetch instruction at address 0x19
   Exec: store to address 0x208

How to Virtualize Memory?

- Problem: How to run multiple processes simultaneously?

  Addresses are “hardcoded” into process binaries.

- How to avoid collisions?

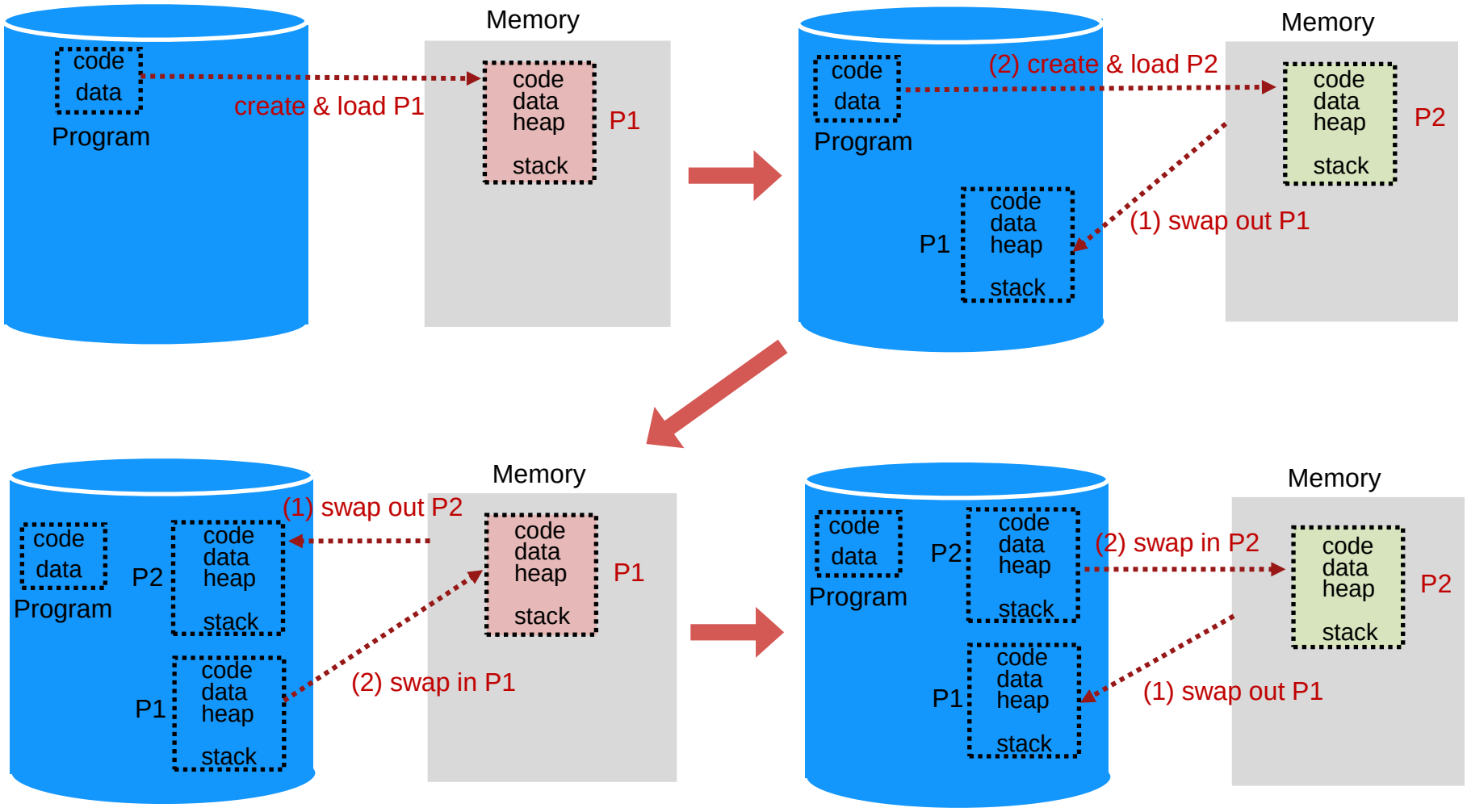
- Possible Solutions for Mechanisms:

  1. Time Sharing
  2. Static Relocation
  3. Base + Bounds

Solution 1: Time Sharing of Memory

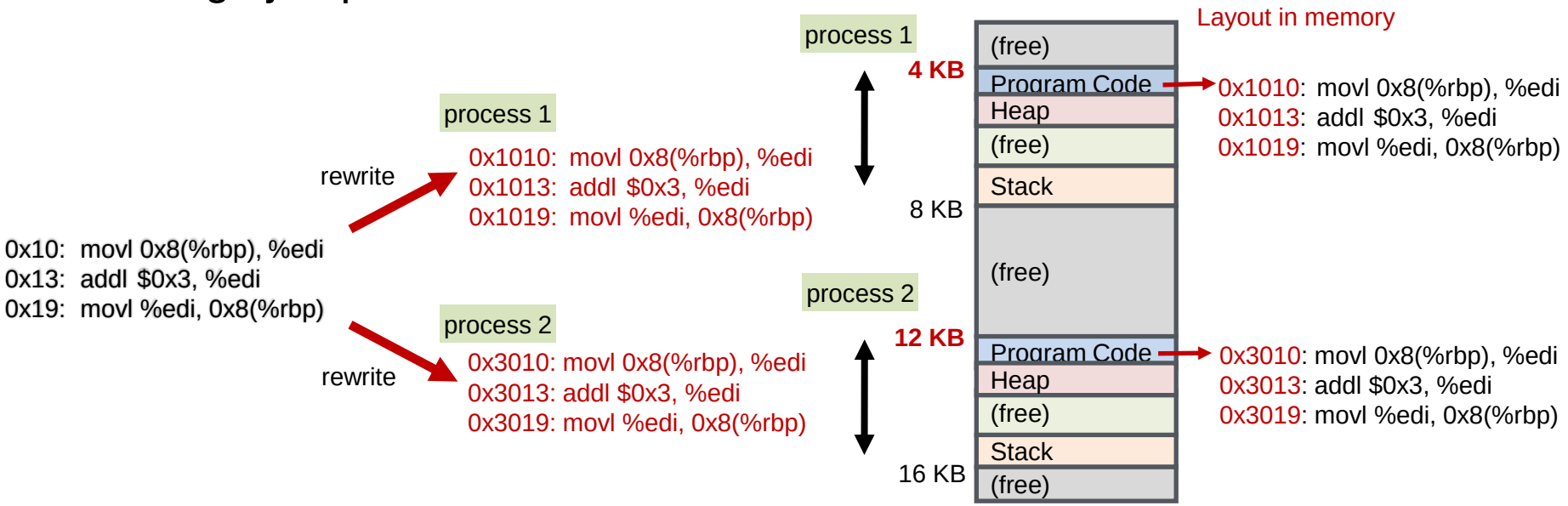
- Try similar approach to how OS virtualizes CPU.
  - Observation: OS gives illusion of many virtual CPUs by saving **CPU registers** to **memory** when a process isn't running.
  - Likewise, we can save **memory** to **disk** when process isn't running (see next slide).
- Problem
  - Ridiculously poor performance.
- Better alternative
  - Space sharing: At same time, space of memory is divided across processes.
- Remainder of solutions all use space sharing.

Example: Time Sharing of Memory



Solution 2: Static Relocation

- OS **rewrites** each program before loading it as a process in memory.
- Each rewrite for different process uses different addresses and pointers.
- Change jumps, loads of static data.



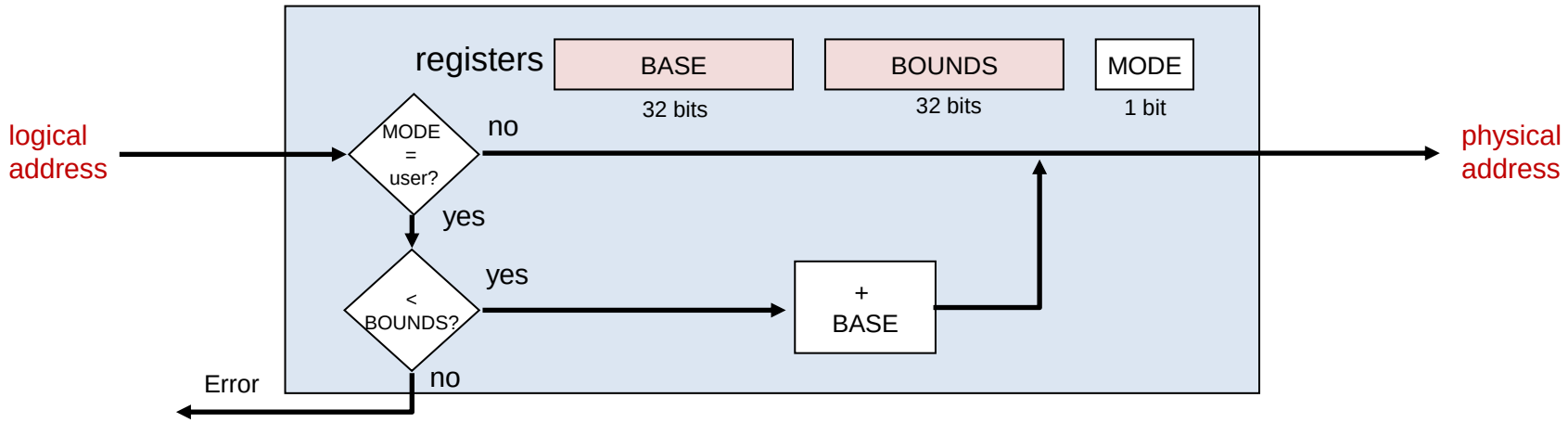
- Disadvantages
  - No protection
    - Process can destroy OS or other processes → No privacy.
  - Cannot move address space after it has been placed.

Solution 3: Dynamic Relocation - MMU

- Requires hardware support: Memory Management Unit (MMU).
- MMU contains BASE register for translation and BOUNDS register for protection.
  - **BASE register**: smallest physical address (or starting location).
  - **BOUNDS register**: size of process's virtual address space.
- Two operating modes
  - Privileged (protected, kernel) mode: OS runs
    - When the mode is changed to privileged mode via trap, system calls, interrupts, exceptions, OS can execute certain privileged instructions (e.g., manipulate contents of MMU).
    - Allows OS to access all of physical memory.
  - User mode: User processes run
    - Perform translation of logical address to physical address

Solution 3: Dynamic Relocation – Address Translation

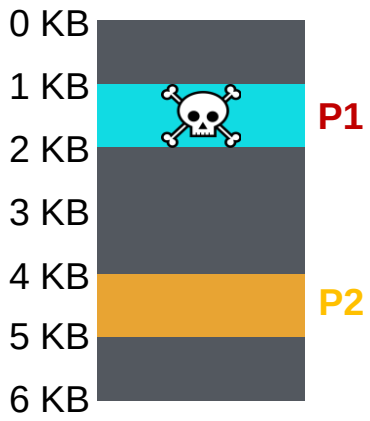
- Translation mechanism
  - Translate on every memory access of user process.
  - MMU compares logical address to BOUNDS register.
  - If logical address is greater, then generate an error (OS kills the process).
  - Otherwise, MMU adds BASE register to logical address to form physical address.



Solution 3: Dynamic Relocation – Managing Processes

- Add BASE and BOUNDS registers to the PCB of each process.
- When context switch occurs,
  - Change to privileged mode.
  - Save BASE and BOUNDS registers of old process.
  - Load BASE and BOUNDS registers of new process.
  - Change to user mode and jump to new process.
- User process cannot change BASE and BOUNDS registers.

P1 (BASE = 1024, BOUNDS = 1024), P2 (BASE = 4096, BOUNDS = 1024)

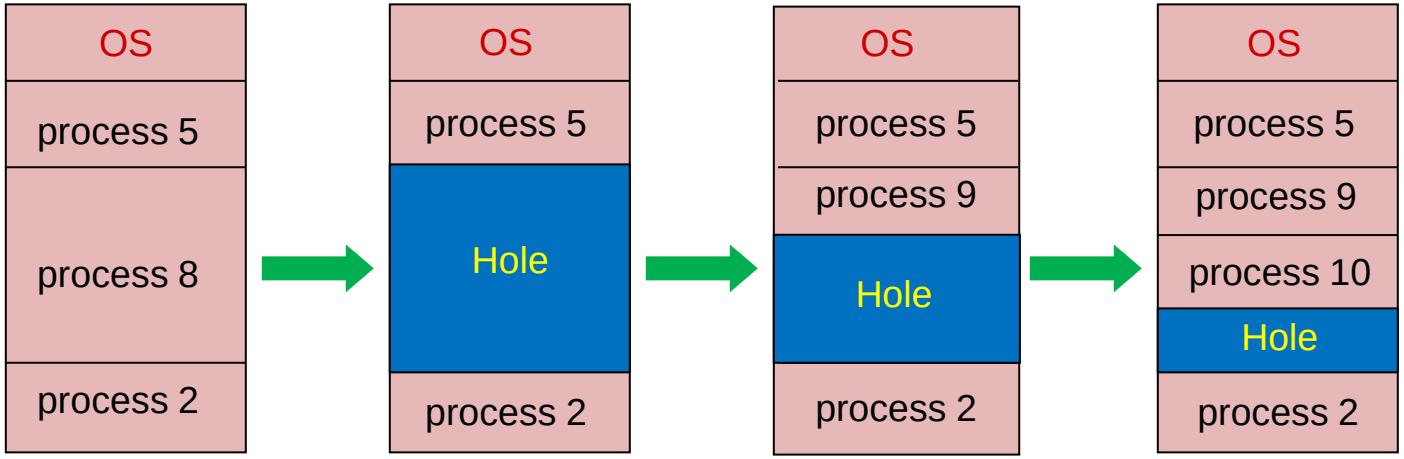


Virtual	Physical
P1: load 100, R1	load 1124, R1
P2: load 100, R1	load 4196, R1
P2: load 1000, R1	load 5096, R1
P1: load 1000, R1	load 2024, R1
P1: store 3072, R1	3072 > 1024 → interrupt OS !

Can **P1** hurt **P2**?

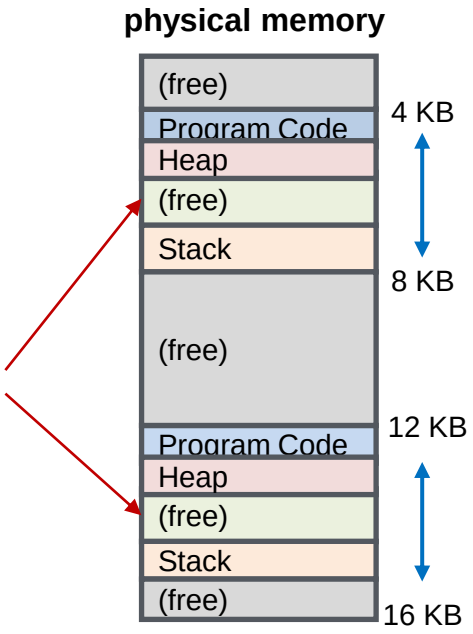
Solution 3: Dynamic Relocation – How to Allocate (Variable Size)

- If we assume that each process has the same size of address space, memory allocation is rather simple.
- However, what if **the size is different**?
  - Hole – block of available memory; holes of various size are scattered throughout memory.
  - When a process arrives, it is allocated memory from a hole large enough to accommodate it (e.g., best fit, worst fit, first fit, next fit, etc.)
  - Operating system maintains information about :
    a) allocated partitions b) free partitions (hole)



Solution 3: Dynamic Relocation – Advantages / Disadvantages

- Advantages
  - Provides protection (both read and write) across address spaces.
  - Supports dynamic relocation.
    - Can place process at different locations initially and also move address spaces.
  - Simple, inexpensive implementation.
    - Few registers, little logic in MMU.
  - Fast
    - Add and compare in parallel.
- Disadvantages
  - Each process must be **allocated contiguously** in physical memory.
    - Must allocate memory that may not be used by process (e.g., free region between heap and stack).
  - **Cannot share** limited parts of address space.
  - **External fragmentation** (when variable size) → big problem !



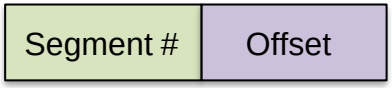
16. Segmentation

What is Segmentation?

- What is segmentation?
  - The idea was born in the early 1960's.
  - A process can have logically-different segments: Code (Data), Stack and Heap., which is just a contiguous portion of the address space.
  - Instead of having just one base and bounds pair in MMU, why not have a base and bounds pair per logical segment?
  - Each segment can:
    - be **placed separately** in physical memory.
    - grow and shrink (i.e., each segment has **variable** size).
    - be **independently protected** (separate read/write/execute protection bits).
- Segmentation allows the OS to place each one of those segments (Code, Stack, and Heap) in different parts of physical memory, and thus avoid filling physical memory with unused virtual address space.

Segmented Addressing

- Logical address now consists of segment # and offset within segment.
- How does process designate a particular segment?



  - Top bits of logical address are used to select segment (segment #).
  - Low bits of logical address are used to select offset within a segment.
- MMU contains segment table or STBR (Segment Table Base Register) / STLR (Segment Table Length Register).
  - Each segment has own BASE and BOUNDS, and protection bits.
  - Example: 14 bit logical address, 4 segments; how many bits are needed for segment and offset? How big is the segment?

Segment	Base	Bounds	R W
0	0x2000	0x6ff	1 0
1	0x0000	0x4ff	1 1
2	0x3000	0xfff	1 1
3	0x0000	0x000	0 0

remember:
1 hex digit->4 bits

Address Translations with Segmentation

Segment	Base	Bounds	R W
0	0x2000	0x6ff	1 0
1	0x0000	0x4ff	1 1
2	0x3000	0xfff	1 1
3	0x0000	0x000	0 0

Translate logical addresses (in hex) to physical addresses

0x0240:

0x1108:

0x265c:

0x3002:

0x1108 = 00**01 0001 0000 1000b**

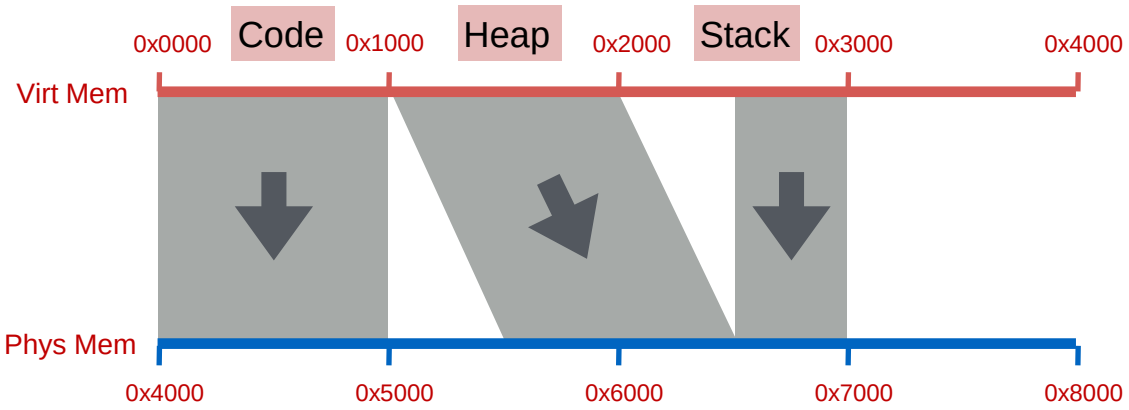
Segment (1) Offset

0x0000 + 0x108 = 0x0108

```
1   // get top 2 bits of 14-bit VA
2   Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT
3   // now get offset
4   Offset = VirtualAddress & OFFSET_MASK
5   if (Offset >= Bounds[Segment])
6        RaiseException(PROTECTION_FAULT)
7   else
8        PhysAddr = Base[Segment] + Offset
9        Register = AccessMemory(PhysAddr)
```

- SEG_MASK = 0x3000(11000000000000)
- SEG_SHIFT = 12
- OFFSET_MASK = 0xFFF (00111111111111)

Example: Segmentation



Segment Table

Segment	Base	Bounds
0 (Code)	0x4000	0xfff
1 (Heap)	0x5800	0xfff
2 (Stack)	0x6800	0x7ff

%rip = 0x0010

```
0x0010:  movl   0x1100, %edi
0x0013:  addl   $0x3, %edi
0x0019:  movl   %edi, 0x1100
```

5 memory references !

→ 3 instruction fetches, 2 *movl*

Physical Memory Accesses?

1) Fetch instruction at logical address 0x0010
   - Physical address: 0x4010

   Exec, load from logical address 0x1100
   - Physical address: 0x5900

2) Fetch instruction at logical address 0x0013
   - Physical address: 0x4013

   Exec, no load

3) Fetch instruction at logical address 0x0019
   - Physical address: 0x4019

   Exec, store to logical address 0x1100
   - Physical address: 0x5900

Advantages and Disadvantages

- Advantages
  - Enables sparse allocation of address space.
    - Stack and heap can grow independently.
  - Different protection for different segments.
    - Read-only status for code.
  - Enables sharing of selected segments.
  - Supports dynamic relocation of each segment.
- Disadvantages
  - Each segment must be allocated contiguously.
    - May not have sufficient physical memory for large segments, resulting in external fragmentation.

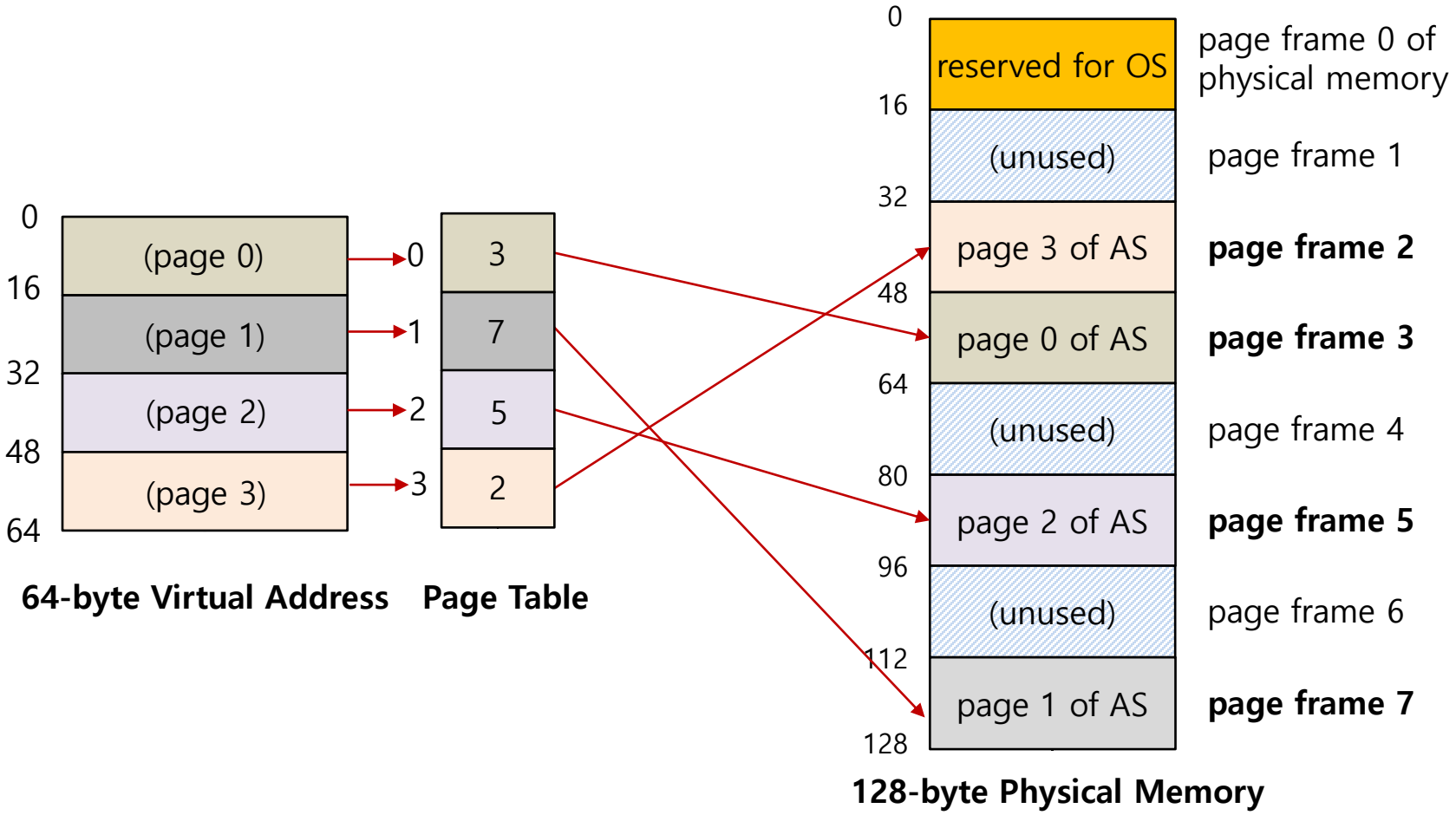
18. Introduction to Paging

Concept of Paging

- Paging splits up address space into fixed-sized unit called a page.
  - Segmentation: variable size of logical segments(code, stack, heap, etc.)
- With paging, physical memory is also split into some number of pages called a page frame.
- Page table per process is needed to translate the virtual address to physical address.
- Advantages
  - Flexibility: Supporting the abstraction of address space effectively.
    - Don’t need assumption how heap and stack grow and are used.
  - Simplicity: ease of free-space management.
    - The page in address space and the page frame are the same size.
    - Easy to allocate and keep a free list.

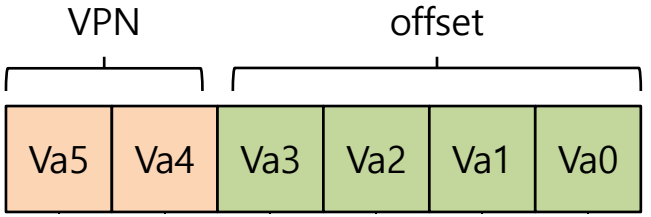
Example: A Simple Paging

- 64-byte address space with 16 bytes pages (4 pages).
- 128-byte physical memory with 16 bytes page frames (8 page frames).

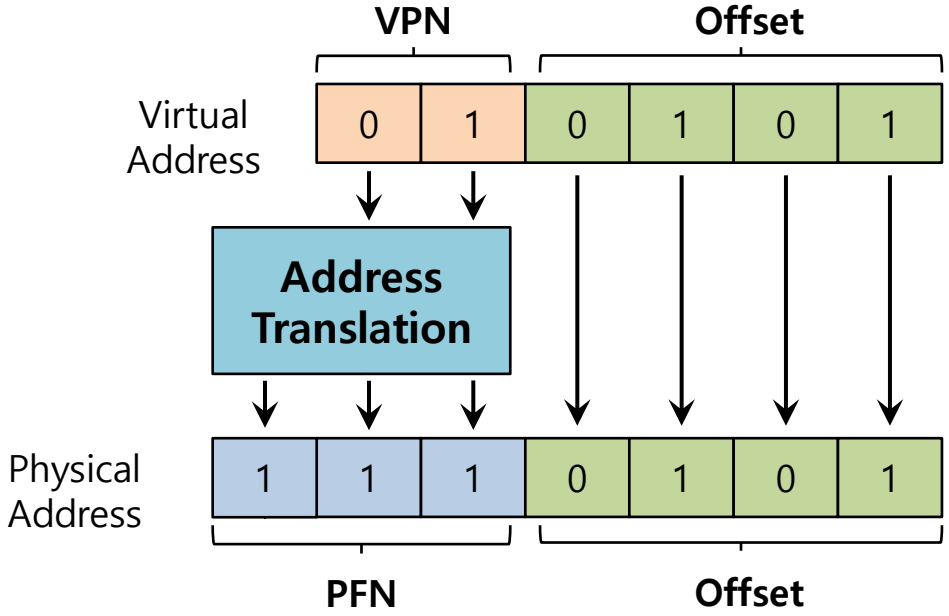


Address Translation

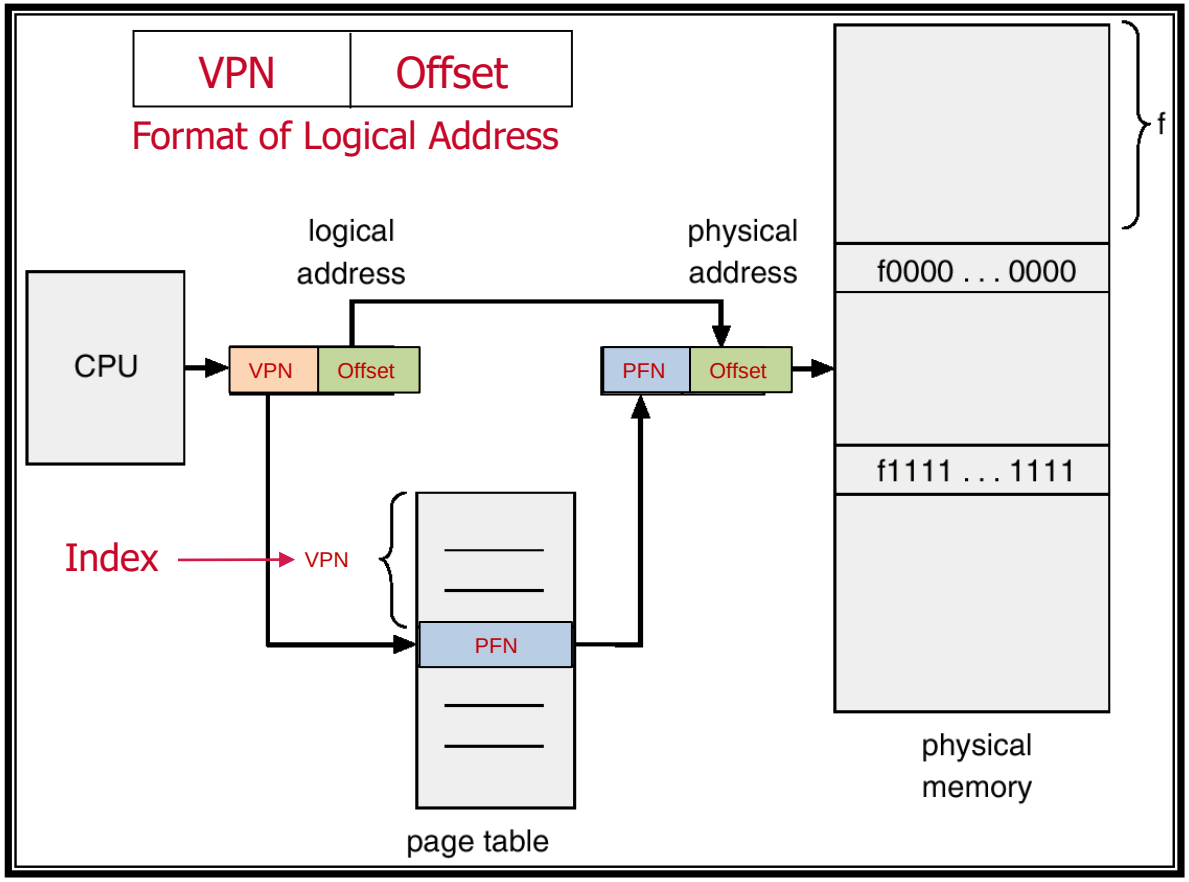
- Two components in the virtual address
  - **VPN**: virtual page number
  - **Offset**: offset within the page



- Example: virtual address 21 (0x15) in 64-byte address space



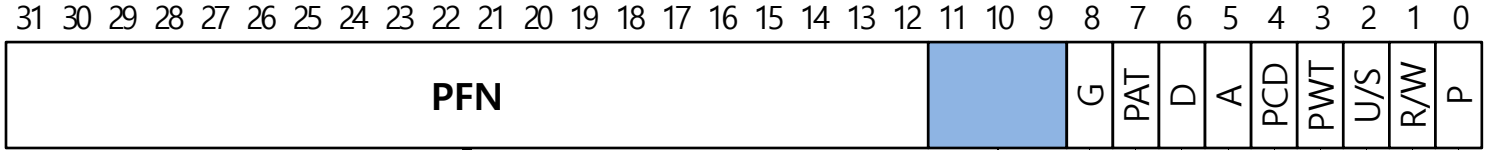
Address Translation Example



What Is In The Page Table?

- The page table is a data structure that is used to map the virtual address to physical address.
  - Simplest form: a linear page table, an array.
- The OS indexes the array by VPN, and looks up the page-table entry.
- Common Entries are:
  - **Valid Bit**: Indicating whether the particular translation is valid.
  - **Protection Bit**: Indicating whether the page could be read from, written to, or executed from.
  - **Present Bit**: Indicating whether this page is in physical memory or on disk (swapped out).
  - **Dirty Bit**: Indicating whether the page has been modified since it was brought into memory.
  - **Reference Bit (Accessed Bit):** Indicating that a page has been accessed.

Example: x86 Page Table Entry



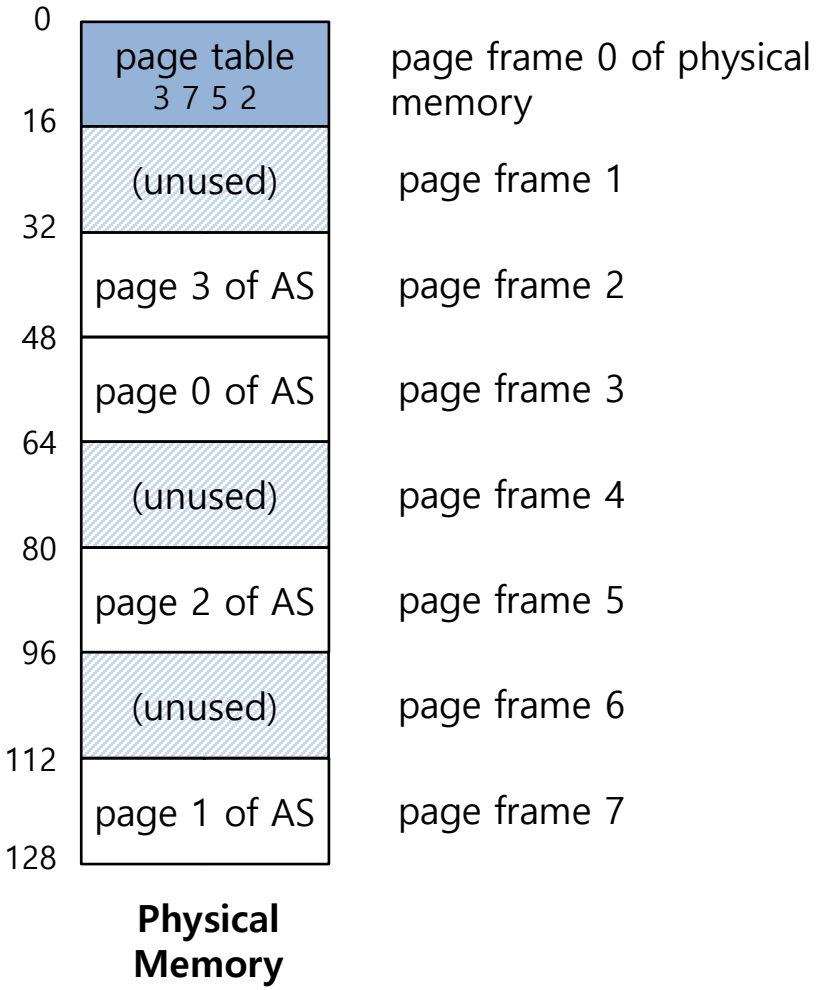
An x86 Page Table Entry(PTE)

- P: present
- R/W: read/write bit
- U/S: supervisor
- A: accessed bit
- D: dirty bit
- PFN: the page frame number

Where Are Page Tables Stored?

- Page tables can get awfully large.
  - 32-bit address space with 4-KB pages, 20 bits for VPN
    - Number of Pages : 20 bits
    - $4MB = 2^{20}\ entries\ *$

      $4\ Bytes\ per\ page\ table\ entry$
- Page tables for each process are stored in memory.
  - To find a location of the desired PTE, the starting location of the page table is needed (e.g., **PTBR** – Page Table Base Register).
  - For every memory reference, paging requires the OS to perform one **extra memory reference** → Too slow !



Accessing Memory With Paging

```
1       // Extract the VPN from the virtual address
2       VPN = (VirtualAddress & VPN_MASK) >> SHIFT
3       // Form the address of the page-table entry (PTE)
4       PTEAddr = PTBR + (VPN * sizeof(PTE))
5       // Fetch the PTE
6       PTE = AccessMemory(PTEAddr)
7
8       // Check if process can access the page
9       if (PTE.Valid == False)
10              RaiseException(SEGMENTATION_FAULT)
11      else if (CanAccess(PTE.ProtectBits) == False)
12              RaiseException(PROTECTION_FAULT)
13      else
14              // Access is OK: form physical address and fetch it
15              offset = VirtualAddress & OFFSET_MASK
16              PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
17              Register = AccessMemory(PhysAddr)
```

A Memory Trace

```
int array[1000];
...
for (i = 0; i < 1000; i++)
        array[i] = 0;
```

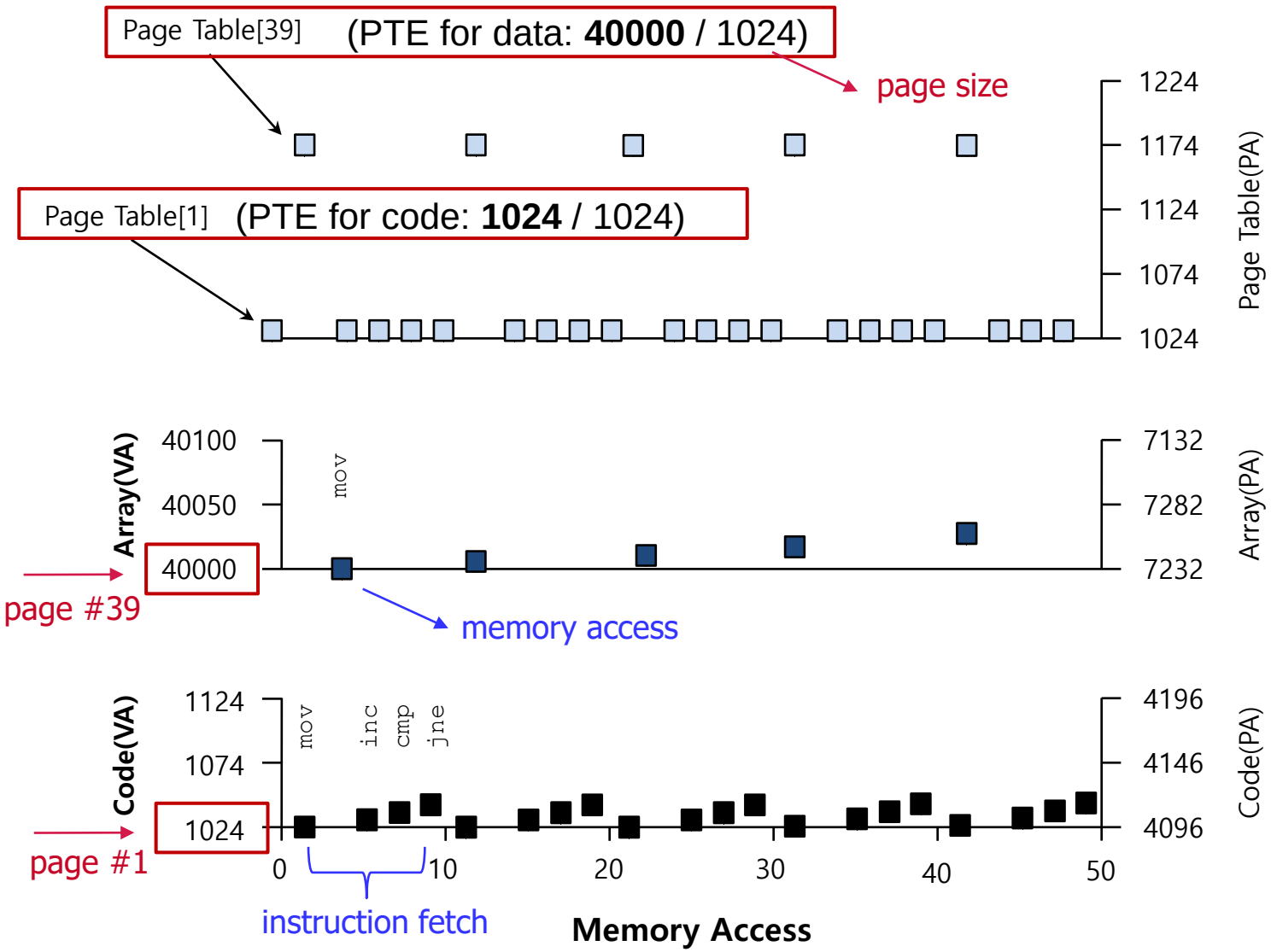
```
prompt> gcc –o array array.c –Wall –o
prompt>./array
```

---

memory access

```
0x1024 movl $0x0,(%edi,%eax,4) //[edi+eax*4]= 0
0x1028 incl %eax
0x102c cmpl $0x03e8,%eax //0000 0011 1110 1000₂ = 1000₁₀
0x1030 jne  0x1024
```

A Virtual (and Physical) Memory Trace



19. Paging: Faster Translations (TLBs)

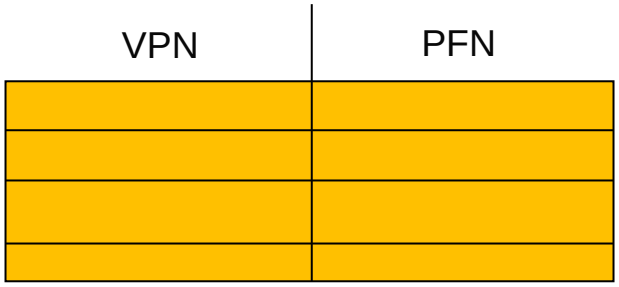
TLBs: Making Paging Faster

- Page table is per-process data structure and is kept in **main** memory.
- Every data/instruction access requires **two** memory accesses.
  - One for the page table and one for the data/instruction.
- This problem can be solved by a special fast-lookup hardware cache **inside the MMU** called associative memory or translation look-aside buffers (TLBs).
  - A typical TLB has 32,64, or 128 entries (small – need replacement policies).
  - TLB is managed by full associative method, which means that any given translation can be anywhere in the TLB and each TLB entry is checked simultaneously for a match.
  - Other bits: valid bits, protection bits, address-space identifier(ASID), dirty bit.



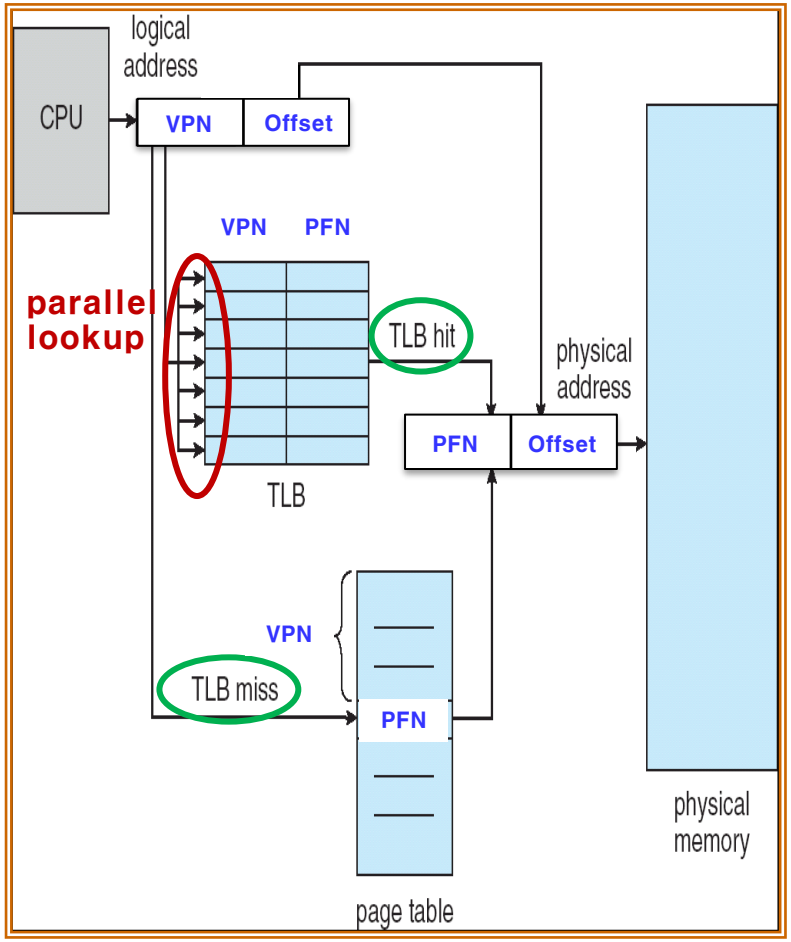
Address Translation in TLB

- Hardware searches the entire TLB in parallel to find the desired translation.



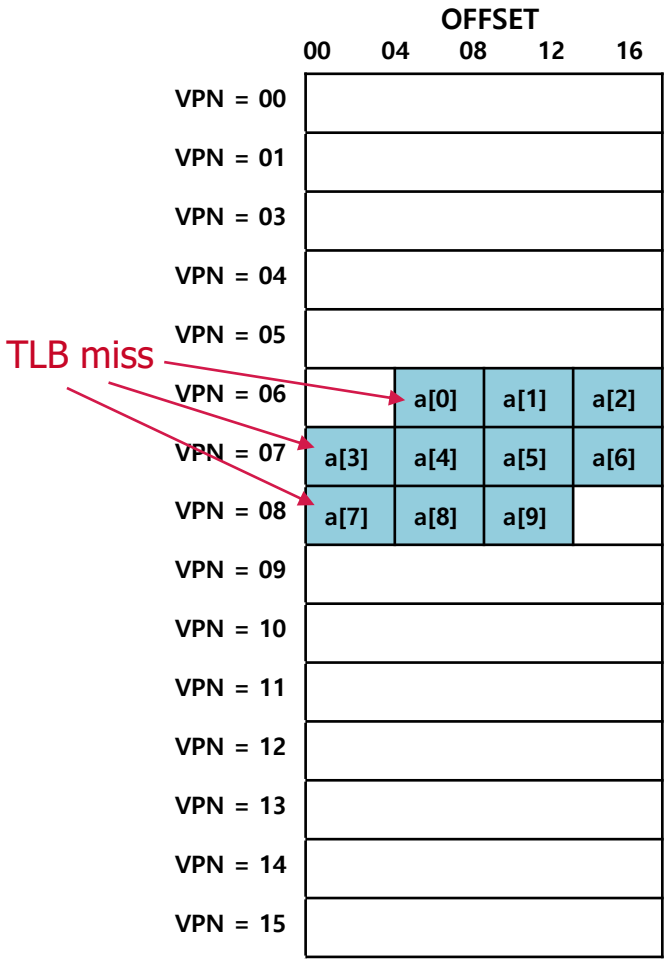
  - TLB hit: If VPN is in a TLB, get PFN out from the TLB.
  - TLB miss: Otherwise, get PFN from the page table in memory.
    - Extra overhead !

- Some entries can be wired down for permanent fast access; Typically, TLB entries for key kernel code are wired down.



Example: Accessing An Array

- How a TLB can improve its performance.




```
0:      int sum = 0 ;
1:      for (i=0; i<10; i++){
2:              sum+=a[i];
3:      }
```

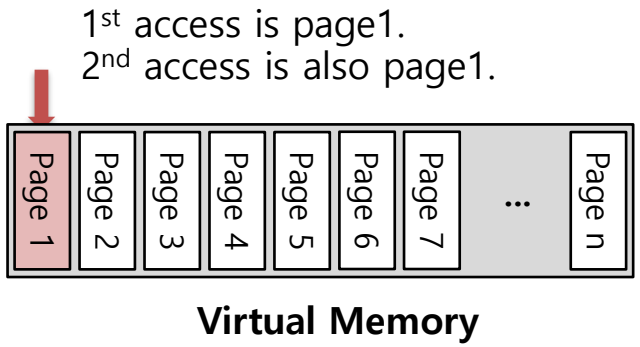
3 misses and 7 hits.
Thus TLB hit rate is 70%.

The TLB improves performance due to spatial locality

Locality: The Idea Behind Cache Design

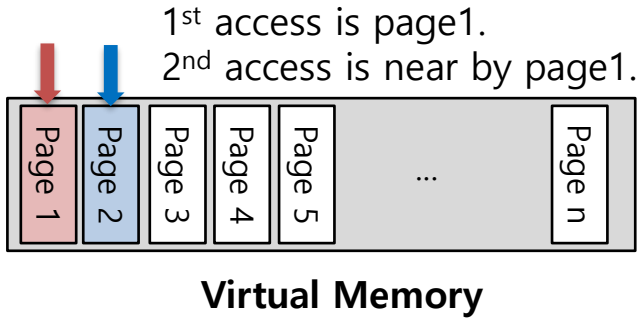
Temporal Locality

- An instruction or data item that has been recently accessed will likely be re-accessed soon in the future.



Spatial Locality

- If a program accesses memory at address `x`, it will likely soon access memory near `x`.



Who Handles The TLB Miss?

- Hardware-managed TLB (mostly on CISC machine)
  - The hardware has to know exactly **where** the page tables are located in memory and their **exact format**.
  - The hardware would “walk” the page table, find the correct page-table entry and extract the desired translation, update TLB and retry instruction.
  - Example: Intel x86 (CR3 register points to the address of current page table.)
- Software-managed TLB (on RISC machine)
  - On a TLB miss, the hardware raises exception (trap handler).
  - Trap handler is code within the OS that is written with the purpose of handling TLB miss.
  - Example: MIPS R10k, Sun's SPARC v9, etc.

TLB Control Flow Algorithm (Hardware Managed TLB)

```
1: VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2: (Success , TlbEntry) = TLB_Lookup(VPN)
3:    if (Success == TRUE) { // TLB Hit
4:       if (CanAccess(TlbEntry.ProtectBit) == True) {
5:          offset = VirtualAddress & OFFSET_MASK
6:          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7:          Register = AccessMemory(PhysAddr)
8:       } else RaiseException(PROTECTION_ERROR)
9:    } else { // TLB Miss
10:         PTEAddr = PTBR + (VPN * sizeof(PTE))
11:         PTE = AccessMemory(PTEAddr)
12:         if (PTE.Valid == False) RaiseException(SEGMENTATION FAULT)
13:         else {
14:            TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
15:            RetryInstruction()
16:         }
17:   }
```

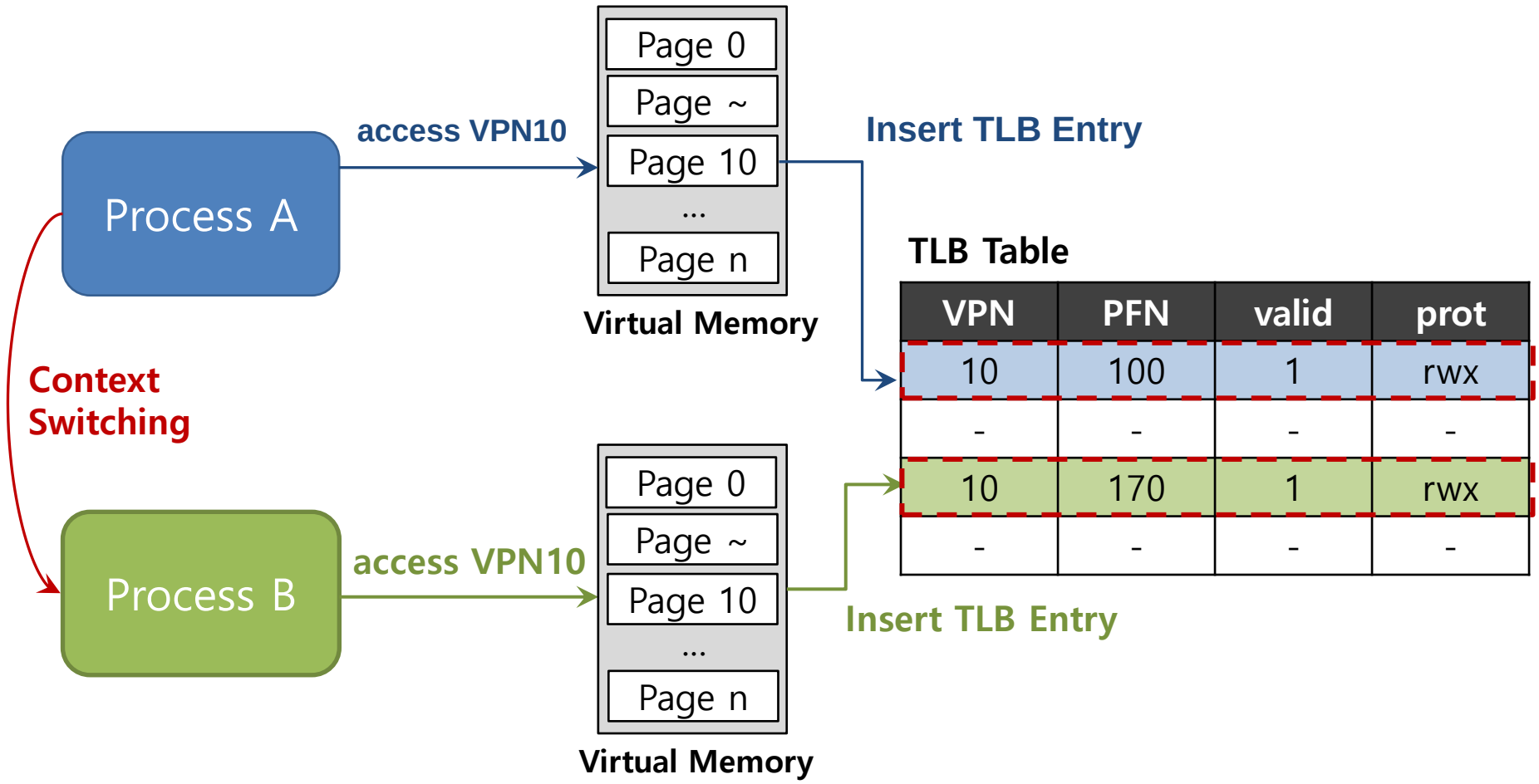
Extra memory reference → big overhead !

After this, the translation is found in the TLB, and the memory reference is processed quickly.

TLB Control Flow Algorithm (Software Managed TLB)

```
1:      VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2:      (Success, TlbEntry) = TLB_Lookup(VPN)
3:      if ( Success == True ) { // TLB Hit
4:          if ( CanAccess(TlbEntry.ProtectBits) == True ) {
5:              Offset = VirtualAddress & OFFSET_MASK
6:              PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7:              Register = AccessMemory(PhysAddr)
8:          }
9:          else RaiseException(PROTECTION_FAULT)
10:     }
11:     else // TLB Miss
12:         RaiseException(TLB_MISS)
```

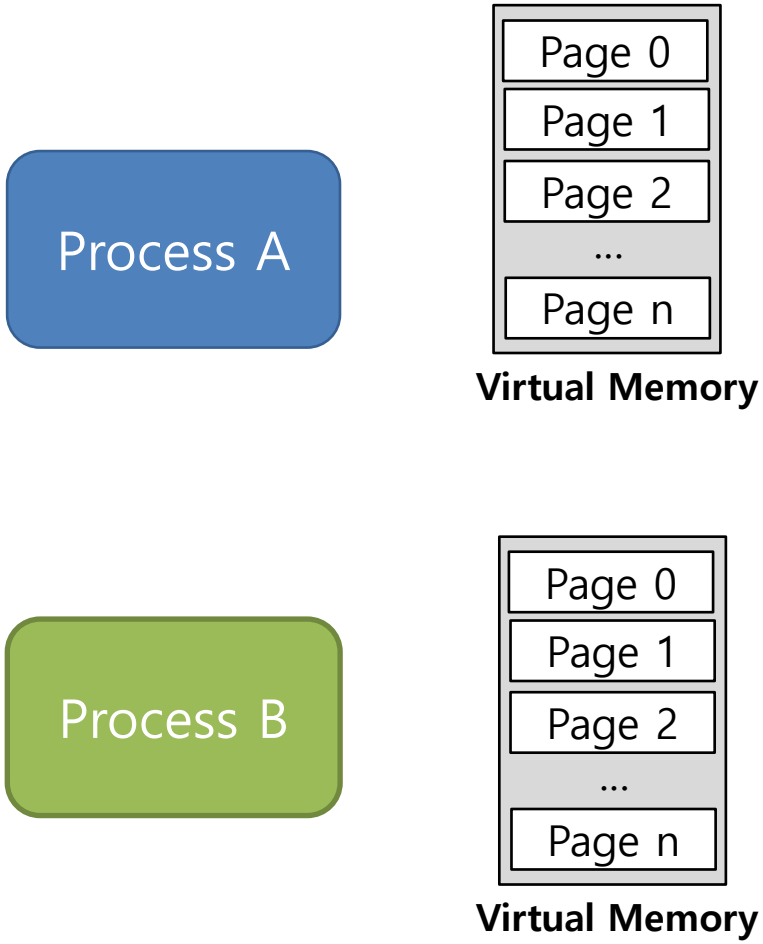
TLB Issue: Can Two Processes Share a TLB?



Can't distinguish which entry is meant for which process.
Flush out after every context switch? →high overhead.

Solution: Address Space Identifier (ASID)

- Provide an address space identifier (ASID) field in the TLB → assigned dynamically at runtime (not process creation time).



TLB Table

VPN	PFN	valid	prot	ASID
10	100	1	rwx	1
-	-	-	-	-
10	170	1	rwx	2
-	-	-	-	-

TLB Issue: Page Sharing

- Two processes share a page.
  - Process 1 is sharing physical page 101 with Process 2.
  - P1 maps this page into the 10th page of its address space.
  - P2 maps this page to the 50th page of its address space.

TLB Table

VPN	PFN	valid	prot	ASID
10	101	1	rwx	1
-	-	-	-	-
50	101	1	rwx	2
-	-	-	-	-

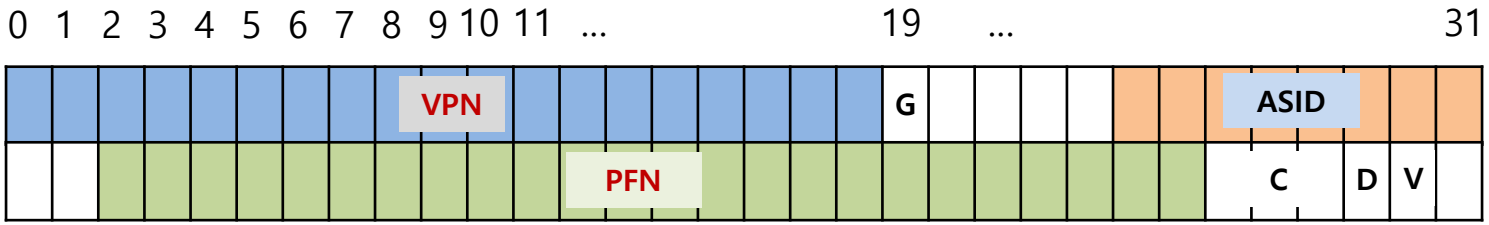
Sharing of pages is useful as it reduces the number of physical pages in use.

TLB Issue: Replacement Policy

- When we are installing a new entry in the TLB, we have to replace an old one, and thus we need to decide which one to replace.
- The goal is to minimize the miss rate (or increase hit rate) and thus improve performance.
  - LRU (Least Recently Used)
    - Evicts an entry that has not recently been used.
    - Takes advantage of *locality* in the memory-reference stream.
  - Random
    - Evicts an entry at random.
    - Useful due to its simplicity and ability corner-case behavior.
      - When a program loops over n+1 pages with a TLB of size n; in this case, LRU misses upon every access, whereas random does much better.

A Real TLB Entry

Example of a TLB entry (e.g., MIPS R4000 (32bit mode) – software-managed TLB)

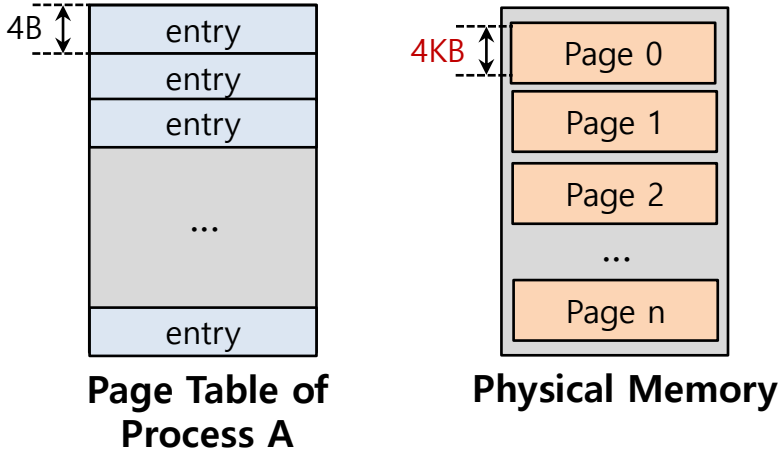


Flag	Content
19-bit VPN	Virtual address space is 2GB.
24-bit PFN	Systems can support with up to 64GB of main memory (pages).
Global bit(G)	Used for pages that are globally-shared among processes.
ASID	OS can use to distinguish between address spaces.
Coherence bit(C)	Determines how a page is cached by the hardware.
Dirty bit(D)	Marking when the page has been written.
Valid bit(V)	Tells the hardware if there is a valid translation present in the entry.

20. Paging: Smaller Tables

Paging Problem: Page Tables are Too Big

- We usually have one page table for every process in the system.
  - Assume that 32-bit address space with 4KB pages and 4-byte page-table entry.

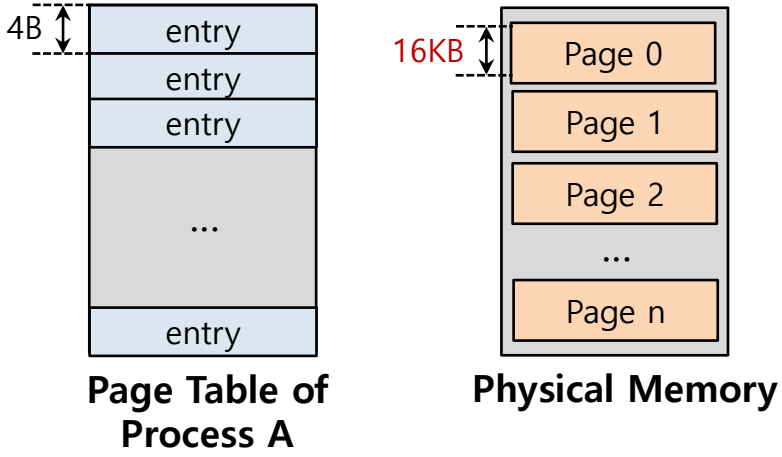


Page table size = $\frac{2^{32}}{2^{12}} * 4\ bytes = 4MB$

Page table are too big and thus consume too much memory.

Simple Solution: Bigger Pages

- What if we use bigger pages?
  - Assume that 32-bit address space with 16KB pages and 4-byte page-table entry.

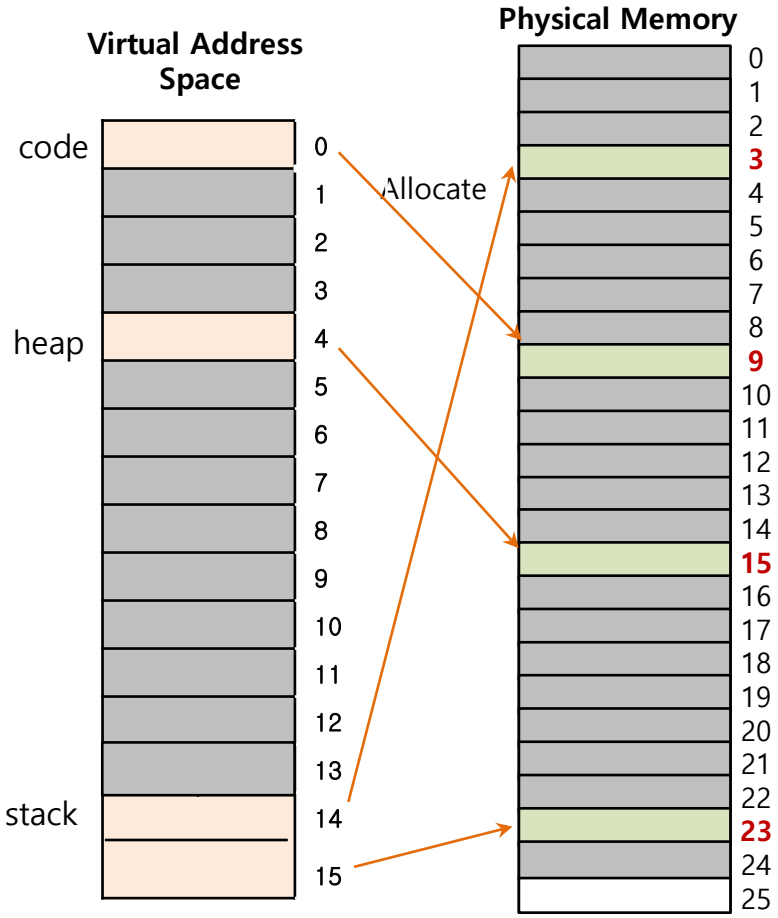


$$\frac{2^{32}}{2^{14}} * 4\ bytes = 1MB$$ per page table

But, big pages lead to internal fragmentation.

Another Problem: Unused Space

- Most of the page table is **unused**, full of invalid entries.



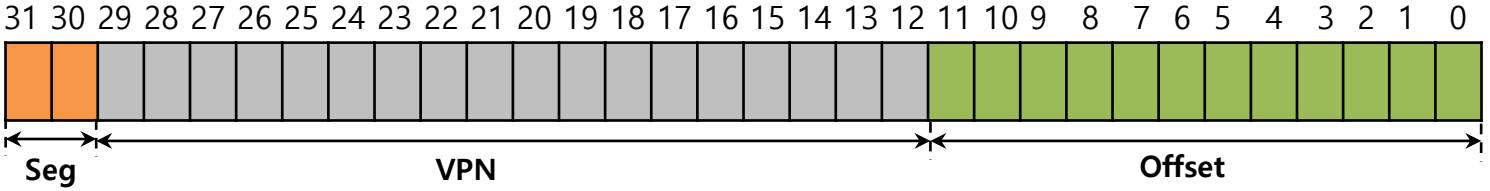
A 16KB Address Space with 1KB Pages

PFN	valid	prot	present	dirty
9	1	r-x	1	0
-	0	-	-	-
-	0	-	-	-
-	0	-	-	-
15	1	rw-	1	1
...	...	...	...	...
-	0	-	-	-
3	1	rw-	1	1
23	1	rw-	1	1

A Page Table For 16KB Address Space

Hybrid Approach: Paging and Segments

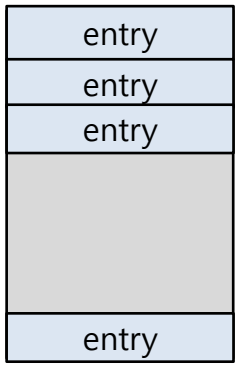
- Page table for each segment
  - The base register for each of these segments contains the physical address of the page table of that segment.
  - The bound register: indicate the end of the page table (# of valid entries).
- Example: Each process has three page tables associated with it.



32-bit Virtual address space with 4KB pages

Segment table

Seg value	Content	Base	Bounds
00	unused segment	X	X
01	code	0x----	0x----
10	heap	0x----	0x----
11	stack	0x----	0x----



Page table for code segment

Hybrid Approach: Problem

- Address translation
  - The hardware uses the segment bits (SN) to determine which base and bounds pair to use.
  - The hardware then takes the physical address of the page table and combines it with the VPN as follows to form the address of the page table

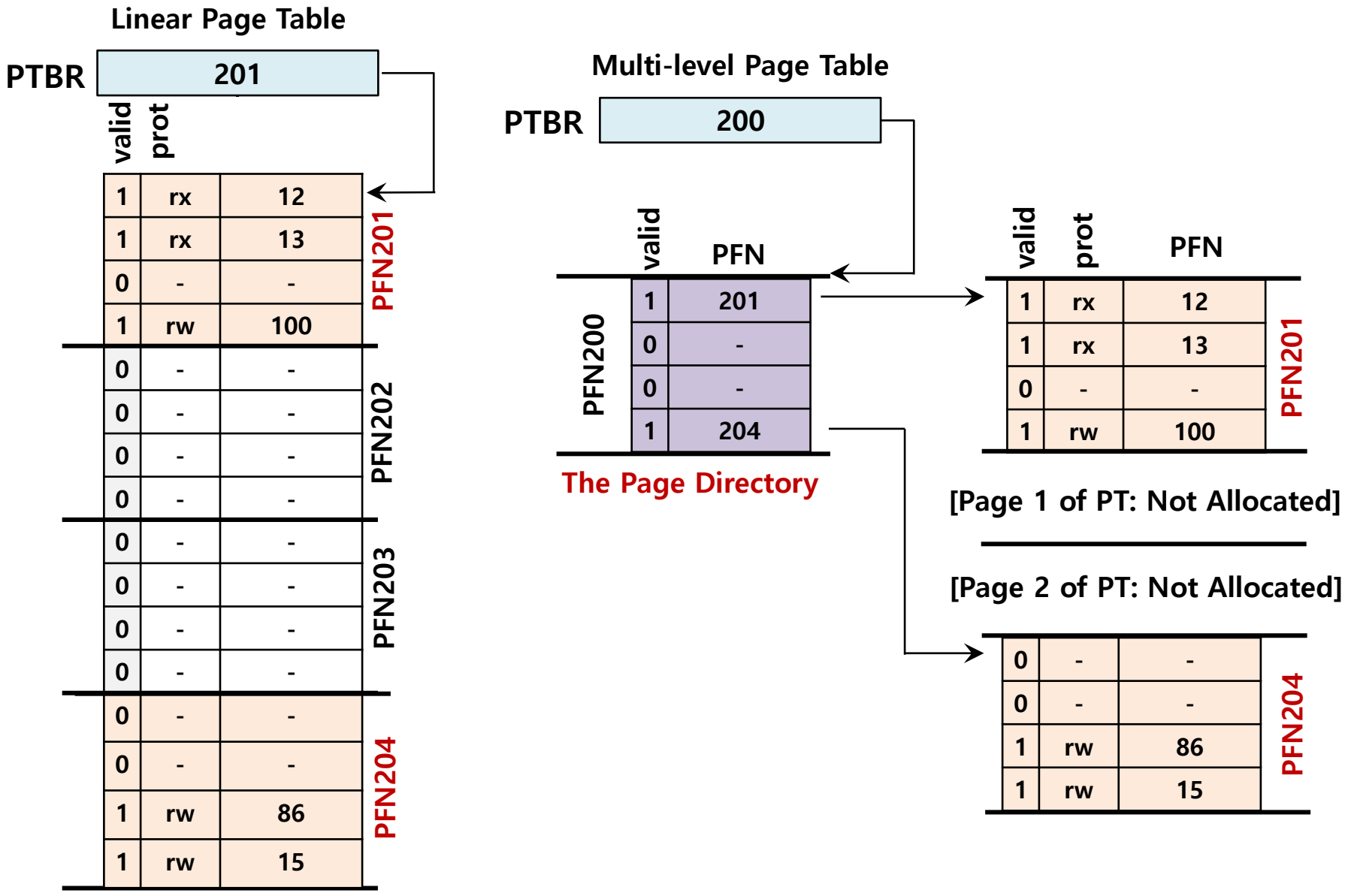
```
01:     SN = (VirtualAddress & SEG_MASK) >> SN_SHIFT
02:     VPN = (VirtualAddress & VPN_MASK) >> VPN_SHIFT
03:     AddressOfPTE = Base[SN] + (VPN * sizeof(PTE))
```

- Hybrid Approach is not without problems.
  - If we have a large but *sparsely-used heap*, we can still end up with a lot of page table waste.

Multi-level Page Tables

- How to get rid of all those invalid regions in the page table instead of keeping them all in memory?
- Turn the linear page table into something like a tree.
  - Chop up the page table into page-sized units.
  - If an entire page of page-table entries is invalid, don't allocate that page of the page table at all.
  - To track whether a page of the page table is valid, use a new structure, called page directory.
  - The page directory contains one entry per page of the page table.
  - It consists of a number of page directory entries (PDE).
  - PDE has a valid bit and page frame number (PFN).

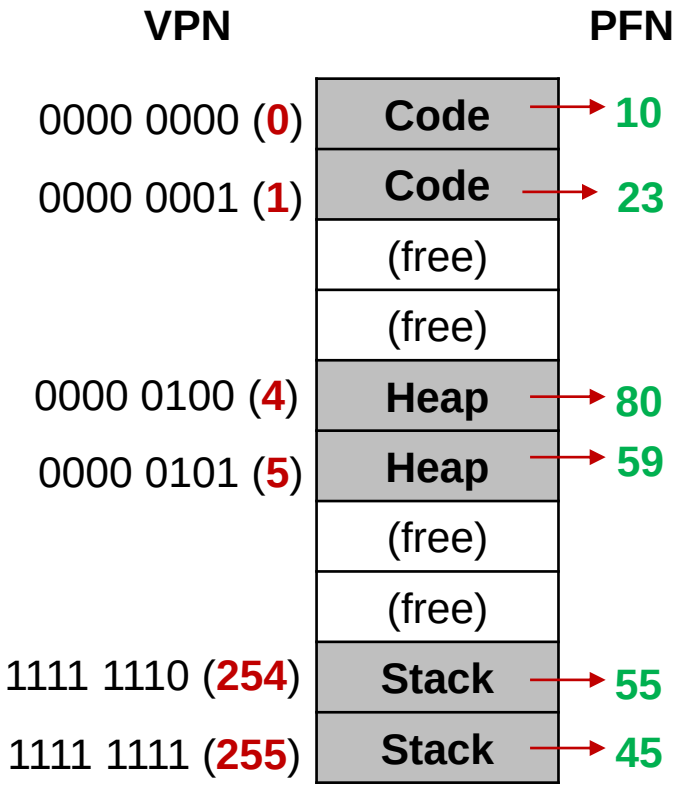
Example: Multi-level Page Tables



Multi-level Page Tables: Advantages and Disadvantages

- Advantage
  - Only allocates page-table space in proportion to the amount of address space you are using.
  - The OS can grab the next free page when it needs to allocate or grow a page table.
- Disadvantage
  - Multi-level table is a small example of a time-space trade-off.
    - On a TLB miss, **two loads from memory** will be required to get the right translation information from the page table.
    - On a TLB hit, performance is obviously identical.
  - Complexity.

Detailed Example

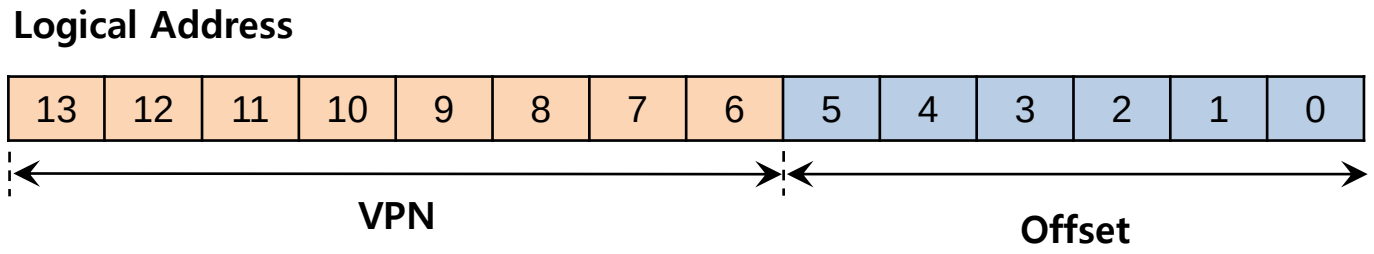


A 16-KB Address Space With 64-byte Pages

Flag	Detail
Address space	16 KB (2^14 Bytes)
Page size	64 bytes
Virtual address	14 bits
VPN	8 bits
Offset	6 bits
Page table entry	4 bytes

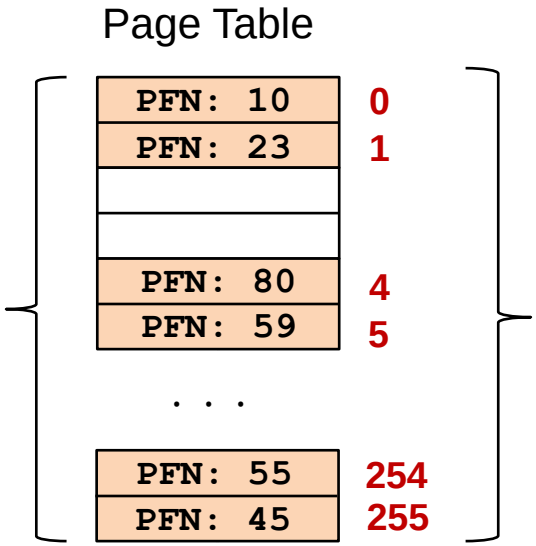
Example: Single-level Page Table

- Single-level paging



- Page table entries: 2^8 = 256 entries
- Page table size: 256 * 4 bytes = 1 KB

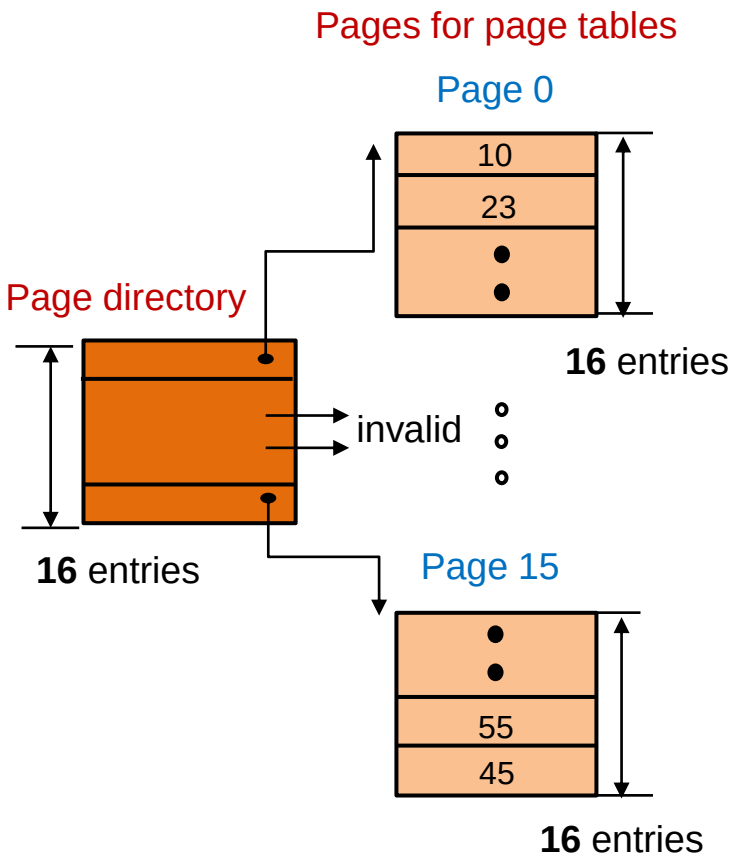
Number of page table entries
= 2^8
= **256 entries**



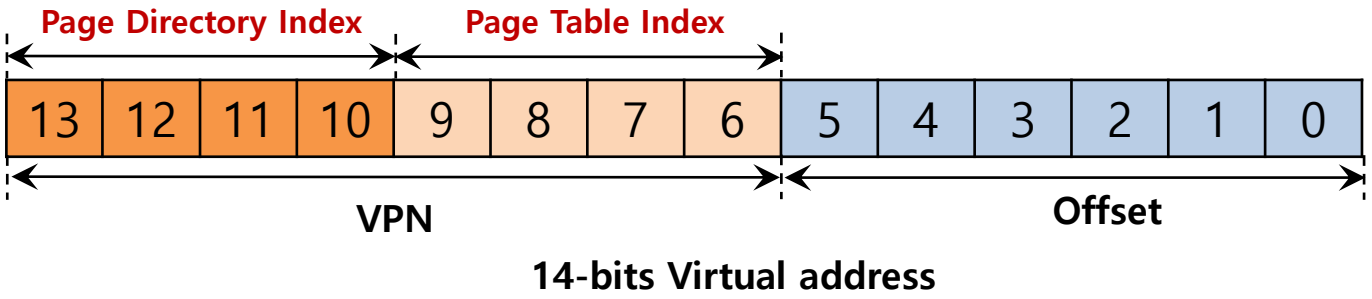
Number of pages
= (256 * 4) / 64
= **16 pages**

Example: Two-level Page Table

- Two-level paging
  - Chop up the page table into page-sized units and create a page directory.
  - If an entire page of page-table entries is invalid, don't allocate that page of the page table at all.
    - The access is invalid → raise an exception.
  - Page directory
    - Consists of 16 entries → one entry per page of the page table (i.e., PFN for page table).
    - Requires 16* 4 bytes = 64 bytes → it can fit in a page.
    - Requires 4 bits for page directory index → used to find the address of the page table entry.



Example: Two-level Page Table – Addressing



- Page directory index (`PDIndex`)
  - It is used to find the address of the page-directory entry (`PDE`) from the VPN.
  - `PDEAddr = PageDirBase + (PDIndex * sizeof(PDE))`
- Page table index (`PTIndex`)
  - It is used to index into the page table itself, giving us the address of our `PTE`.
  - `PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE))`
- `PhysAddr = (PTE.PFN << SHIFT) + offset`

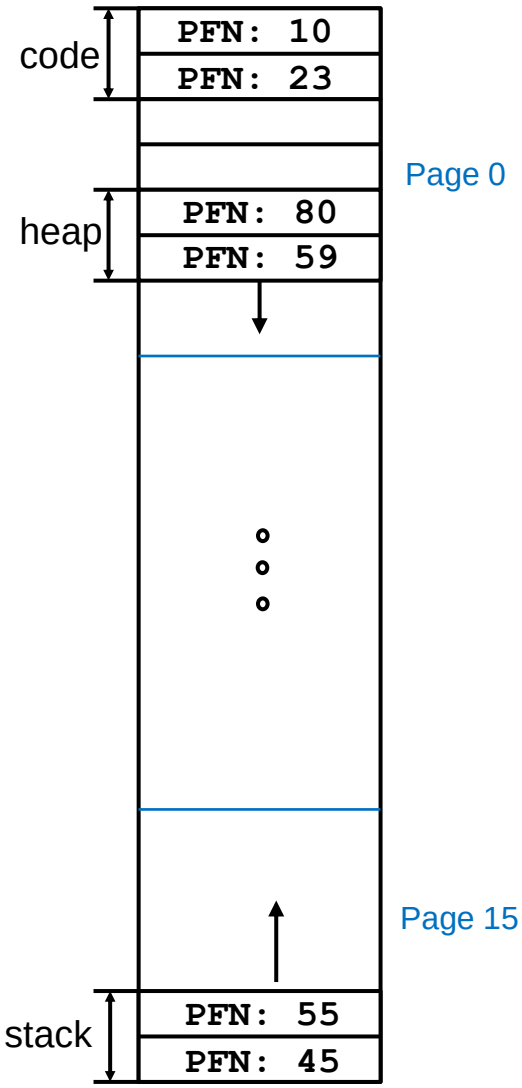
Examples: Two-level Page Table – Detailed Memory View

Page Directory		Page of PT (@PFN:100)			Page of PT (@PFN:101)		
PFN	valid?	PFN	valid	prot	PFN	valid	prot
100	1	10	1	r-x	—	0	—
—	0	23	1	r-x	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	80	1	rw-	—	0	—
—	0	59	1	rw-	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	—	0	—
—	0	—	0	—	55	1	rw-
101	1	—	0	—	45	1	rw-

Single level paging: **16** pages

→ Two level paging: **3** pages

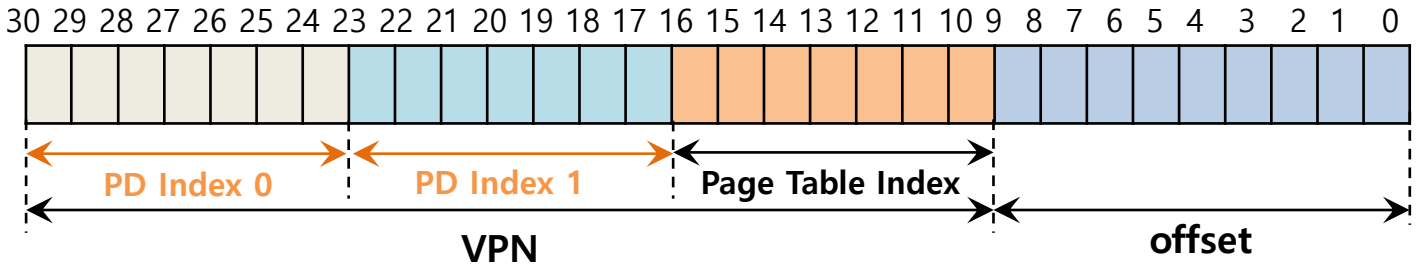
Page Table (single-level)



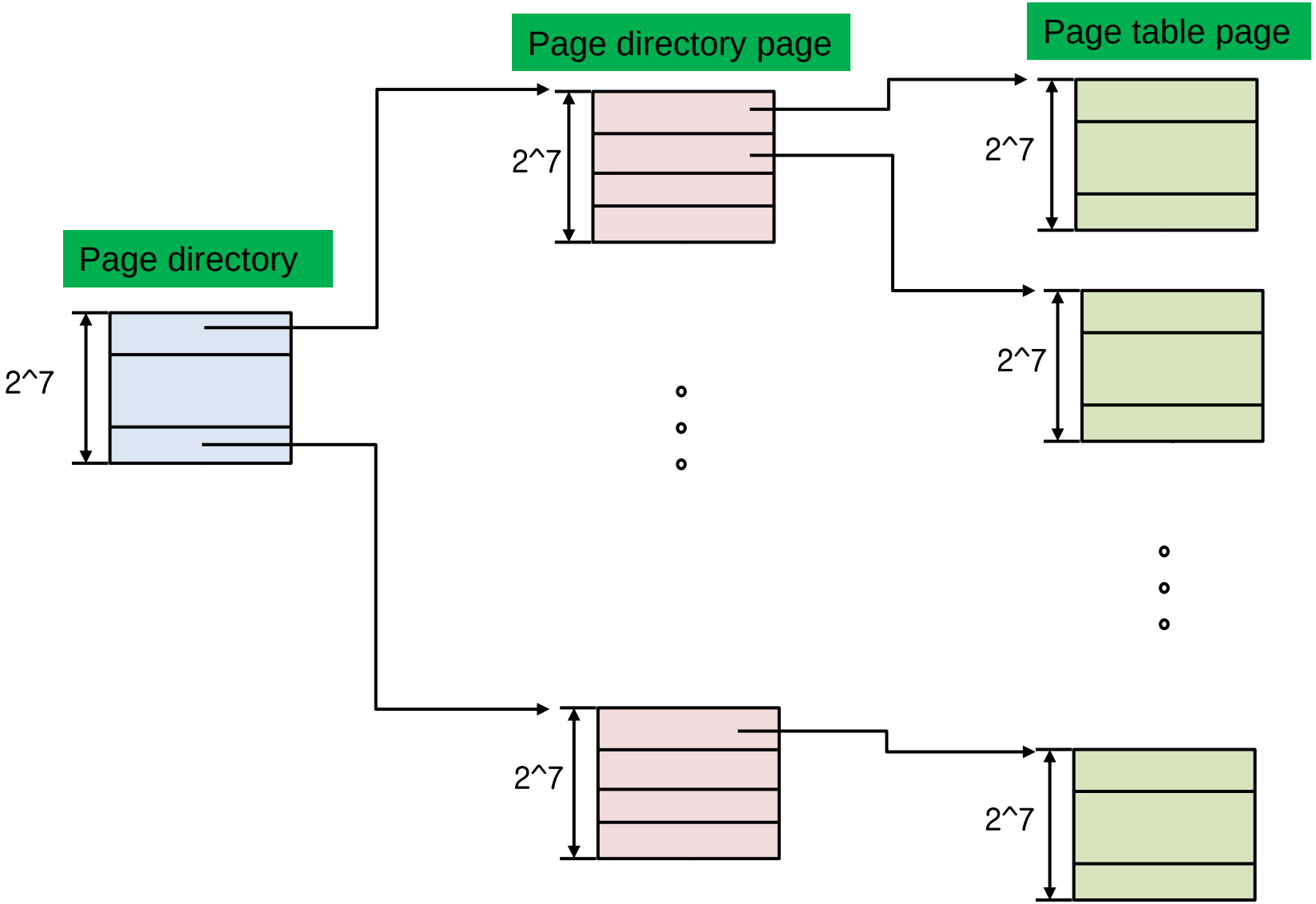
More Than Two Levels : Three-level Page Table

Flag	Detail
Virtual address	30 bits
Page size	512 bytes
VPN	21 bits
Offset	9 bits
Page table entry	4 bytes

- Make each piece of the page table and page directory fit within a page !
- How many levels are needed?
  - Number of PTE's in a single page: (512/4 = 128 (2^7)) → Page Table Index (**7 bits**).
  - Page Directory Index needs **14 bits** (2^14 entries) → Too big !
  - Page the page directory: 2^14 / 2^7 = 2^7 entries in a page directory → fit within a page ! (**3 levels** are needed)



Example: Three-level Page Table



Multi-level Page Table Control Flow – TLB Hit

```
01:     VPN = (VirtualAddress & VPN_MASK) >> SHIFT
02:     (Success,TlbEntry) = TLB_Lookup(VPN)
03:     if ( Success == True ) // TLB Hit
04:          if ( CanAccess(TlbEntry.ProtectBits) == True )
05:               Offset = VirtualAddress & OFFSET_MASK
06:               PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
07:               Register = AccessMemory(PhysAddr)
08:          else RaiseException(PROTECTION_FAULT);
09:     else
10:          // TLB Miss : perform the full multi-level lookup
```

Multi-level Page Table Control Flow – TLB Miss

```
11:     else // perform the full multi-level lookup
12:         PDIndex = (VPN & PD_MASK) >> PD_SHIFT
13:         PDEAddr = PDBR + (PDIndex * sizeof(PDE))
14:         PDE = AccessMemory(PDEAddr) // fetch PDE from PD
15:         if ( PDE.Valid == False ) RaiseException(SEGMENTATION_FAULT)
16:         else // PDE is Valid
17:               PTIndex = (VPN & PT_MASK) >> PT_SHIFT
18:               PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE))
19:               PTE = AccessMemory(PTEAddr) // fetch PTE from PT
20:               if ( PTE.Valid == False )
21:                     RaiseException(SEGMENTATION_FAULT)
22:               else if ( CanAccess(PTE.ProtectBits) == False )
23:                     RaiseException(PROTECTION_FAULT);
24:               else // now refresh TLB and restart
25:                     TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
26:                     RetryInstruction()
```

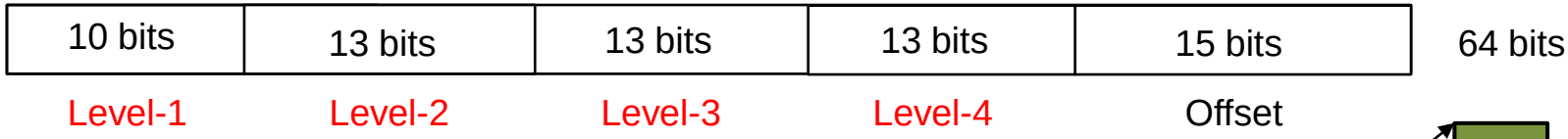
Exercise

Given 64 bits logical address, 32 KB page, and 4 B page table entry,

Q1. How many bits are needed for page offset? **(Ans)** 32 K = 2^{15} → 15 bits

Q2. How many page table entries are in each page? **(Ans)** 32 KB / 4 B = 2^{15} / 2^{2} = 2^{13} → 2^{13} entries

Q3. How many levels are needed if each page table fits within a page boundary? **(Ans)** 4 levels



Q4. What is total number of pages needed for this scheme?

(Ans) 1 + 2^{10} + (2^{10} * 2^{13}) + (2^{10} * 2^{13} * 2^{13})

Q5. How many frames can this page table address?

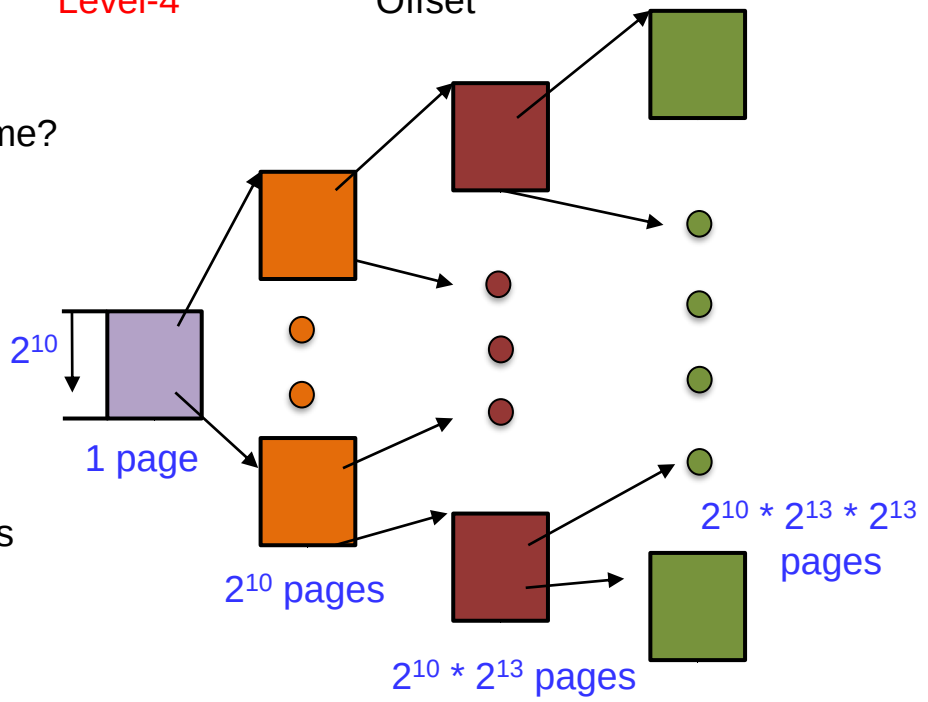
(Ans) 2^{10} * 2^{13} * 2^{13} * 2^{13} = 2^{49}

Q6. What is the size of all page tables? (bytes)

(Ans) (1 + 2^{10} + (2^{10} * 2^{13}) + (2^{10} * 2^{13} * 2^{13})) * 2^{15}

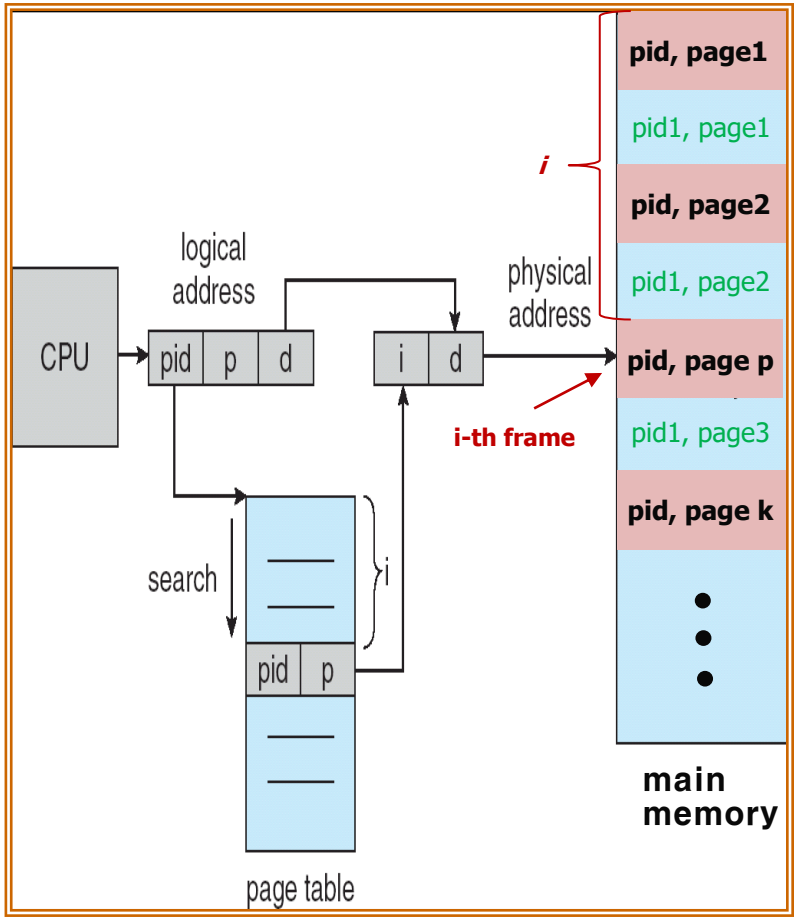
Q7. How many memory accesses are needed to address a certain page?

(Ans) 5



Inverted Page Table

- One entry for each real page of memory.
  - Each table entry consists of the virtual address of the page stored in that real memory location, with information (i.e., *pid or asid*) about the process that owns that page.
- Decreases memory needed to store each page table, but increases time needed to search the table when a page reference occurs.
- Use hash table to limit the search to one — or at most a few (e.g. 64-bit UltraSPARC and PowerPC).

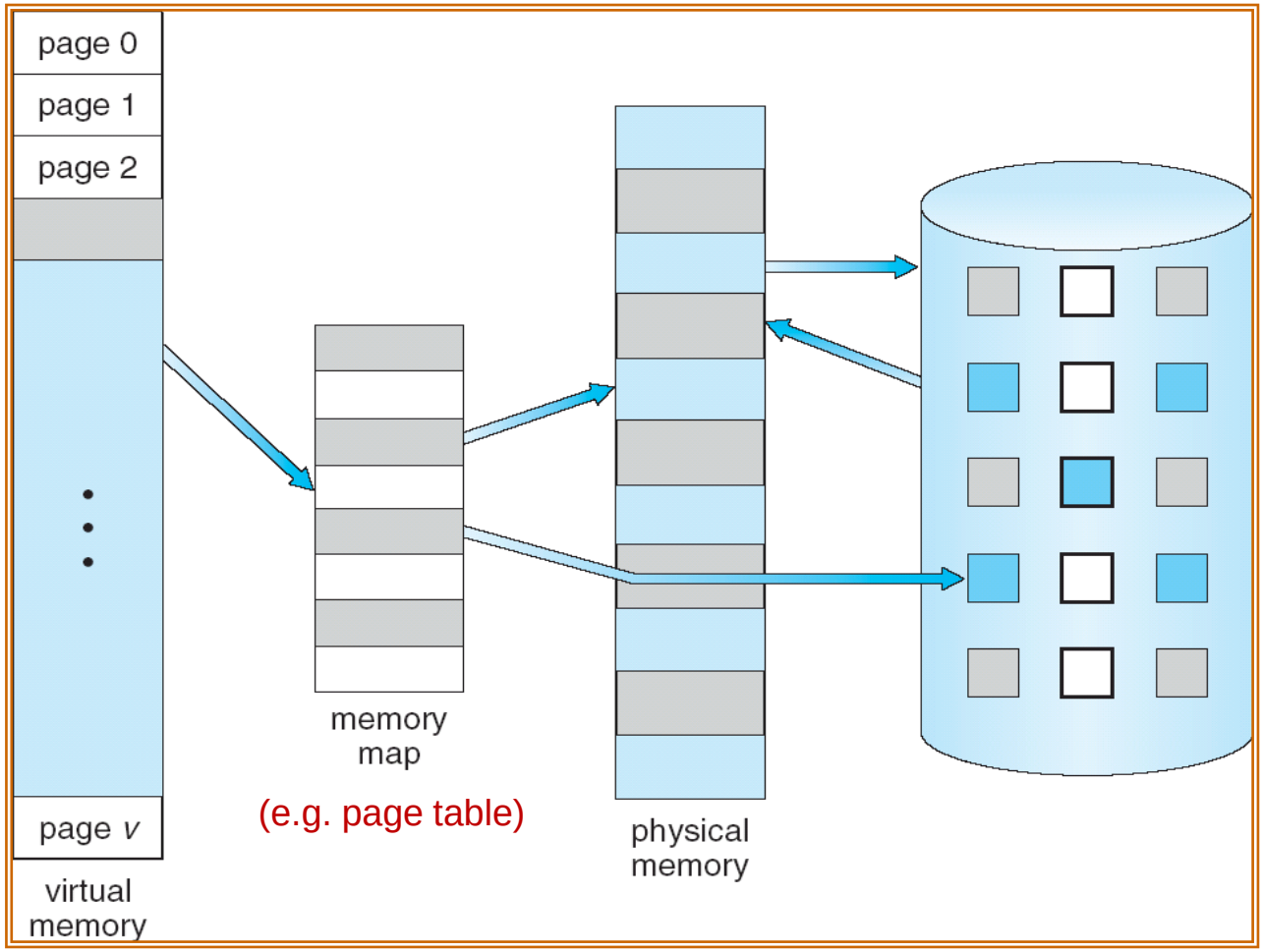


21. Beyond Physical Memory: Mechanisms

Beyond Physical Memory – Virtual Memory

- Program needs to be in memory to execute, but entire program rarely used.
  - Error code, unusual routines, large data structures
- Entire program is not needed at same time.
- The ability to execute a program that is only partially in memory would have many benefits.
- Virtual memory – separation of user logical (virtual) memory from physical memory.
  - Allows an extremely large virtual memory to be provided for programmers when only a smaller physical memory is available (e.g., virtual memory > physical memory).
  - Makes the programming much easier, because the programmer no longer needs to worry about the size of physical memory.
  - Only part of the program needs to be in memory for execution.

Virtual Memory > Physical Memory

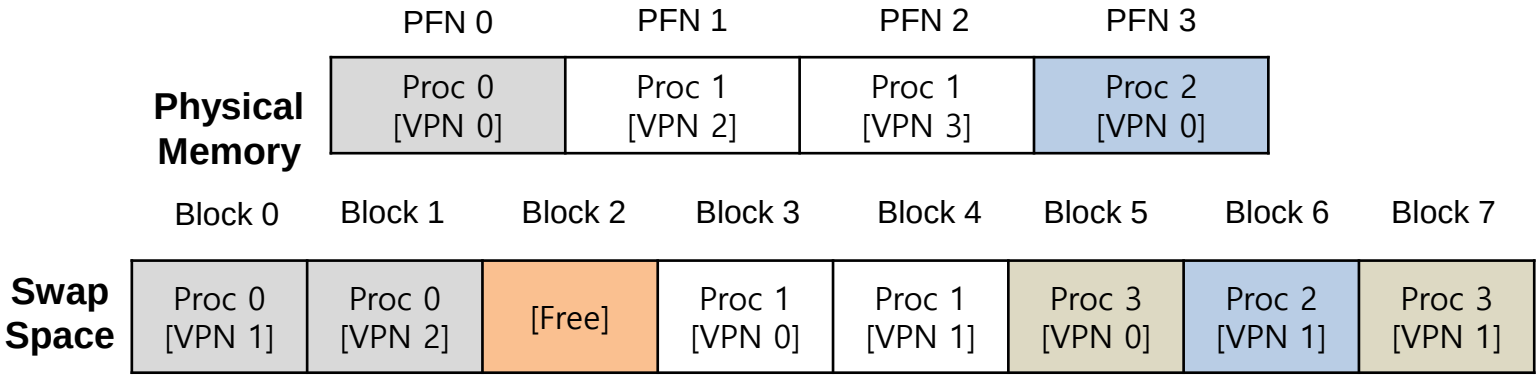


Demand Paging

- The concept of virtual memory can be implemented via demand paging.
  - Pages are loaded only when they are *demanded* during execution.
- In the extreme case, we can start executing a process with no pages in memory – pure demand paging.
  - Never bring a page into memory until it is required.
- Demand paging is similar to a paging system with swapping.
  - Swap a page rather than an entire process into memory only it is needed.
- Benefits of demand paging are:
  - Less I/O and less memory needed.
  - Faster response.
  - Accommodate more users.
- Some pages are in memory, while others are in secondary storage.
  - Need hardware support to distinguish between in-memory and not-in-memory pages.

Swap Space and Present Bit

- Reserve some space on the disk for moving pages back and forth → swap space.
- OS needs to manage the swap space, in page-sized unit.

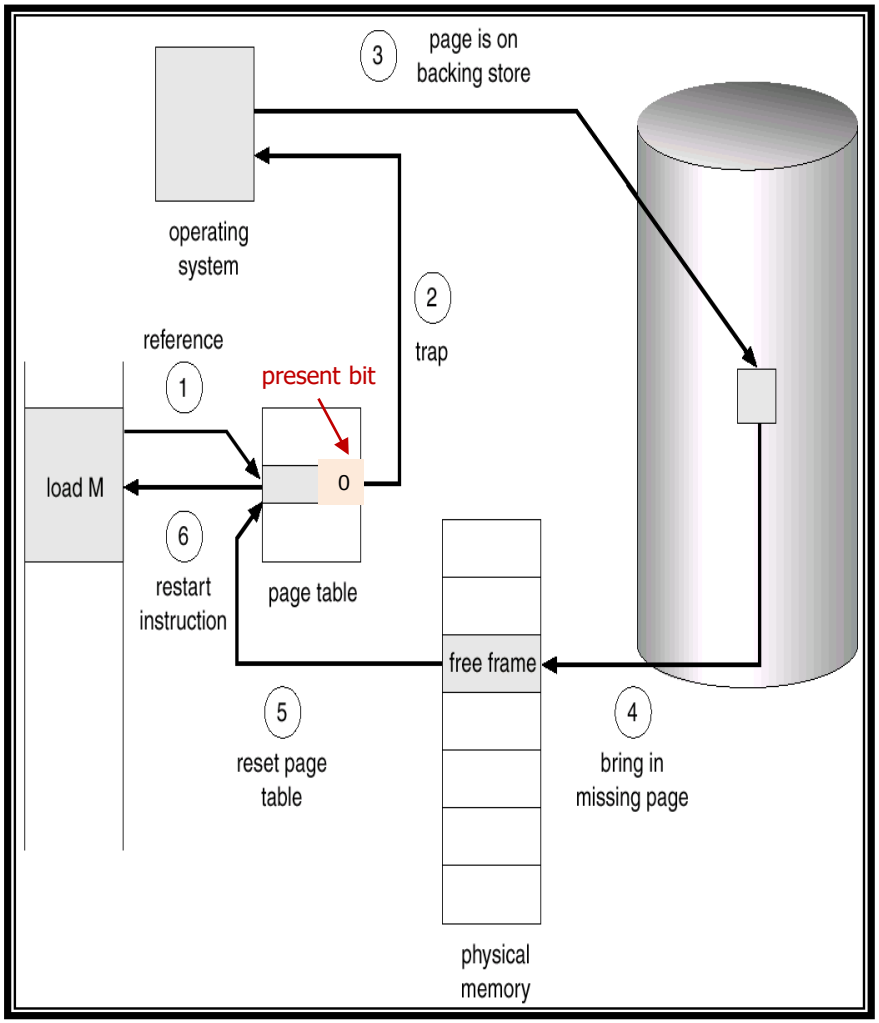


- When the hardware looks in the PTE, the present bit indicates that the page is not present in physical memory.

Present Bit	Meaning
1	The page is present in physical memory.
0	The page is not in memory but rather on disk.

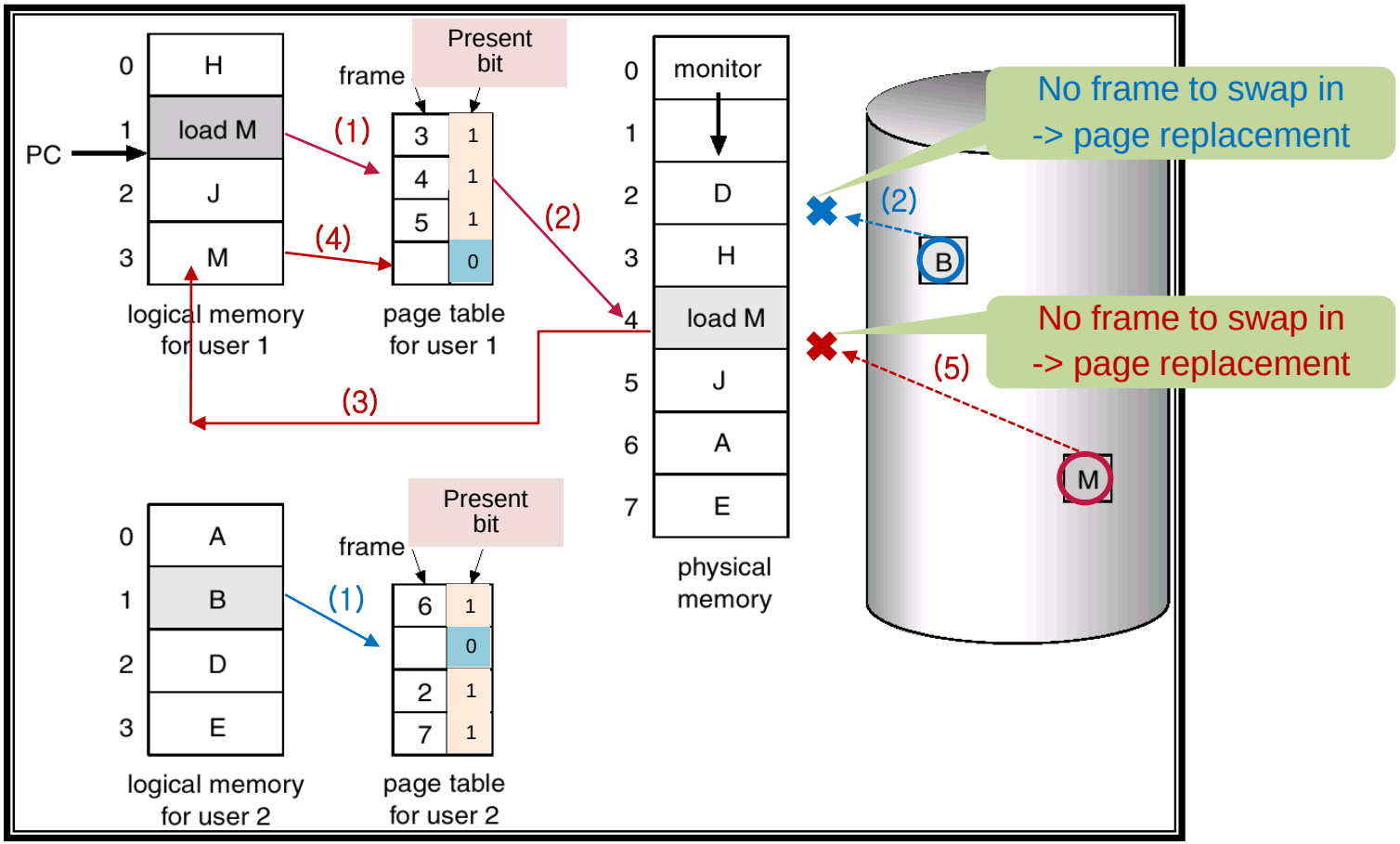
Page Fault

- If there is ever a reference to a page, ① first reference (not present in memory) will ② trap to OS → page fault.
- OS checks an internal table (or page table) to decide the location of the page in the disk ③.
- Find an empty frame.
- Read the desired page into the newly allocated frame ④.
- Reset page table ⑤ (set present bit).
- Restart instruction ⑥.



Example: Page Replacement

- In case of no free frame → page replacement.



When to Perform Replacement

- Lazy approach
  - OS waits until memory is entirely full, and only then replaces a page to make room for some other page.
  - This is unrealistic.
- Swap daemon or page daemon
  - There are fewer than LW (low watermark) pages available, a background thread that is responsible for freeing memory runs.
  - The thread evicts pages until there are HW (high watermark) pages available.

Page Fault Control Flow – Hardware

```
1:      VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2:      (Success, TlbEntry) = TLB_Lookup(VPN)
3:      if ( Success == True ) // TLB Hit
4:         if ( CanAccess(TlbEntry.ProtectBits) == True )
5:               Offset = VirtualAddress & OFFSET_MASK
6:               PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7:               Register = AccessMemory(PhysAddr)
8:         else RaiseException(PROTECTION_FAULT)
9:      else // TLB Miss
10:         PTEAddr = PTBR + (VPN * sizeof(PTE))
11:        PTE = AccessMemory(PTEAddr)
12:        if ( PTE.Valid == False )
13:              RaiseException(SEGMENTATION_FAULT)
14:        else
15:           if ( CanAccess(PTE.ProtectBits) == False )
16:              RaiseException(PROTECTION_FAULT)
17:           else if ( PTE.Present == True )
18:              // assuming hardware-managed TLB
19:              TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
20:              RetryInstruction()
21:           else if ( PTE.Present == False )
22:              RaiseException(PAGE_FAULT)
```

Page Fault Control Flow – Page Fault Handler

```
1:      PFN = FindFreePhysicalPage()
2:      if ( PFN == -1 )              // no free page found
3:              PFN = EvictPage()     // run replacement algorithm
4:      DiskRead(PTE.DiskAddr, pfn)   // sleep (waiting for I/O)
5:      PTE.present = True            // update page table with present
6:      PTE.PFN = PFN                 // bit and translation (PFN)
7:      RetryInstruction()            // retry instruction
```

- The OS must find a physical frame for the page to bring in.
- If there is no such page, waiting for the replacement algorithm to run and kick some pages out of memory.

Considering Dirty Pages

- The page table also includes a modified bit (also known as dirty bit)
  - Page has been **modified** and is thus **dirty**, it must be written back to disk to evict it.
  - Page has not been modified → the eviction is free.

Performance of Demand Paging

- Demand paging can significantly affect the performance of a computer system.

- Page fault rate: $0 \leq p \leq 1.0$
  - If $p = 0$, no page faults.
  - If $p = 1$, every reference generates a page fault.

- Effective access time (EAT)
  - EAT = (1 – p) x memory access time + p x page fault time
  - Page fault time consists of:
    - Page fault overhead
    - Time for swapping page in
    - Time for swapping page out, if necessary (swap out modified page)
    - Restart overhead (includes memory access time after page fault)

Example

- **Example 1**
  - Memory access time = 1 *microsec*
  - 50% of the pages that are being replaced have been modified and therefore need to be swapped out.
  - Swap page time = 10 *millisec* = 10,000 *microsec*
  - EAT = (1 – *p*) x 1 + *p* x (15,000) ~ 1 + 15,000 *p* (in *microsec*)
- **Example 2**
  - Memory access time = 200 *nanosec*
  - Average page-fault service time = 8 *millisec*
  - EAT = (1 – *p*) x 200 + *p* x 8 *millisec*

    = (1 – *p*) x 200 + *p* x 8,000,000

    = 200 + *p* x 7,999,800
  - If one access out of 1,000 causes a page fault, then EAT = 8.2 *microsec*
  - This is a slowdown by a factor of 40 !!

22. Beyond Physical Memory: Policies

Page Replacement Algorithms

- Want lowest page-fault rate.
- Evaluate algorithm by running it on a particular string of memory references (**reference string**) and computing the number of page faults on that string.
- For example, if we trace a particular process, we might record the following address sequence:

  0100, 0432, 0101, 0612, 0102, 0103, 0104, 0101, 0611,

  0102, 0103, 0104, 0101, 0610, 0102, 0103, 0104, 0101, 0609, 0102, 0105

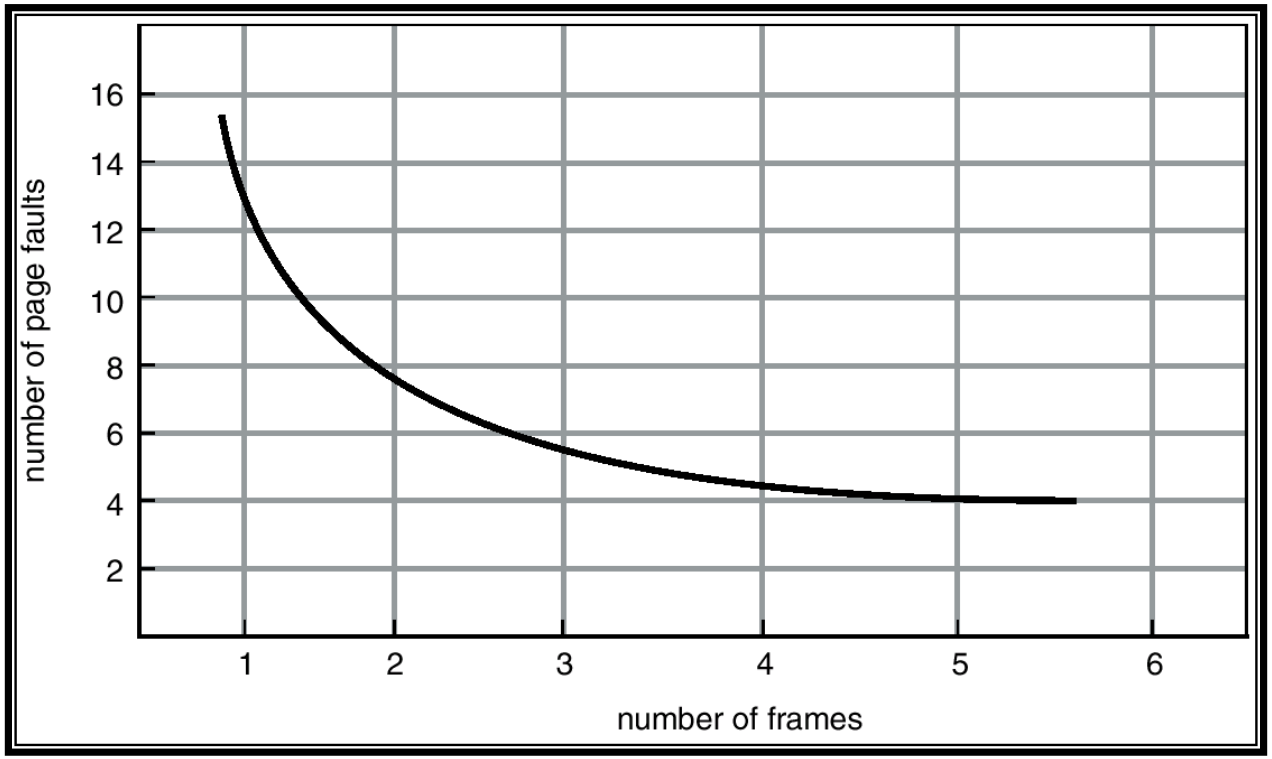
- At 100 bytes per page, the reference string is :

  1, 4, 1, 6, 1, 6, 1, 6, 1, 6, 1

- Immediate reference after the 1st reference will not fault !

Page Faults vs Number of Frames

- In general, as the number of frames **increases**, the number of page faults **decreases** and drops to some minimal level.



A Simple Policy: FIFO (First In First Out)

- Pages were placed in a queue when they enter the system.
- When a replacement occurs, the first-in page is evicted.
  - It is simple to implement but can't determine the importance of frames.

Reference String

0 1 2 0 1 3 0 3 1 2 1

7 page faults !

Even though page 0 had been accessed a number of times, FIFO still kicks it out.

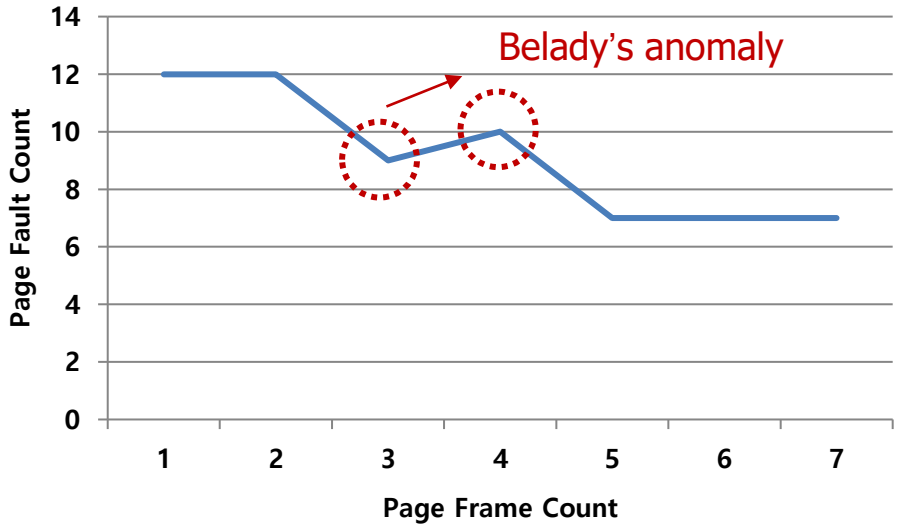
Access	Hit/Miss?	Evict	Memory State (3 frames)
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	1,2,3
0	Miss	1	2,3,0
3	Hit		2,3,0
1	Miss	2	3,0,1
2	Miss	3	0,1,2
1	Hit		0,1,2

Belady's Anomaly

- We would expect the page fault rate to decrease when we have more frames. But in this case, with FIFO, it gets worse.

Reference String

1 2 3 4 1 2 5 1 2 3 4 5



Another Simple Policy: Random

- Picks a random page to replace under memory pressure.
  - It doesn't really try to be too intelligent in picking which frames to evict.
  - Random does depends entirely upon how lucky random gets in its choice.

Reference String

0 1 2 0 1 3 0 3 1 2 1

6 page faults !

The performance of random is sometimes good. It depends on the workload !

Access	Hit/Miss?	Evict	Memory State (3 frames)
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	1,2,3
0	Miss	1	2,3,0
3	Hit		2,3,0
1	Miss	3	2,0,1
2	Hit		2,0,1
1	Hit		2,0,1

The Optimal (OPT or MIN) Replacement Policy

- Lead to the fewest number of misses overall.
  - Replace the page that will be accessed furthest in the future.
  - Result in the fewest-possible page faults (no Belady's anomaly).
- Serve only as a comparison point, to know how close we are perfect.

Reference String

0 1 2 0 1 3 0 3 1 2 1

5 page faults !

How do you know this ?
Requires future knowledge !

→ Can we use history as an approximation of the future ?

Access	Hit/Miss?	Evict	Memory State (3 frames)
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	2	0,1,3
0	Hit		0,1,3
3	Hit		0,1,3
1	Hit		0,1,3
2	Miss	3	0,1,2
1	Hit		0,1,2

Using History: LRU (Least Recently Used)

- Replace the page that has not been used for the longest period of time.
- An approximation to the optimal algorithm. Use the recent past as an approximation of the near future.

Reference String

0 1 2 0 1 3 0 3 1 2 1

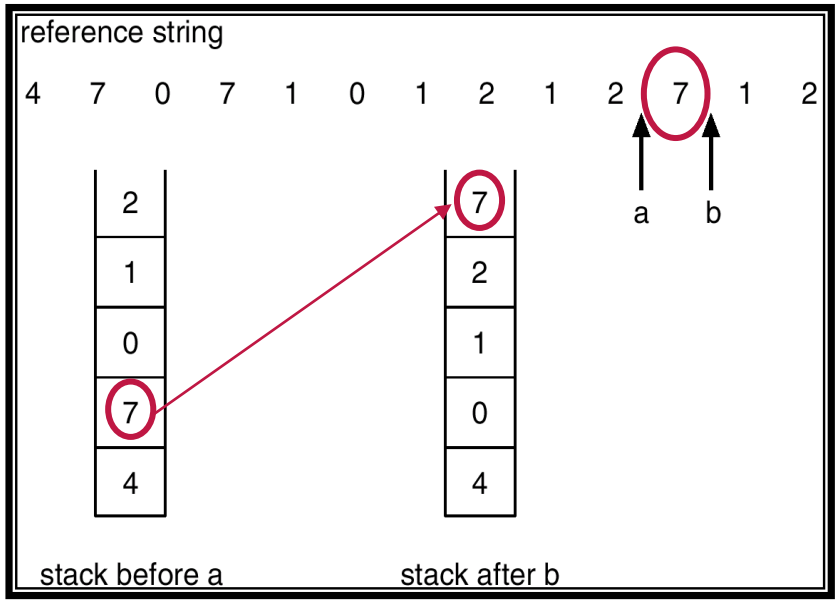
5 page faults !

- The performance is considered to be good.
- The problem is how to implement LRU replacement; may need hardware support.

Access	Hit/Miss?	Evict	Memory State (3 frames)
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3
1	Hit		0,3,1
2	Miss	0	3,1,2
1	Hit		3,2,1

Implementing LRU Algorithm

- Counter implementation
  - Every page table entry has a counter. Every time a page is referenced through this entry, copy the (logical) clock into the counter. The clock is incremented for every memory reference.
  - When a page needs to be changed, look at the counters to determine which page is to be changed.
- Stack implementation
  - Keep a stack of page numbers in a doubly linked list form.
  - Whenever a page is referenced, move it to the top; the top of the stack is always the most recently used page and the bottom is the LRU page.
  - Each update is a little more expensive. No search for a replacement.



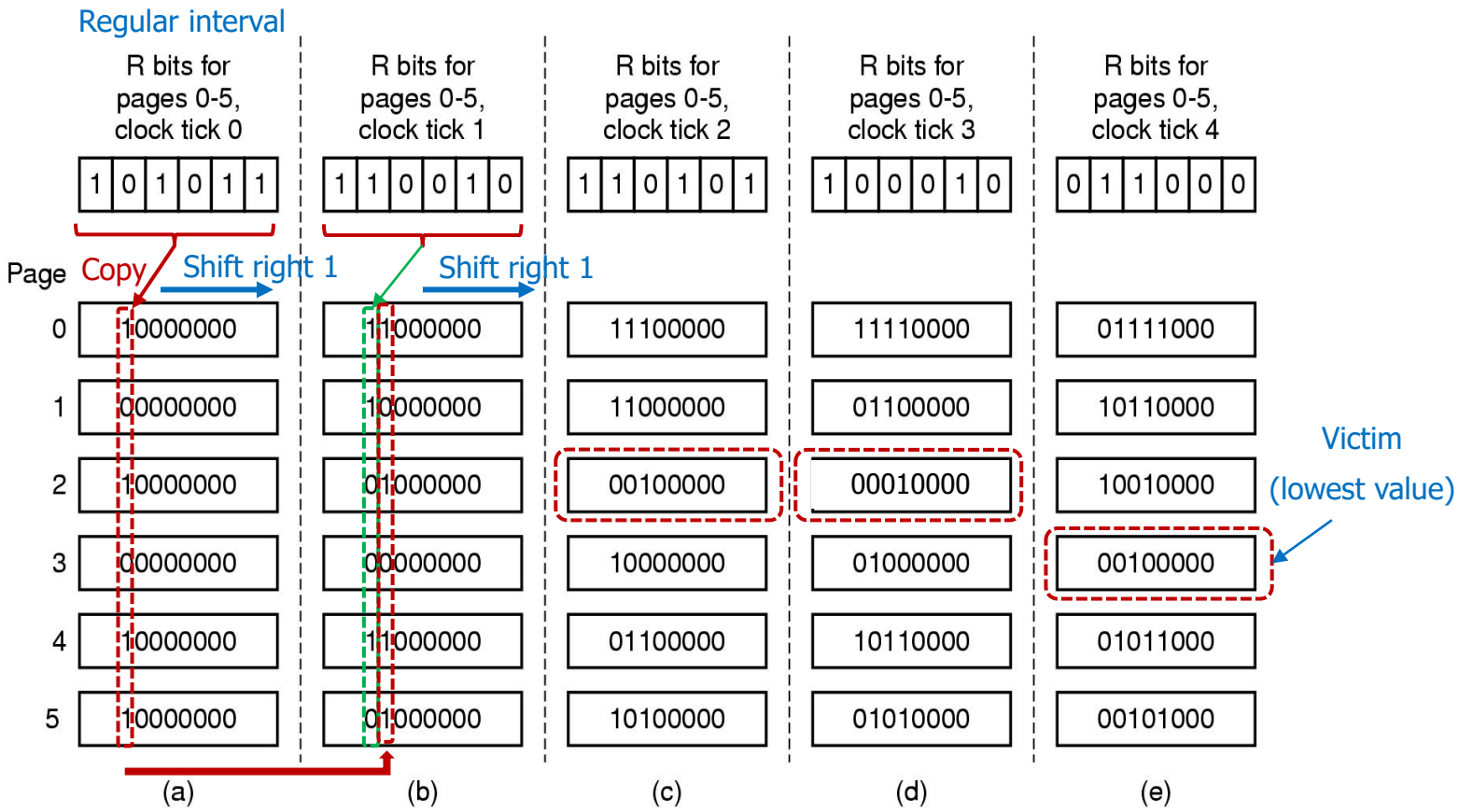
LRU Approximation Algorithm: Reference Bit / Byte Algorithm

- Reference Bit Algorithm
  - With each page, associate a reference bit (**set to 0** when loaded in memory).
  - When a page is referenced, set the reference bit to 1 (by hardware).
  - Replace the one which is 0 (if one exists). We do not know the order, but we know which pages were used and which were not used.

- Additional Reference Bits Algorithm (Reference Byte Algorithm)
  - Use multiple bits. Record the reference bits at regular intervals.
  - Example:
    - Keep an 8-bit byte for each page table in memory.
    - At regular interval, shift the reference bit into the **high order bit** of its 8-bit byte, shifting the other bits **right** 1 bit, discarding the low order bit.
    - These 8-bit shift registers contain the history of page use for the last eight time periods.
    - When page fault occurs, we interpret these 8-bit bytes as unsigned integers and the page with the **lowest number** is the LRU page and it can be replaced.

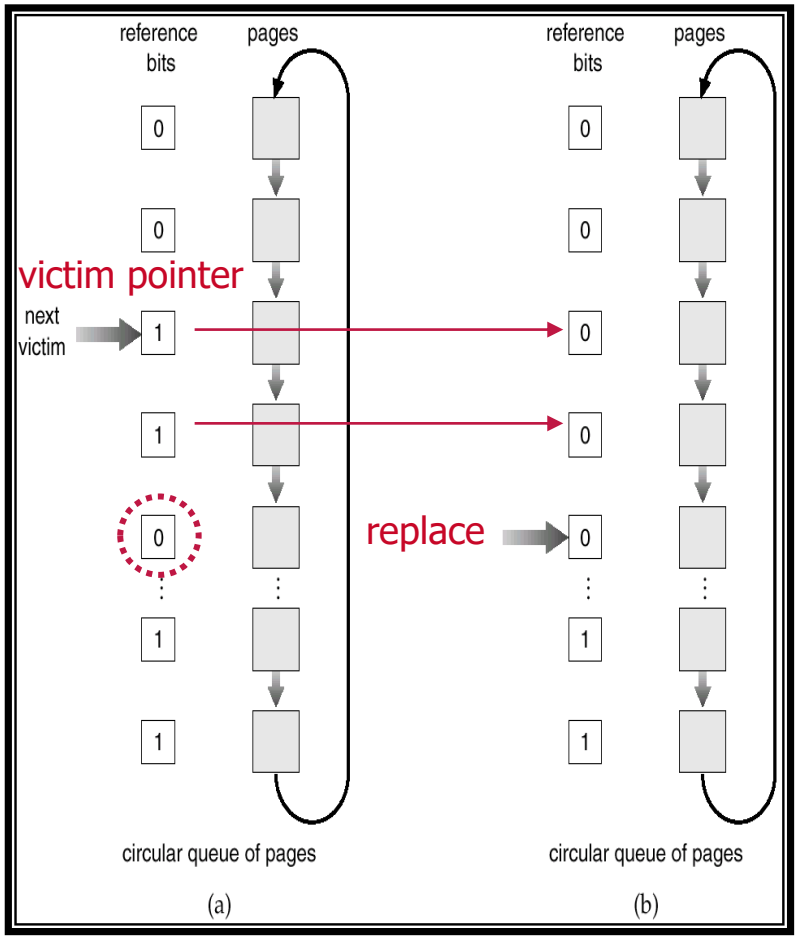
Example: Reference Byte Algorithm

- Additional Reference Bits Algorithm (Example)



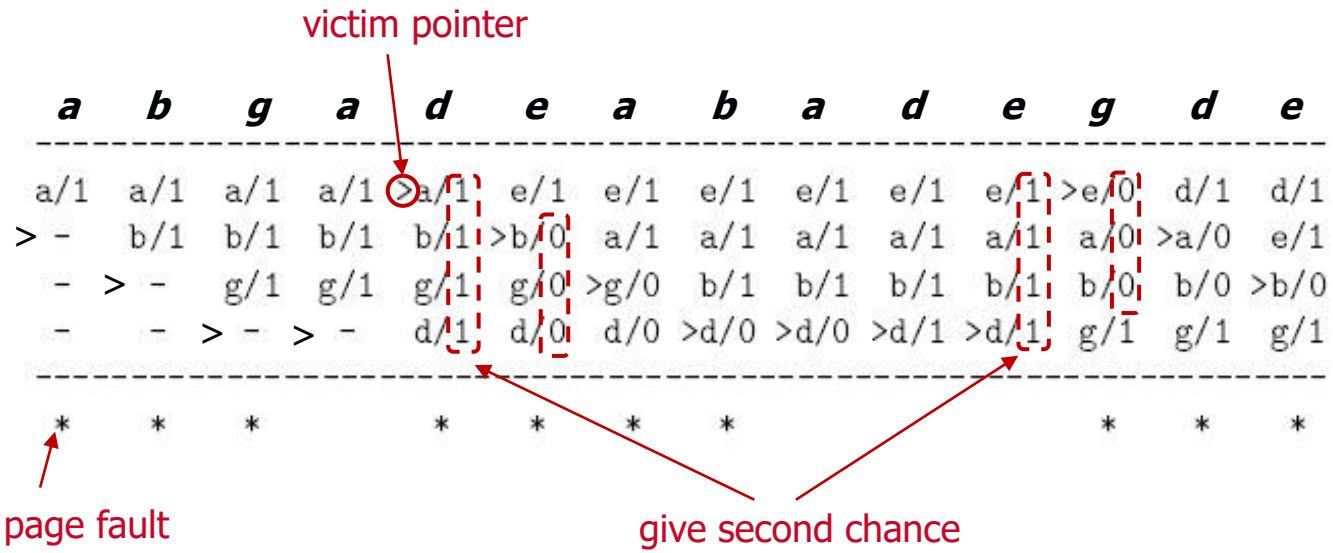
FIFO Approximation Algorithm: Second Chance Algorithm

- Second Chance (Clock) Algorithm
  - Keep the **circular FIFO queue** of pages **loaded in memory**.
    - Reference bit is **set to 1** when page is initially loaded (this bit can be set to 0).
    - Victim pointer points to the **oldest** page.
  - When we need to replace a page in page fault handler, we inspect the reference bit of page in victim pointer.
    - If it is 0, replace and advance pointer.
    - If it is 1, then
      - Set reference bit to 0 (**give second chance**) to see if it will be referenced again. → Leave the page in memory.
      - Advance pointer and inspect next page in **clock order.**
    - If all pages have reference bits 1, this algorithm is the same as FIFO algorithm.



Example: Second Chance Algorithm

- Second Chance (Clock) Algorithm (Example)
  - Reference string : ***a b g a d e a b a d e g d e***
  - Number of frames = 4



- Number of page faults = 10

Enhanced Second Chance Algorithm

Enhanced Second Chance Algorithm

- Enhance the second chance algorithm and consider both reference and modify bits.
- Four classes
  1. (0, 0): neither recently used nor modified – best candidate !
  2. (0, 1): not recently used but modified.
  3. (1, 0): recently used but clean.
  4. (1, 1): recently used and modified.
- When page replacement is called for, examine the class to which that page belongs, and replace the first page encountered in the lowest nonempty class.
- May have to scan the circular queue several times.

Counting-Based Algorithms

- Keep a counter of the number of references that have been made to each page.

- Least Frequently used (LFU) Algorithm
  - Replaces the page with smallest count.
  - Based on the assumption that an actively used page should have a large reference count.
- Most Frequently Used (MFU) Algorithm
  - Based on the argument that the page with the smallest count was probably just brought in and has yet to be used.

Allocation of Frames

- How do we allocate the fixed amount of free memory among processes ?
  - Cannot allocate more than the total number of available frames.
  - Must allocate at least a minimum number of frames.
- Each process needs minimum number of pages.
  - Defined by the instruction set architecture.
  - Need enough frames to hold all the different pages that any single instruction can reference.
    - Example 1: 1-level indirect addressing (e.g., load 10) needs 3 frames.
    - Example 2: PDP-11 needs 6 frames to handle *MV from, to* instruction:
      - instruction is 6 bytes (larger than a word) and might span 2 pages
      - 2 pages to handle *from*
      - 2 pages to handle *to*
- Two major allocation schemes exist.
  - Equal allocation / Proportional allocation

Frame Allocation Algorithm

- Equal allocation – e.g., if 100 frames and 5 processes, give each 20 pages.
- Proportional allocation – Allocate according to the size of process.

s_i = size of process p_i

$S = \sum s_i$

m = total number of frames

a_i = allocation for $p_i = \frac{s_i}{S} \times m$



$m = 64$

$s_1 = 10$

$s_2 = 127$

$a_1 = \frac{10}{137} \text{ x } 64 \approx 5$

$a_2 = \frac{127}{137} \text{ x } 64 \approx 59$

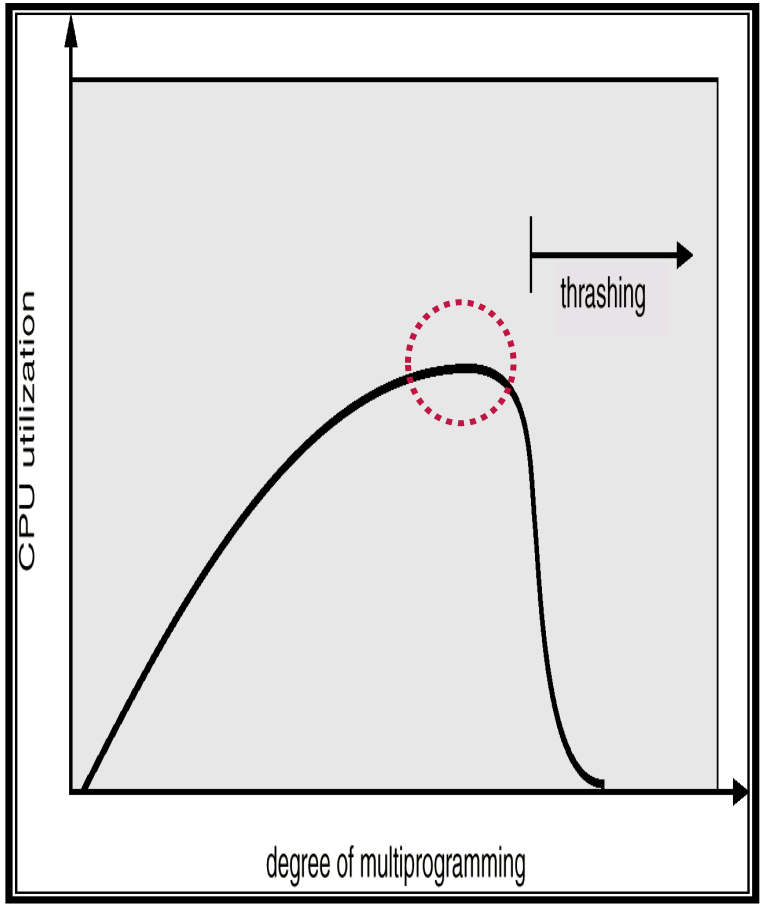
- The allocation to each process may vary according to the multiprogramming degree (how many processes are in the memory).
- Priority allocation – Use a proportional allocation scheme using priorities rather than size.

Global vs. Local Replacement

- If process P_i generates a page fault, the page fault handler
  - Select a free frame from **all** frames for replacement – **Global** replacement.
  - Select a free frame from one of **its** frames for replacement – **Local** replacement.
- Global replacement
  - Process selects a replacement frame from the set of all frames; one process can take a frame from another.
  - Cannot control its own page fault rate; the same process may perform quite differently.
  - Generally, global replacement results in greater throughput. More common method.
- Local replacement
  - Each process selects a replacement frame from only its own set of allocated frames.
  - Prevent a process from making less-used pages of memory available to other processes.

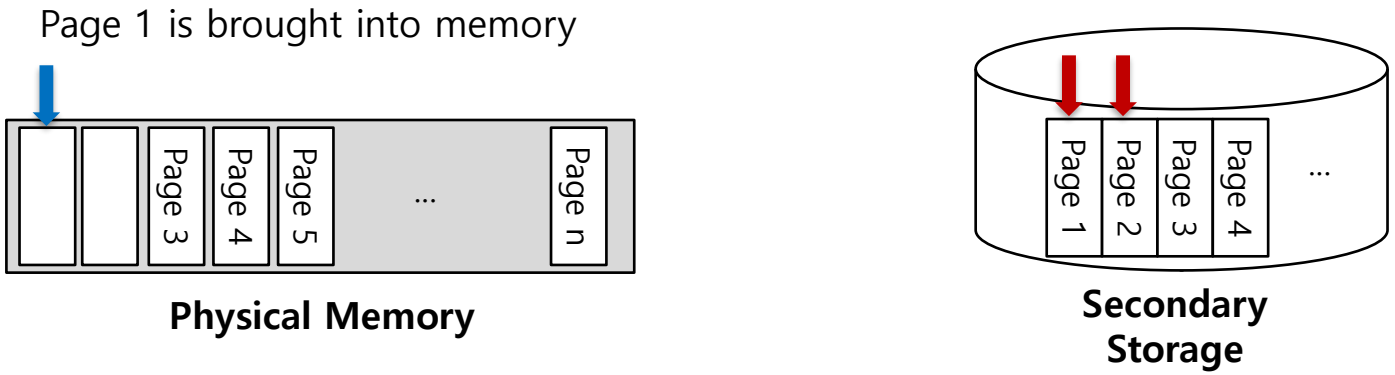
What is Thrashing?

- What if a process does not have enough frames (even less than the minimum) ?
  - Page-fault rate is very high (assume global replacement).
- This leads to:
  - Low CPU utilization.
  - Increases the degree of multiprogramming continuously by adding another process to the system.
  - Cause severe page faults.
- Thrashing → a process is spending more time swapping pages in and out than executing.
- Local replacement can reduce the effects but not entirely solve.



Prefetching

- The OS guesses that a page is about to be used, and thus bring it in ahead of time.



Page 2 likely soon be accessed and should be brought into memory too !

Clustering and Grouping

- Collect a number of pending writes together in memory and write them to disk in one write.
  - Perform a **single large write** more efficiently than **many small ones**.

